全国大学生嵌入式芯片与系统设计竞赛

FPGA 创新设计赛道　中科亿海微　选题一：基于以太网 MAC 总线实现 AFDX

AFDX 端系统

FPGA 设计文档

方案与具体设计

队伍名称：希望有奖

团队编号：49913

参赛成员：温海林、陈建林、郭俊杰

指导教师：雷登云

目标器件：EQ6HL130，CSG484_H 封装

开发环境：eLinx 3.0.7，ModelSim SE-64 2020.4

2026 年 11 月

摘要

本设计针对全国大学生嵌入式芯片与系统设计竞赛“基于以太网 MAC 总线实现 AFDX”赛题要求，基于中科亿海微 EQ6HL130 FPGA 设计并实现了双网冗余的 ARINC 664 Part 7 航电端系统。端系统逻辑由单一 125 MHz（8 ns 周期）主时钟驱动，以 Verilog HDL 实现了 128 条发送虚拟链路（VL）与 4096 条接收虚拟链路的线速硬件处理，支持 1–8192 B 应用层消息的 IPv4 自动分片发送与 16 路并发顺序硬件分片重组，并集成 ARINC 615A 软件数据加载（Data Loading）、SNMPv1/v2c 网络管理与 SNTPv4/RTC 时钟同步服务。

针对大规模虚拟链路状态维护、长报文硬件分片重组与 125 MHz 物理时序收敛的要求，系统采用“顶层流式边界封装 + 片上块 RAM 表格化管理 + 单主控状态机”架构，全芯片划分为 34 个 RTL 文件（42 个设计单元）。顶层 `AFDX_top`（172 引脚）封装带 2 字节大端 VLID 前缀的应用流式接口（`tx_data`/`rx_data`），内部例化端系统核心 `afdx_es`；接收映射表采用 32-Bank 双写分片与三级或树读回流水，分片重组引擎采用双存储体（2-Bank × 8 槽位）16 路并发上下文与 4 通道 128 KiB 载荷缓存，并通过写数据与写使能 LUT 门控消除短路径保持时间违例；管理面协议引擎（ASN.1/BER 与 TFTP）引入 2 分频局部时钟使能（CE_DIV = 2），将复杂状态转移约束在 16 ns 多周期路径中；报文线速收发、RAM 访问与配置总线维持 125 MHz 单周期运行。

经系统级仿真验证与官方工具链物理签核，设计达成以下指标：

1. 吞吐率与确定性时延：按单端 63 条发送 / 63 条接收 VL 双向全双工满载规格（BAG = 1 ms，$L_{\max}$ = 1471 B，双网冗余，线路占用率 77.5%），单向去重交付有效载荷吞吐率达 741.384 Mbps（双向合计 1482.768 Mbps）；在 77 VL 与 81 VL 压力工况下，单向吞吐分别达 906.136 Mbps 与 953.208 Mbps，多 VL 混合变长高负载（`var_high`）双向合计达 1809.504 Mbps；63 VL 结合 1472 B（2 分片）、3000 B（3 分片）与 8192 B（6 分片）的混合满载工况下，双向 12,206 条消息 100% 无损重组交付。在顶层流式接口口径下，63 VL 周期稳态最大端到端时延为 80.336 µs，旋转突发（B2）最大时延为 1175.432 µs；在核心分段口径下，单帧硬件流水时延典型值为 24.424 µs、满载最大为 24.432 µs（发送段 L_{tx} 最大 0.312 µs、至首前导码 0.256 µs，接收段 L_{rx} 首字节 12.360 µs、末字节 24.120 µs），满足赛题 1–2 ms 的时延要求。

2. 传输抖动控制：稳态发送间隔抖动实测为 0 µs。在顶层流式接口下，63 VL 满载周期激励同 VL 端到端时延极差抖动在 1471 B 满载帧下为 0.016 µs、跨 1–1471 B 全包长最大为 3.360 µs，旋转三元组背靠背突发（B2）下为 96.752 µs（< 100 µs）；77 VL 定长满载同 VL 极差为 0.008 µs（1 拍，8 ns），多 VL 变长循环工况下核心 `afdx_es` R97 口径为 9.992 µs、顶层 `AFDX_top` R119 口径为 19.544 µs。
3. 硬件资源与物理时序收敛：逻辑综合消耗查找表（LUT）62274 个（占比 45.74%）、触发器（FF）35597 个（占比 26.14%）、逻辑簇（CLB）10230 个（占比 75.13%）、嵌入式存储块 814 个 M4K/EBR（占比 67.83%）、引脚 172 个（占比 50.74%）。在 eLinx 3.0.7 官方流程（`tools/build_1030.py` + `PRES 1.30` + 11044 个物理簇坐标锁定）下，计入 0.30 ns（300 ps）建立时间时钟不确定度后，布局布线后全芯片 96142 个时序端点实现 0 违例收敛，净建立时间裕量（Worst Setup Slack）为 $+0.029$ ns（未扣不确定度的原始物理建立裕量为 $+0.329$ ns）、最差保持时间裕量（Worst Hold Slack）为 $+0.604$ ns，等效工作频率达 125.45 MHz（原始物理频率 130.36 MHz）。
4. 全流程验证覆盖：验证平台包含 67 项自动化回归用例（63 项快速回归 + 4 项长时系统回归），累计通过 2348850504 次硬件断言自检与 4180179 项独立参考模型（Golden Model）逐字节比对；34 个 RTL 文件（42 个设计单元）语句覆盖率达 96.82%（12416/12824）、分支覆盖率达 94.61%（7230/7642）、FSM 状态覆盖率达 83.94%（115/137）；综合后 VQM 门级网表后仿真通过核心自检、五功能联合测试、分片重组窗口及顶层流包装窗口比对。

关键词：AFDX 端系统；ARINC 664；FPGA；高完整性通信；静态时序收敛；布局布线

目录

摘要 **I**

第 1 章 概述 **1**
1.1 设计目标 . . . 1
1.2 主要成果 . . . 2
1.3 文档结构 . . . 4

第 2 章 需求分析与规格分解 **5**
2.1 赛题需求 . . . 5
2.2 官方答疑澄清与设计实现对照 . . . 5
2.3 设计规格 . . . 7
2.4 协议依据 . . . 8
2.5 规格与设计章节对应 . . . 8

第 3 章 总体架构 **9**
3.1 系统框图 . . . 9
3.2 模块层次与 34 个 RTL 文件划分 . . . 10
3.3 关键设计决策 . . . 12
3.4 时钟与复位 . . . 13
3.5 时间基准 . . . 14
3.6 端到端数据通路 . . . 14
3.7 设计预算 . . . 14

第 4 章 接口设计 **20**
4.1 顶层端口 . . . 20
4.2 GMII 接口 . . . 21
4.3 以太网帧与 IPv4 分片格式 . . . 22
4.4 应用接口 . . . 24
4.5 配置总线 . . . 25
4.6 事件接口 . . . 26
4.7 内部接口 . . . 26

第 5 章 发送通路 **28**
5.1 顶层发送流包装器 . . . 28
5.2 发送核心与配置表 . . . 28
5.3 空闲页池 . . . 30
5.4 分页缓存 . . . 31
5.5 调度引擎 . . . 33
5.6 组帧器 . . . 37
5.7 双网发送 MAC . . . 39

第 6 章 接收通路 **41**
6.1 接收核心与配置表 . . . 41
6.2 接收 MAC . . . 42
6.3 帧头解析与预写入缓存 . . . 43
6.4 完整性检查与冗余管理 . . . 45
6.5 交付与 16 路并发顺序分片重组 . . . 48
6.6 统计分类 . . . 50

第 **7** 章 服务功能 **51**
7.1 服务通道 . . . 51
7.2 RTC 时钟同步 . . . 52
7.3 SNMP 网络管理 . . . 54
7.4 ARINC 615A 软件数据加载 . . . 57

第 **8** 章 配置、复位与统计 **61**
8.1 地址映射 . . . 61
8.2 配置表格式 . . . 61
8.3 配置读写与仲裁 . . . 62
8.4 配置写与查表的冲突仲裁 . . . 63
8.5 复位与初始化 . . . 64
8.6 统计计数器 . . . 64
8.7 异常处理汇总 . . . 64
8.8 主机编程流程 . . . 66
8.9 配置约束与网络规划规格 . . . 67

第 **9** 章 时序与存储设计 **68**
9.1 时延预算与流水账本 . . . 68
9.2 片上 RAM 分配与物理存储规划 . . . 69
9.3 静态时序收敛与关键路径工程优化 . . . 72
9.4 时序设计规则与流水划分 . . . 73

第 **10** 章 可验证性设计与实测摘要 **74**
10.1 验证体系与自动化回归策略 . . . 74
10.2 性能与多虚拟链路变长/分片压力实测 . . . 74
10.3 代码覆盖率签核 . . . 75
10.4 综合与布局布线后门级网表对照仿真 . . . 75
10.5 硬件可观察性与可控性机制 . . . 76

第 **11** 章 工程交付与复现指南 **77**
11.1 统一的设计方法与架构收敛 . . . 77
11.2 工程交付清单与 34 个 RTL 源文件 . . . 77
11.3 仿真测试复现指南 . . . 79
11.4 eLinx 物理实现与时序签核四步闭环 . . . 80

附录 **A** 源文件与默认参数 **81**
A.1 源文件 . . . 81
A.2 默认参数 . . . 82

附录 **B** 服务寄存器 **84**

附录 **C** 内部接口信号 **85**
C.1 消息接口 . . . 85
C.2 入队与授权 . . . 85
C.3 组帧器与 MAC . . . 86
C.4 帧描述与交付队列 . . . 87
C.5 服务发送总线 . . . 87

附录 **D** 术语与参考文献 **89**
D.1 术语 . . . 89
D.2 参考文献 . . . 89

第 1 章　概述

1.1　设计目标

AFDX（ARINC 664 Part 7）端系统是连接机载航空电子主机与确定性交换网络的核心枢纽。在发送方向，端系统将上层应用报文按 UDP/IP/以太网协议栈线速封装，严格依据各虚拟链路（Virtual Link, VL）设定的带宽分配间隔（Bandwidth Allocation Gap, BAG）执行整形调度，并向 A、B 双冗余物理网络锁步并发；在接收方向，端系统对双网到达帧流执行线速协议头解析与完整性检查（Integrity Checking, IC），并在冗余管理（Redundancy Management, RM）窗口内完成去重，确保向上层应用单次交付有效数据。赛题明确要求实现 ARINC 664 P7 基础通信、高完整性通信、SNMP 网络管理、ARINC 615A 软件数据加载（Data Loading）及 RTC 时钟同步五大功能，同时满足线速吞吐 ≥ 100 Mbps、端到端时延 < 1–2 ms、抖动数十微秒级以及 128 发送 / 4096 接收 VL 的性能指标。

图 1-1 以四字节消息为例，从应用提交到双网发送，再到接收端的单次交付。

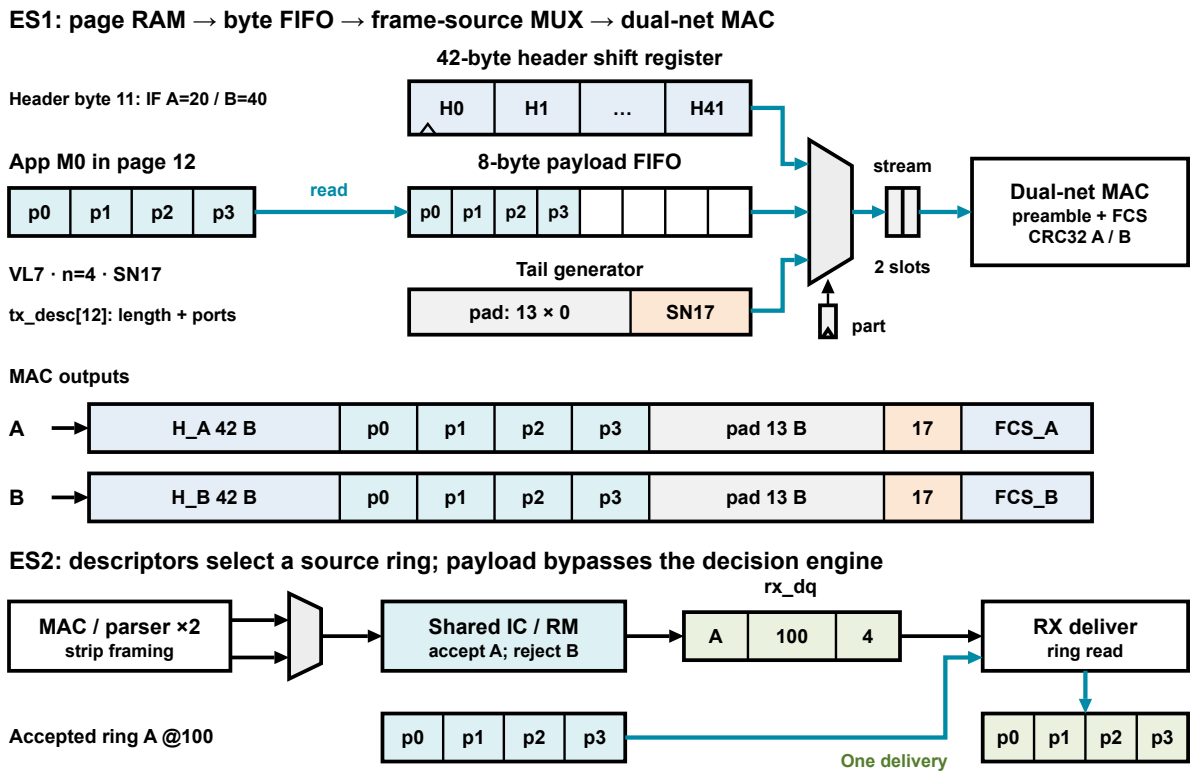


图 1-1　一条四字节消息的端到端表示。ES1 将同一组 p0–p3 加上 42 字节帧头、13 字节补零、SN 与 FCS 后发往两网；ES2 去掉帧封装并交付一次。示例采用 VL 7、SN 17，两帧均正确、IC/RM 启用、处于冗余窗口内且交付队列有空位，A 描述先被处理

在 FPGA 上实现全硬件化的 AFDX 端系统，核心矛盾在于逐字节线速流水处理与大规模链路状态存储之间的架构平衡。4096 条接收 VL 与 128 条发送 VL 的配置与状态需要较大的片上存储容量；采用分布式寄存器阵列将导致逻辑资源随链路规模线性膨胀，而采用片上块 RAM（BRAM）虽可大幅节约逻辑，但同步读延迟要求引擎显式安排查表、判定和回写流水。此外，大容量映射表还会产生严重的地址扇出与读出选择延

迟。为此，设计确立了“32-Bank 分块存储与读出流水、管理面隔拍时钟使能解耦、业务面 125 MHz 单周期全速运行”的系统微架构，在保证高存储密度的同时实现了 125 MHz 严格时序闭环。

本设计采用 Verilog-2001 HDL 构建全硬件化 AFDX 端系统顶层模块 `AFDX_top`（172 引脚）与核心模块 `afdx_es`，以单一 125 MHz（8 ns 周期）主时钟直接驱动八位 GMII 物理接口，实现双网双向 1 Gbps 物理线速（团队声明的 63 VL 满载规格下双向有效载荷吞吐达 1482.768 Mbps，81 VL 超规格极限单向有效载荷吞吐达 953.208 Mbps）。架构包含以下五项关键技术：

1. 流式应用边界封装与基于片上 **BRAM** 的全表格化状态维护架构：顶层 `AFDX_top` 提供 2 字节大端 VLID 前缀的应用流式接口（`tx_data`/`rx_data`），通过双侧 8×4 KiB 分体应用缓冲、发送侧 3 级流水 128 路 VLID 比较器与接收侧 4096×16 bit 映射 RAM 实现应用消息流与核心接口的桥接；128 条发送 VL 与 4096 条接收 VL 的配置参数及动态状态全部收敛入片上块 RAM，由专用硬件引擎按定拍流水执行读写更新，字节流水通路与深层控制逻辑由全部链路时分复用，实现每拍处理一字节的高效线速处理。
2. 1–8192 **B IPv4** 硬件自动分片与 **16** 路并发顺序分片重组引擎：发送端对超过单帧最大载荷（$L_{\max} = 1471$ B）的长消息自动拆分为共享同一 IP 标识（Identification）的多个 IPv4 分片（首分片携带 8 字节 UDP 头——按 RFC 768 将 IPv4 UDP 校验和置 `0x0000`——及最多 1464 B 载荷，后续分片携带最多 1472 B 载荷，逐片硬件计算 IPv4 首部校验和，最大 8192 B 拆分为 6 个以太网帧），各分片独立分配 AFDX 序列号（SN）并严格按所属 VL 的 BAG 调度发送；接收端在逐帧 IC/RM 去重后，通过双存储体（2-Bank $\times$ 8 槽位）共 16 个并发重组上下文及 128 KiB 四通道重组载荷缓存执行严格按片偏移顺序的分片拼接（拒绝缺首片、乱序及重叠分片），支持 2 s 硬件老化超时清理与非零全报文 UDP 校验和验证。
3. 前瞻预取与准点放行调度机制（**Lookahead Prefetch & Scheduled Release**）：发送引擎在 BAG 允许时刻到达前 512 拍提前检索候选 VL 并完成帧头组装。当报文提前排队且无线路冲突时，MAC 在允许发送时刻准点输出首前导码字节，实现稳态零发送抖动。
4. 双网锁步自研线速 **MAC**（**Lockstep Dual-MAC**）：采用单主控状态机同步驱动 A、B 双网 CRC-32 累加器与物理输出寄存器，保证双网副本在时钟周期级完全同拍起始；GMII 发送接口直接由 MAC 级寄存器引出，空闲启动延迟严格为 0 拍。
5. 服务通道状态折叠与流水时延精密补偿：SNMP 与 615A 协议引擎通过公共状态序列复用字节解析、字段拼装与响应发送逻辑，大幅收敛 FSM 逻辑面积；RTC 模块采用逐位移位加减流水实现纳秒/NTP 小数定点换算，并对片内 5 拍固定写回流水（40 ns）执行硬件确定性补偿。

根据帧长、时钟周期和各级流水拍数，可计算端系统的载荷容量与端到端时延。表 1-2 汇总相关指标，第 3.7 节给出预算推导。

1.2 主要成果

表 1-1 列出赛题五项功能的实现要点，表 1-2 给出技术指标要求、设计目标与实测结果。

表 1-1 赛题功能与实现要点

赛题功能	实现要点	详见
ARINC 664 P7 通信	128 条发送 VL 按 BAG 调度，支持 1–8192 B 应用消息 IPv4 自动分片与 UDP/IP 封装，A、B 双网锁步发送；4096 条接收 VL 按 VLID 直接查表与 16 路并发顺序分片重组	第 5、6 章
高完整性通信	FCS、帧长、IP/UDP 头逐帧检查（分片报文执行顺序偏移检查与非零全报文 UDP 校验和验证）；每网独立的序号完整性检查（IC）；冗余管理（RM）按处理顺序交付首个有效副本	第 6 章
SNMP 网络管理	v1/v2c 的 Get、GetNext、Set 和 v2 Trap，36 个管理对象，BER 逐字节编解码，多周期解耦	第 7.3 节
615A 软件数据加载	基于 TFTP 的单文件上传（LUI、LUS、LUR、LUH、数据文件）与介质定义下载（Media-Defined Download, MDD），8 KiB 加载区，CRC-16 硬件校验	第 7.4 节
RTC 时钟同步	SNTPv4 时钟同步，32 位秒加 32 位纳秒，8 ns 硬件时钟分辨率与 40 ns 延迟精确补偿（5 拍流水）	第 7.2 节

表 1-2 技术指标与实测映射

指标	赛题要求	设计目标	实测结果	依据与说明
传输速率	$\geq$ 100 Mbps	63 VL 规划满载 741.384 Mbps；物理线速上限 956.44 Mbps	63 VL 满载单向 741.384 Mbps（双向 1482.768 Mbps）；77/81 VL 压力达 906.136 / 953.208 Mbps	按接收端去重有效载荷计数，63 VL 线路占用率 77.5%（第 3.7 节、表 10-2）
端到端时延	$<$1–2 ms	核心单帧 24.424 µs；顶层满载 $<$ 2 ms	顶层流口径：63 VL 周期最大 80.336 µs，B2 突发最大 1175.432 µs；核心流水口径：满载最大 24.432 µs（L_{tx} 0.312 µs，L_{rx} 24.120 µs）	顶层流口径含整包串行输入与输出；单条 8192 B（6 分片）跨 5 个 BAG 完成时延 5.212 ms
传输抖动	$\leq$ 几十微秒	定长无争用为 0；63 VL 周期 $\leq$ 5 µs、突发 $<$ 100 µs	顶层流口径：63 VL 周期 1471 B 为 0.016 µs、全包长最大 3.360 µs，B2 突发最大 96.752 µs；核心同 VL 极差 0.008 µs	同 VL 多帧端到端时延极差（第 3.7.3 节、第 10.2 节）
工作时序	125 MHz 稳定工作	施加 0.30 ns 时钟偏移后布线零违例	布局布线后（`PRES 1.30`）扣除 0.30 ns 时钟不确定度后净建立裕量 +0.029 ns（原始物理裕量 +0.329 ns）、保持裕量 +0.604 ns	125 MHz（8.000 ns）主时钟 + 0.30 ns（300 ps）时钟不确定度约束，96142 端点 0 违例，等效频率 125.45 MHz

接下页

表 1-2 （续）

指标	赛题要求	设计目标	实测结果	依据与说明
帧与消息格式	VLID、SN 1–255，单帧载荷 ≤1471 B，长包分片	单帧 1–1471 B；应用消息 1–8192 B IPv4 分片与顺序重组	帧长 64–1518 字节，复位首帧 SN 为 0，之后 1–255 循环；1472–8192 B 自动 2–6 分片与 16 路并发顺序重组	符合 ARINC 664 P7、IEEE 802.3 及官方 1026/1030 答疑 R120/R138 规范
VL 容量	128 发送 / 4096 接收	硬件表满配 128 发送 / 4096 接收；声明满载规格 63 发送 / 63 接收	128 发送与 4096 接收全配置抽测 100% 交付；63 VL 双向满载 + 混合分片 0 丢包	块 RAM 状态表管理，接收映射表 65536 项全覆盖

1.3 文档结构

第 2–4 章说明规格、架构与接口，第 5–8 章说明收发、服务及配置，第 9–10 章说明时序、存储与可验证性，第 11 章总结设计。附录提供源文件、参数、寄存器、内部接口、术语及参考文献。

第 2 章　需求分析与规格分解

本章依据选题指南、官方《FPGA 创新大赛问题汇总》（1022、1026 及 1030 版，涵盖 R95–R97、R110–R112、R119–R126、R138–R139 等最新澄清）及表 2-3 所列协议规范，将赛题需求细化为功能、性能和接口规格。

2.1　赛题需求

选题指南（正文第 2–4 页）及后续官方答疑对 AFDX 端系统提出三类要求：

- 功能：ARINC 664 Part 7 通信（含超过单帧上限长报文的 IPv4 分片发送与接收重组）、高完整性数据通信、SNMP 网络管理、ARINC 615A 软件数据加载、RTC 纳秒时钟同步。
- 技术指标：传输速率 ≥ 100 Mbps（按官方 1030 答疑 R138-3/R139-3 口径，满载 VL 数量由参赛队伍结合机载网络规划自主声明并一致验证；本设计声明满载规格为单端 63 条发送 / 63 条接收 VL、BAG = 1 ms、单帧最大载荷 1471 B、双网冗余全双工，单向去重有效载荷吞吐 741.384 Mbps、双向 1482.768 Mbps，线路占用率 77.5%）；单帧采用 IEEE 802.3 格式及 ARINC 664 扩展，含 VLID 和 1–255 的序列号，单帧有效载荷不超过 1471 字节，总帧长不超过 1518 字节，应用消息支持 1–8192 字节自动分片与顺序重组；端到端时延小于 1–2 ms，抖动不超过几十微秒；硬件容量支持最多 128 条发送 VL、4096 条接收 VL。
- 接口与编码：顶层模块统一采用 `AFDX_top` 模板的 8 个核心端口（承载带 2 字节 VLID 前缀的应用层流式数据），并引出 A/B 双网 GMII 物理端口与配置统计端口；端口、信号和寄存器用小写，参数用大写，重要代码段须有简洁注释。

2.2　官方答疑澄清与设计实现对照

端到端时延支持核心内部接口与顶层公开流式接口双重测量口径：

1. 核心分段口径（**R96/R97**）：从核心应用接口成功提交消息末字节（m_{last} 握手完成）开始，至接收端完成首字节/末字节交付（$d_{\text{valid}}/d_{\text{last}}$）结束：

$$L_{\text{app}} = L_{\text{tx}} + L_{\text{gmii}} + L_{\text{rx}}, \qquad L_{\text{app,last}} = L_{\text{tx}} + L_{\text{gmii}} + L_{\text{rx,last}} \tag{2-1}$$

   其中 L_{tx} 为提交末字节至 GMII 输出以太网帧起始定界符（SFD，`8'hD5`）的时间；L_{gmii} 为发送端 SFD 至接收端 SFD 的物理链路传输时延；L_{rx} 与 $L_{\text{rx,last}}$ 分别为接收端收到 SFD 至输出首字节与末字节交付的时间。
2. 顶层公开流式口径（**R119/R139**）：从顶层 `AFDX_top` 发送端接收消息首字节（2 字节 VLID 前缀第 1 字节，`tx_valid && tx_ready`）或末字节（$t_{\text{tx,last}}$）起，至接收端输出该消息末字节（`rx_valid` 连续消息流的末尾拍，$t_{\text{rx,last}}$）止，分别记为全消息入口至末字节时延 $L_{\text{entry,last}} = t_{\text{rx,last}} - t_{\text{tx,first}}$ 与末字节至末字节时延 $L_{\text{last,last}} = t_{\text{rx,last}} - t_{\text{tx,last}}$。对于多帧分片长消息，端到端完成时延包含跨 $(F-1)$ 个 BAG 周期的确定性调度间隔。

表 2-1 官方 1022/1026/1030 答疑澄清与设计实现对照

项目与答疑序号	官方答疑要求与澄清	本设计的实现与架构边界
顶层应用流与前缀封装 (R62, R110, R120-3, R124, R126)	明确模板 8 端口中 `tx_data`/`rx_data` 作为顶层应用数据流接口，每条收发消息以 2 字节大端 VLID 作为前缀；双网 GMII 作为独立物理网络引脚引出	顶层 `AFDX_top`（172 引脚）将 `tx_data`/`tx_valid`/`tx_ready` 与 `rx_data`/`rx_valid`/`rx_ready`（按 R62 口径保留 `rx_ready` 端口兼容，接收侧无反压连续输出）封装为带 2 字节大端 VLID 前缀的应用流接口，通过 `afdx_app_tx`/`afdx_app_rx` 桥接内部核心 `afdx_es`，并完整引出 `gmii_tx*_a/b` 与 `gmii_rx*_a/b` 物理端口
长消息分片与重组 (R120-1, R120-2, R138)	超过单帧最大载荷（1471 B）的应用层长消息需支持硬件分片发送与接收重组，分片按所属 VL 的 BAG 调度，每片独立携带 AFDX 序列号 SN	支持 1–8192 B 应用层消息：发送侧自动拆分为共享同一 IP 标识的 IPv4 分片（首片含 8 字节 UDP 头——UDP 校验和按 RFC 768 置 `0x0000`——与最多 1464 B 载荷，中间片最多 1472 B，逐片计算 IPv4 头校验和，最多 6 片），按 BAG 逐片调度并递增 SN；接收侧配备 2-Bank × 8 槽位共 16 路并发顺序重组上下文与 128 KiB 缓存，支持非零全报文 UDP 校验和验证与 2 s 超时清理
满载规格与吞吐率 (R95-1, R119, R138-3, R139-3)	接收端去重后的有效载荷吞吐率须达到指标要求；满载 VL 数由团队按网络规划自主声明并一致验证	硬件表保持 128 发送 / 4096 接收全容量；按团队声明的 63 发送 / 63 接收 VL 满载规格（BAG = 1 ms，$L_{\max}$ = 1471 B，双网冗余，线路占用率 77.5%），实测单向去重交付吞吐率 741.384 Mbps（双向 1482.768 Mbps）；77 VL 与 81 VL 超规格极限压力实测达 906.136 Mbps 与 953.208 Mbps
端到端时延与抖动口径 (R96, R97, R119, R139)	单帧端到端时延不超过 2 ms；抖动定义为同一 VL 多帧端到端时延的最大值减最小值（极差），并在顶层流式边界（`tx_data` 至 `rx_data`）与核心分段边界同时验证	核心单帧满载最大时延 24.432 µs（同 VL 极差 0.008 µs）；顶层 `AFDX_top` 63 VL 满载周期激励下 1471 B 抖动仅 0.016 µs、全包长最大抖动 3.360 µs（最大时延 80.336 µs），B2 旋转三元组背靠背突发最坏抢占下最大抖动 96.752 µs（< 100 µs）、最大时延 1175.432 µs（< 2 ms）
MAC 实现与时钟复位 (R3, R46, R57, R59, R111)	允许调用厂商 IP 或自研 MAC；单一 125 MHz 全局时钟同步驱动，同步复位	采用纯 RTL 自研双网锁步 MAC（第 5.7 节），双网同拍起始、0 拍空闲启动延迟；全芯片由单一 125 MHz（8 ns）主时钟驱动，管理面复杂状态机采用 CE_DIV = 2 多周期使能树解耦
目标器件与考核 (R30, R112)	器件自主选择；以仿真结果、波形截图、门级网表与静态时序报告作为考核依据	目标器件选用中科亿海微 EQ6HL130（CSG484_H）；建立包含 67 项自动化回归、42 单元代码覆盖率、VQM 门级网表后仿真及主动施加 0.30 ns 时钟不确定度的布局布线后 STA 签核闭环

2.3 设计规格

表 2-2 列出细化后的设计规格，并按功能（F）、性能（P）、接口（I）和实现约束（C）编号。

表 2-2 设计规格

编号	规格	内容
F1	用户数据发送与自动分片	每个端系统支持 128 条发送 VL，每条均可独立配置 BAG（以时钟周期计，不超过 2^{30}）、VLID、源/目的 IP 与端口、单帧最大载荷 $L_{\max}$（1–1471 B）和队列深度；应用消息长度为 1–8192 字节，超过 $L_{\max}$ 时硬件自动执行 IPv4 分片（首分片携带 8 字节 UDP 头且 UDP 校验和置 `0x0000`，逐片计算 IPv4 首部校验和、按 BAG 独立调度并分配递增 SN）；按 VL 配置选择 A 网、B 网或双网发送
F2	用户数据接收与并发重组	支持 4096 条接收 VL，可映射任意 16 位 VLID；检查前导码、FCS、帧长、IP 头校验和、UDP 长度与校验和；两网分别进行完整性检查（IC）；启用冗余管理（RM）且副本到达间隔在 `skew_max` 内时，同一帧或分片只交付一次；配备 16 路并发硬件重组上下文（2-Bank × 8 槽位），支持 1–8192 字节分片报文的严格顺序偏移重组、非零全报文 UDP 校验和验证与 2 s 超时回收
F3	SNMP 网络管理	SNMPv1/v2c 的 Get、GetNext、Set 和 SNMPv2-Trap；36 个管理对象；Community（`community`）校验
F4	615A 软件数据加载	基于 TFTP 的单文件上传与介质定义下载（Media-Defined Download, MDD）；LUH 头文件与数据文件 CRC-16 硬件校验；超时重传机制
F5	RTC 时钟同步	SNTPv4 纳秒级时钟同步；32 位秒加 32 位纳秒时间戳；应答报文回显客户端 Transmit 时间戳
P1	吞吐	每方向有效载荷不低于 100 Mbps（63 VL 规划满载实测单向 741.384 Mbps、双向 1482.768 Mbps；77/81 VL 超规格压力实测 906.136 / 953.208 Mbps，变长高负载实测 904.752 Mbps）
P2	端到端时延	单帧不超过 2000 µs（核心满载最大实测 24.432 µs；63 VL 顶层周期激励最大 80.336 µs，B2 背靠背突发最大 1175.432 µs；分片长包按 $(F-1)\times$ BAG 确定性完成）
P3	传输抖动	63 VL 满载周期激励下 1471 B 抖动 0.016 µs、全包长最大 3.360 µs，B2 背靠背突发最大 96.752 µs；定长满载同 VL 极差仅 0.008 µs，变长循环核心 `afdx_es` R97 为 9.992 µs（顶层 `AFDX_top` R119 为 19.544 µs）
P4	帧格式	以太网帧长 64–1518 字节（含 FCS）；复位后首帧 SN 为 0，之后按 1–255 循环
P5	容量	128 条发送 VL、4096 条接收 VL 全部可配置并可同时启用；团队声明单端满载网络规划规格为 63 发送 / 63 接收 VL
I1	官方顶层端口	`AFDX_top` 的 8 个模板端口名称、方向和位宽保持完全兼容，承载带 2 字节大端 VLID 前缀的应用层流式收发报文
I2	线路接口	A、B 两路独立八位千兆 GMII（`gmii_tx_a/b`、`gmii_rx_a/b`），每拍一字节，帧间隔不少于 12 拍
I3	扩展接口	32 位字寻址配置总线 `cfg_*`、核心统计脉冲 `ev_*` 与应用包装层丢包事件 `ev_app[2:0]`；内部核心 `afdx_es` 同时保留显式信号接口 `app_tx_*`/`app_rx_*`

接下页

表 2-2　（续）

编号	规格	内容
C1	时钟与物理时序	单一 125 MHz 全局时钟，周期 8 ns；管理面协议状态机采用 CE_DIV = 2 多周期解耦；在主动施加 0.30 ns（300 ps）时钟建立不确定度约束下，eLinx 布局布线后静态时序完全收敛（净建立裕量 +0.029 ns、原始物理建立裕量 +0.329 ns、保持裕量 +0.604 ns，96142 端点 0 违例）
C2	编码规范	信号小写、参数大写；每个模块的文件开头说明其作用、接口和关键时序

2.4　协议依据

表 2-3　协议依据与版本

功能	依据	实现范围
AFDX 网络	ARINC 664 Part 7（2005-06-27）	帧格式、VL 与 BAG、完整性检查、冗余管理
数据加载	ARINC 615A-2（2002-05-10）、ARINC 665-1、RFC 1350	单文件上传与介质定义下载（MDD）；协议文件版本 A4，向下兼容 A3
网络管理	RFC 1157、RFC 3416、RFC 3418	SNMPv1/v2c，BER 编码
时钟同步	RFC 4330	SNTPv4 48 字节报文

2.5　规格与设计章节对应

表 2-4　规格与设计章节对应

规格	设计章节	规格	设计章节
F1	5.4–5.7	P4	4.3
F2	6.2–6.5	P5	5.2、6.1
F3	7.3	I1–I3	第 4 章
F4	7.4	C1	第 3.4 节、第 9 章
F5	7.2	C2	第 9 章、附录 A.1
P1–P3	3.7、5.5		

第 3 章　总体架构

3.1　系统框图

系统采用“顶层应用流式包装器 AFDX_top + 端系统协议核心 afdx_es”的两级顶层结构。图 3-1 展示了端系统核心 afdx_es 的共享存储与双网数据通路，外围由 AFDX_top 集成发送应用流包装器 afdx_app_tx 与接收应用流包装器 afdx_app_rx，对外提供带 2 字节大端 VLID 前缀的流式应用接口（tx_data/rx_data）。端系统核心由八个一级模块组成：发送核心和接收核心构成线速数据通路；应用口与 RTC、SNMP、615A 三个服务通过服务通道共用一对收发核心；配置模块提供寄存器、统计计数器和三张配置表的访问接口；时间基准为整个设计提供自由运行的周期计数。各模块的分工见表 3-1。

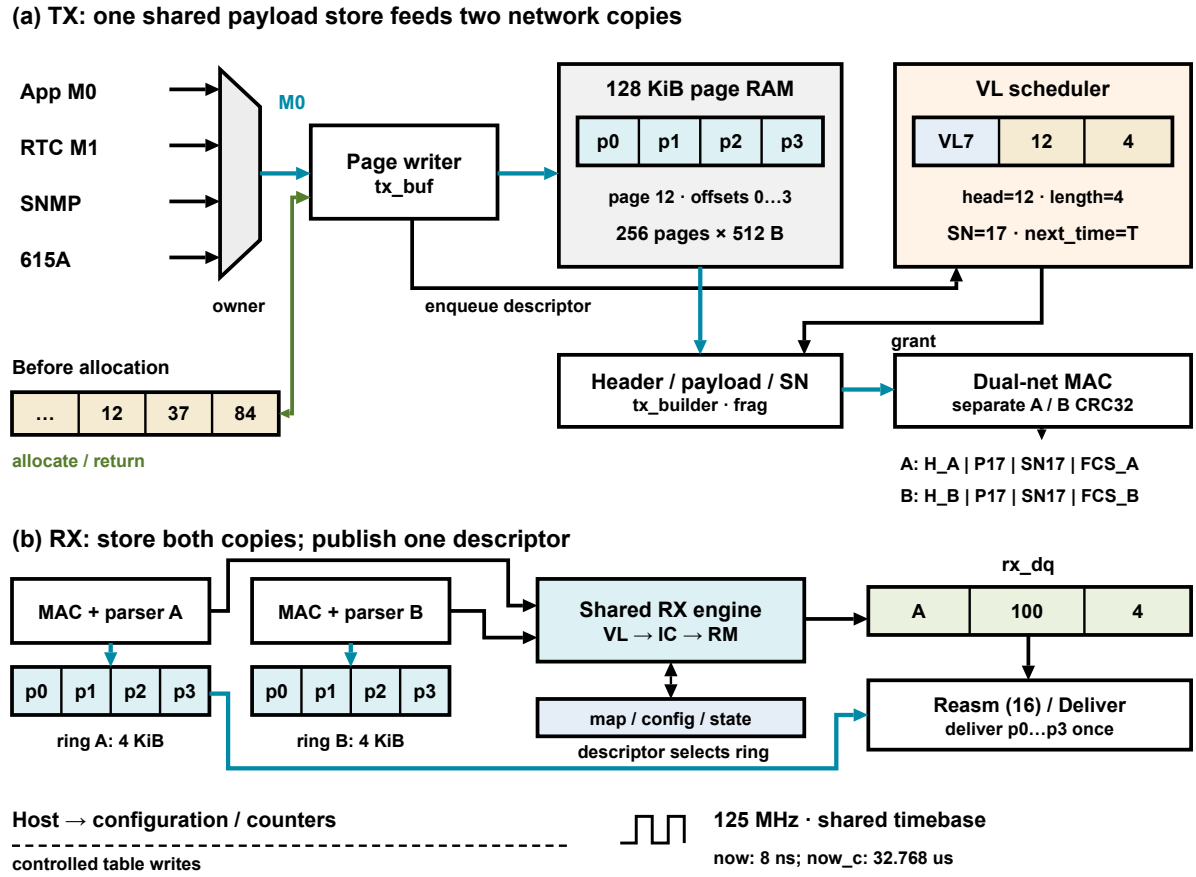


图 3-1　端系统核心的共享存储与双网数据通路。示例消息的载荷仅存一份，由描述符授权组帧并发往已启用的两网；接收侧载荷预写在本网环形缓存中，共享引擎只发布获准交付的描述符（分片报文经 afdx_rx_reassembly 重组后统一交付）。

表 3-1　一级模块分工

模块	职责
afdx_app_tx / afdx_app_rx	顶层应用流包装器：解析/添加 2 字节大端 VLID 前缀，提供双侧 8×4 KiB（32 KiB）分体应用消息缓冲、发送侧 3 级流水 128 路 VLID 比较映射与接收侧 4096×16 bit 索引反查

接下页

表 3-1 （续）

模块	职责
afdx_es	端系统协议核心顶层，集成下述八个一级核心子系统
afdx_tx_core	发送核心：入口两槽弹性缓冲、消息分页缓存与超长回滚（支持 1–8192 B）、128 条 VL 的 BAG 调度与 IPv4 自动分片、组帧与逐片 IPv4 首部校验和计算、双网锁步 MAC（第 5 章）
afdx_rx_core	接收核心：双网 MAC、帧头与分片解析、完整性检查（IC）、冗余管理（RM）、16 路并发 IPv4 顺序分片重组、交付（第 6 章）
afdx_svc	服务通道：应用与三个服务按整条消息轮询发送；按 UDP 目的端口将接收消息送至对应服务或应用
afdx_rtc	SNTPv4 纳秒级时钟同步（第 7.2 节）
afdx_snmp	SNMP 代理（第 7.3 节）
afdx_615a	615A 软件数据加载目标端（第 7.4 节）
afdx_cfg	地址译码、寄存器、27 个统计计数器、配置表写入握手（第 8 章）
afdx_timebase	32 位周期计数 now 与粗粒度计时 now_c

3.2 模块层次与 34 个 RTL 文件划分

AFDX_top 顶层下共划分为 34 个 Verilog RTL 源文件（累计 15856 行代码，包含 42 个 module 设计单元，源码位于 rtl/ 目录），模块归属、层次关系与片上同步 RAM 实例数量见表 3-2（各源文件的具体职责见附录 A.1）。全设计共例化 90 个通用简单双端口 RAM（afdx_ram_sdp）与 160 个针对物理布线与写保持时序定制的专用同步 RAM 单元（合计 250 个同步 RAM 实例），经综合与装箱映射为 814 个 M4K/EBR 物理存储块（占比 67.83%）。

为兼顾新增的应用流封装、16 路并发顺序分片重组、管理面协议状态机复杂度与 125 MHz 物理时序收敛，设计实施了清晰的层次解耦：

1. 顶层流式包装器（**afdx_app_tx.v**、**afdx_app_rx.v**、**afdx_app_map.v**、**afdx_app_payload.v**、**afdx_app_read.v**）：发送包装器从 tx_data 提取前 2 字节大端 VLID，通过内部 3 级流水 128 路 VLID 比较器完成 VLID → VL_idx 映射，并将 1–8192 B 载荷写入 8×4 KiB 分体缓存（afdx_app_payload，每体拆分为 $4 \times 1024 \times 9$ bit 子片并配置写数据 LUT 门控隔离）后经 afdx_app_read 8 信用预取器送入核心；接收包装器通过 afdx_app_map（4096×16 bit 同步 RAM）将核心输出的 12 位接收 VL 索引还原为 2 字节大端 VLID 前缀，经 32 KiB 接收应用缓冲无反压连续输出至 rx_data。
2. 发送入口弹性级与接收 **16** 路分体顺序重组（**afdx_tx_buf.v**、**afdx_rx_reassembly*.v**）：afdx_tx_buf.v 内置两槽位 49 bit 入口弹性缓冲（afdx_tx_ingress_elastic）与 256 B 超长报文回滚 RAM（afdx_tx_rollback_ram），核心载荷池由独立封装的 afdx_tx_payload_store（131072×8 bit，带 27 bit 本地写捆绑与 LUT 写隔离）实现；接收核心在 afdx_rx_engine 准入阶段与 afdx_rx_reassembly 协同查询并预留槽位，经 afdx_rx_deliver 从双网 4 KiB 环形缓存读出后送入 2 个重组存储体（afdx_rx_reassembly_bank $\times 2$，每体含 4 张 afdx_rx_reassembly_ram 元数据表与 8 个槽位），每个槽位由 4 通道交织的

8192×2 bit 通道 RAM（afdx_rx_reassembly_payload_lane，全芯片 64 通道共配置 128 个 lut_mask="8888" 写数据隔离 LUT 驱动全部 256 个 RAM 数据输入端）构成，单拍并行写入 8 位分片字节并消除短路径保持时间违例。

3. **afdx_snmp.v**、**afdx_615a.v** 与 **afdx_mgmt_tx.v**：管理面核心状态机（afdx_snmp_engine.v 与 afdx_615a_engine.v）由 2 分频局部时钟使能（CE_DIV=2）驱动，运行于 16 ns 多周期使能域；外围报文捕获、RAM 接口及 afdx_mgmt_tx.v 流式预取发送适配器运行于 125 MHz 单周期主时钟域。

表 3-2 34 个 RTL 文件（42 个设计单元）分工与下级层次

所属域与模块	子模块及下级关系	同步 **RAM (SDP/专用)**
AFDX_top	顶层流式包装器：例化 afdx_app_tx、afdx_app_rx 与 afdx_es	—
afdx_app_tx	3 级流水 128 路 VLID 比较器、descriptor_ids（afdx_ram_sdp $\times 1$）、afdx_app_payload（↪ afdx_app_payload_bank $\times 8$ ↪ afdx_app_payload_slice $\times 32$，1024×9）、afdx_app_read	1 / 33
afdx_app_rx	afdx_app_map（4096×16 VLID 反查 RAM）、descriptor_ids（afdx_ram_sdp $\times 1$）、afdx_app_payload（↪ 8 Bank / 32 Slice）、afdx_app_read	1 / 34
afdx_es	端系统协议核心顶层：集成 afdx_timebase、afdx_cfg、afdx_svc、afdx_tx_core、afdx_rx_core、afdx_rtc、afdx_snmp、afdx_615a	—
afdx_timebase / **afdx_cfg** / **afdx_svc**	周期计数 now[31:0] 与 now_c[15:0]；寄存器译码与 27 个核心统计计数器；应用与 3 项服务 4 路轮询仲裁与 UDP 端口分发	0 / 0
afdx_tx_core	afdx_tx_buf（含 afdx_tx_ingress_elastic 与 afdx_tx_rollback_ram）、afdx_tx_payload_store（131072×8） afdx_tx_engine（含 IPv4 分片表 afdx_ram_sdp $\times 7$）、afdx_tx_builder（12 拍帧头与 IPv4 校验和） afdx_tx_free（afdx_ram_sdp $\times 1$）、afdx_mac_tx（↪ afdx_crc32_d8 $\times 2$） 核心表 afdx_ram_sdp $\times 5$：tx_cfg、tx_state、tx_desc（256×47）、tx_mnext、tx_pnext	13 / 2
afdx_rx_core	afdx_mac_rx $\times 2$（各含 1 个 afdx_crc32_d8）、afdx_rx_parse $\times 2$、afdx_rx_engine、afdx_rx_deliver rx_map（32 Bank $\times 2$ 写分片，afdx_ram_sdp $\times 64$）、rx_dq（32×73 bit，afdx_ram_sdp $\times 1$）、rx_ring_a/b（afdx_ram_sdp $\times 2$）、rx_cfg（afdx_rx_table_slice $\times 8$）、rx_state（afdx_rx_table_slice $\times 11$） afdx_rx_reassembly ↪ afdx_rx_reassembly_bank $\times 2$（每体含 afdx_rx_reassembly_ram $\times 4$ 元数据表 + afdx_rx_reassembly_payload_slot $\times 8$ ↪ afdx_rx_reassembly_payload_lane $\times 32$，8192×2）	67 / 91
afdx_rtc	afdx_ntp_fraction $\times 3$（NTP 小数换算单元）	0 / 0
afdx_snmp	afdx_snmp_engine（CE_DIV=2 多周期域）、afdx_mgmt_tx、afdx_ram_sdp $\times 2$：request_ram、response_ram	2 / 0

接下页

表 3-2 （续）

所属域与模块	子模块及下级关系	同步 **RAM (SDP/专用)**
afdx_615a	afdx_615a_engine（CE_DIV=2 域）、afdx_mgmt_tx、afdx_ram_sdp ×6：packet_rx、packet_tx、lur_ram、header_ram、load_ram、denial_ram	6 / 0
全设计合计：**34** 个 **RTL** 文件（**42** 个设计单元），物理映射为 **814** 个 **M4K/EBR**		**90 / 160**

3.3 关键设计决策

表 3-3 关键设计决策

设计问题	候选方案	选择	理由
时钟结构	双网各自时钟域；统一 125 MHz 单时钟	单时钟	GMII 接口线速为 1 Gbit/s，载荷容量为 956.44 Mbps，满足赛题 100 Mbps 的要求；核心内部收发与控制逻辑在同一时钟域协同运行
顶层应用接口	直接暴露多信号核心口；2B VLID 前缀流式包装	2B VLID 流式包装	严格符合官方 1026/1030 答疑 R120-3/R124/R126 要求，保持 AFDX_top 模板端口定义，同时内部 afdx_es 保留完整显式控制接口
长包分片与重组	整包拆为多条独立消息入队；单描述符按片调度 + 16 路双 Bank 顺序重组	单描述符按片调度 + 16 路重组	8192 B 长包仅占 1 个消息描述符与 16 个缓存页，由 tx_engine 记录剩余字节数与片偏移并按 BAG 逐片授权；接收端在逐帧 IC/RM 后经 2-Bank × 8 槽位支持 16 条 VL 并发顺序偏移重组
MAC 实现	eLinx 官方 MAC IP；自研	自研	直接控制前导码、FCS 和帧间隔；输出寄存器直接驱动 GMII，空闲发送延迟为 0 拍
双网 MAC	双 MAC 独立发送；单 MAC 同时驱动两网	单 MAC 驱动两网	两网副本同拍起始，节省一套 MAC；单网 VL 也按整帧占用双网发送时隙（第 5.7 节）
VL 状态存储	寄存器阵列；块 RAM 状态表	块 RAM 状态表	寄存器用量随 VL 数量线性增长；采用块 RAM 后，由对应引擎按固定流水读写，配置写入须等待访问许可
VLID 映射	全地址直接查表；哈希加 CAM；限制 VLID 范围	直接查表	接收核心采用 32-Bank ×2 分片 65536 × 13 bit 直接查表（3 级或树归并流水，无哈希冲突）；应用发送采用 3 级流水 128 路比较器完成映射
发送调度	轮询扫描；最早到期优先；预取窗口扫描	预取窗口扫描	每拍至多一级比较；512 拍窗口为扫描回访与组帧留出时间，无争用时可在允许时刻启动发送
接收写入	整帧检查后写入；边接收边预写入	先预写入后提交	载荷边接收边写入环形缓存，检查失败时回退预写入指针，释放本帧占用的空间

接下页

表 3-3 （续）

设计问题	候选方案	选择	理由
服务发送	服务独立发送通路；按消息轮询共享发送核心	共享发送核心	服务报文与用户报文遵守同一 BAG 调度；无服务报文时应用直通，零附加延迟
服务实现	软核处理器；硬件状态机	硬件状态机	协议解析与报文组装均由硬件状态机完成；RTC 时间戳在报文握手当拍采样
服务状态序列	各分支独立展开；公共状态序列加返回状态寄存器	公共状态序列	SNMP 和 615A 复用取字节、整数拼接与报文输出过程，统一同步 RAM 的访问时序

3.4 时钟与复位

端系统逻辑采用单一主时钟 `clk`，工作频率 125 MHz，周期 8 ns，所有内部触发器均在 `clk` 上升沿同步更新。`reset` 为高有效的同步复位，外部须保证其释放与 `clk` 上升沿严格同步；板级实现时由物理复位同步器提供。

为满足航电高层管理协议（SNMP 与 ARINC 615A）在 125 MHz 主频下的静态时序收敛要求，系统在统一主时钟树下设置了管理面局部时钟使能结构：

1. 多周期引擎结构（**CE_DIV** $= 2$）：在 `afdx_snmp` 与 `afdx_615a` 顶层生成每两拍有效一次的时钟使能信号 `ce`，并通过多级低扇出寄存器复制树（`CE_GROUPS`）驱动核心引擎内部逻辑。全工程共有 2747 个触发器（由 253 条多周期约束规则覆盖）约束为 2 周期建立路径（16 ns 建立时间预算），大幅缓解深层协议分支的组合时序压力。
2. 单周期外围与接口隔离：应用流包装器、以太网报文接收捕获、分片重组引擎、双端口协议 RAM 写口、32 位配置总线、RTC 纳秒时钟基准以及用于流式预取的 `afdx_mgmt_tx` 适配模块均保持在 125 MHz 单周期全速运行。这种结构既避免了异步跨时钟域（CDC）带来的亚稳态风险与同步 FIFO 开销，又使关键控制与存储接口维持周期级确定性。

核心顶层采用与 `clk` 同步的 GMII 字节流接口。连接外部物理 PHY 时，接收数据随 PHY 恢复的 RX_CLK 变化，发送侧需向外输出 GTX_CLK；因此板级外围需在接收引脚与核心之间设置帧级弹性缓冲，将 RX_CLK 域字节流同步至 `clk` 域，发送侧则由 `clk` 经输出寄存器驱动 GTX_CLK。

复位结束后，发送核心清零 `tx_state`、`tx_cfg` 及分片进度表，并将 256 个页号写入空闲页池；接收核心清零 `rx_map`、`rx_cfg` 和 `rx_state`，分片重组引擎复位 16 个重组槽位状态，应用包装器清零映射表与描述符槽。深度最大的 `rx_map` 物理划分为 32 个子库并行清零，配合初始化定时器计数需 65536 拍。两个核心的 `init_done` 经流水打拍并在核心顶层相与锁存输出（从解除复位到 `afdx_es` 就绪为 65539 拍，即 524.312 µs；顶层 `AFDX_top` 的应用入口 `tx_ready` 在包装器同步完成清零后于约第 65,550 拍拉高）。表初始化完成之前，`cfg_ready` 保持 0、配置写不被接受，凡在此期间开始接收的以太网帧均整帧忽略。

3.5 时间基准

`afdx_timebase` 为全设计产生三个信号：

- `now[31:0]`：自由运行的周期计数，复位为 0，每拍加 1，约 34.36 s 回绕。帧起点以 1 拍（8 ns）为粒度，所有时间戳的先后比较均通过作差并检查符号位实现，因此 BAG 可配置为不超过 2^{30}（约 8.59 s）的任意整数拍（例如 100003 拍）。
- `now_c[15:0]`：粗粒度计时，等于 `now[27:12]`，每 4096 拍（32.768 µs）加 1，仅用于接收侧的冗余管理。
- `byte_en`：字节使能，复位期间为 0，此后恒为 1，即每拍都是字节边界。

3.6 端到端数据通路

图 1-1 给出一帧用户消息从发送端应用到接收端应用的完整路径。在顶层 `AFDX_top` 中，应用发送流先经 `afdx_app_tx` 剥离 2 字节大端 VLID 前缀并映射为内部 VL 索引，随后进入 `afdx_es` 发送侧，依次经过入口弹性级、分页缓存、调度与分片引擎、组帧器和双网 MAC；两份以太网帧经 A、B 网到达接收端后，各自经 MAC 和解析器检查，由接收引擎做完整性检查和冗余管理，送入交付模块；单帧消息直通输出，IPv4 分片消息则进入 `afdx_rx_reassembly` 完成分片拼接与全报文 UDP 校验，最后经 `afdx_app_rx` 加上 2 字节大端 VLID 前缀流式交付给上层应用。

帧内数据通路每拍处理一个字节，对应 1 Gbit/s。发送侧先把应用消息写入分页缓存，满足调度条件后，一边读取载荷，一边生成以太网帧；接收侧将到达的载荷预写入环形缓存，帧检查通过后再提交给交付与重组模块。各段的拍数和上界在第 3.7 节推导。

3.7 设计预算

本节由各模块的状态机拍数推导吞吐、时延、抖动、缓存和握手等待的设计值，作为第 5–8 章模块设计的约束。拍数均以 125 MHz 时钟计，1 拍 = 8 ns；仿真实测结果见测试报告。

本节及第 5–8 章以相邻两级输出有效时刻之间的周期差作为阶段延迟。状态机从 IDLE 态响应请求并启动操作需要 1 个周期，因此阶段延迟比操作本身的执行时间多 1 拍。例如，调度引擎的入队阶段共耗时 6 拍，其中响应请求需 1 拍，执行 ENQ 操作需 5 拍。

3.7.1 吞吐

单帧载荷为 n 字节（$1 \le n \le 1471$）的以太网帧连同 8 字节前导码、4 字节 FCS 和 12 拍帧间隔，在物理线路上占用的时长为

$$C(n) = 8 + 47 + \max(n, 17) + 12 = 67 + \max(n, 17) \text{ 拍}. \tag{3-1}$$

满载帧 $n = 1471$ 占 $C = 1538$ 拍，最短帧 $n \le 17$ 占 84 拍。每网每方向的单帧最大有效载荷容量为

$$R_{\max} = \frac{1471 \times 8 \text{ bit}}{1538 \times 8 \text{ ns}} = 956.44 \text{ Mbps}. \tag{3-2}$$

设 BAG 为 B 拍，N 条 VL 各按该间隔发送满载帧时，单向有效载荷率为 $N \times 1471 \times 8\,\text{bit}/(B \times 8\,\text{ns})$，可调度条件为 $N \times C \le B$。当 BAG $= 1$ ms（$B = 125000$ 拍）时：

- 声明 **63 VL** 满载规划基准（**R119/R138-3/R139-3**）：单端激活 $N = 63$ 条发送 VL 与 63 条接收 VL，每毫秒占用物理线路 $63 \times 1538 = 96,894$ 拍，线路占用率为 $96894/125000 = 77.5152\%$（约 77.5%），单向去重有效载荷吞吐率为 $63 \times 1471 \times 8\,\text{bit}/1\,\text{ms} = 741.384$ Mbps（若计入顶层 2 字节 VLID 前缀则输入流速率为 742.392 Mbps），双向全双工合计有效载荷吞吐率为 1482.768 Mbps。
- **77 VL** 与 **81 VL** 超规格极限压力基准：$N = 77$ 条 VL 时单向有效载荷率为 906.136 Mbps（线路占用率 94.7%）；物理极限 $N = \lfloor 125000/1538 \rfloor = 81$ 条 VL 时，单向有效载荷率达 953.208 Mbps（线路占用率 99.66%）。

线速通路上各环节的吞吐预算如下：

- 应用消息接收：顶层 `afdx_app_tx` 以每拍 1 字节无间断接收带 2 字节 VLID 前缀的消息并写入 8×4 KiB 分体缓冲；核心 `afdx_tx_buf` 正常每拍接收一字节，每次申请新缓存页暂停 6 拍；消息接收结束后，写入描述符及等待入队应答共暂停约 10 拍。一条占 3 页的 1471 字节消息在核心口共耗时 $1471 + 3 \times 6 + 10 = 1499$ 拍，小于该帧占用线路的 1538 拍；一条占 16 页的 8192 字节最大分片消息写入核心耗时 $8192 + 16 \times 6 + 10 = 8298$ 拍（66.384 µs），而其拆分为 6 个以太网帧跨 5 个 BAG 周期发送，入口写入速率远高于线路消耗速率。
- 授权与准备：引擎完成一帧或一片授权约需 9–12 拍，组帧器 `afdx_tx_builder` 采用 12 拍（步骤 `0..11`）准备序列在帧头阶段硬件计算 20 字节 IPv4 首部校验和（IPv4 首分片 UDP 校验和按 RFC 768 固定置 `0x0000`，无需预读整包载荷），并在计划起始时刻前 8 拍（`START_LEAD_REGISTERED = -8`）完成准备，均在当前帧发送期间重叠完成，远小于最短帧的 84 拍时隙。
- 接收重组与交付：核心交付口与 16 路重组引擎均支持每拍 1 字节线速吞吐；最短以太网帧占 84 拍，接收引擎 30 拍（相位 `0..30`）帧处理日程可在连续两帧最短帧到达期间处理双网副本；单网到达的载荷率至多 1471/1538 字节每拍，低于交付与重组速率。

3.7.2 端到端时延

端到端时延的起止点定义见第 2 章。下文按收发通路无其他消息争用、当前 VL 已满足 BAG 发送间隔的基准条件，给出核心分段口径下各区段的典型拍数和设计上界（表 3-4），发送侧和接收侧的逐拍时序见图 3-2。若从顶层 `AFDX_top` 流式接口计时（R119/R139 全消息入口首字节至接收末字节），由于 `afdx_app_tx` 与 `afdx_app_rx` 均采用“整包收齐并完成 VLID 映射后转发”的存储转发边界，单条 n 字节消息在无排队争用时额外叠加顶层发送写入（$n + 2$ 拍加跨层预取）与顶层接收缓存转发（n 拍加 2 字节 VLID 前缀输出）；例如在 63 VL 周期稳态激励下，1471 B 消息顶层全消息最大端到端时延实测为 80.336 µs，旋转突发（B2）最大时延为 1175.432 µs；对于拆分为 F 片的 8192 B 长消息（$F = 6$，BAG $= 1$ ms），端到端完成时延由 $(F - 1) \times$ BAG 主导，实测为 5212.304 µs。

图 3-2(a) 中，消息入队时直接成为候选。若末字节在 RECV 状态下正常接收，且无需在消息尾部申请新页，由于载荷写口增加了 1 拍流水寄存、核心输出端插入了 1 级流

缓冲（双缓冲结构），从接收末字节到输出首个前导码字节固定为 32 拍（0.256 µs）；按 1022 答疑 R95 规则计至以太网帧 SFD 结束（8 字节前导码尾），发送处理时延 L_{tx} 典型为 39 拍（0.312 µs）。单字节消息或末字节跨页时多约 6 拍。图 3-2(b) 给出单网单帧的接收过程，由于 `rx_map` 采用 32 子库多 Bank 分块存储增加 2 拍读回流水，从 GMII 最后一个 FCS 字节到首个交付字节由 24 拍更新为 26 拍（0.208 µs）。

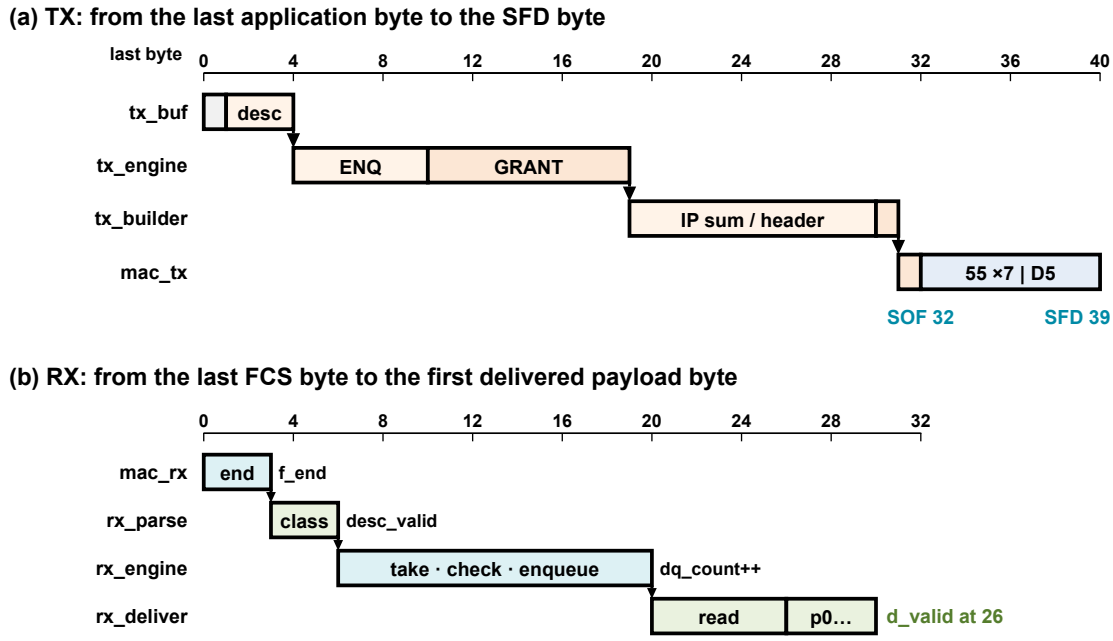


图 3-2 无其他消息争用、VL 已到期时的流水时序：(a) 发送准备与起始；(b) 接收检查与交付

表 3-4 端到端时延预算（拍）

区段	典型	上界	构成
发送侧至前导码	32	100	分页缓存写载荷寄存 1 拍、写描述符并请求入队 3 拍，引擎入队 6 拍，授权 9 拍，组帧准备 11 拍，输出流缓冲与 MAC 起始 2 拍；计至 SFD 结束（L_{tx}）为 39 拍
线路	1525	1525	前导码首字节至 FCS 末字节，共 $8+1518$ 字节（$n=1471$）
接收至首字节	26	200	MAC 检测帧尾 3 拍，解析器分类 3 拍，多 Bank 分块映射与引擎判定入队 14 拍，交付模块读块 RAM 队头并读取环形缓冲 6 拍；上界另计另一网描述与过期清理让出
逐字节交付	$n-1$	$n-1$	首字节到末字节，每拍一字节，$n=1471$ 时为 1470 拍
合计	3053	3295	24.424 µs（遇 AGE 扫描 1 拍为 3054 拍 / 24.432 µs）/ 26.360 µs

存在其他消息时，还须计入发送排队、提前授权占槽、线路占用与接收排队交付的等待；端到端时延的上界由各 VL 的 BAG、帧长和流量配置共同决定。

3.7.3 发送抖动

记 VL v 的帧允许时刻为 T_v（上一帧实际起始时刻加 BAG），实际起始时刻为 S_v，发送抖动为 $J_v = S_v - T_v$，即同一 VL 相邻两帧起始间隔与 BAG 之差。

与第 3.7.2 节考察消息到达时 VL 已到期的直通延迟不同，本节考察消息在允许时刻之前已提前排队时的准点发送精度。当线路和待发槽位均未被其他帧占用时，发送抖动为 0。调度器能否在 T 时刻准点启动发送，取决于候选能否在 T 之前完成扫描、授权和准备。一条 VL 在 $T-512$ 进入提前窗口，此后：

- 扫描回访：无候选时，引擎每 2 拍检查一条 VL，轮询 128 条共需 256 拍。每扫描 16 条 VL 至多插入一次入队操作（6 拍），每发送一帧还需插入一次发送起始确认（TX Start Confirm）（4 拍）；每次插入操作都会清空 5 级扫描流水并重新发出（约 6 拍，见第 5.5 节），折合每次入队约 12 拍、每次发送起始确认（TX Start Confirm）约 10 拍。计入这些开销后，任一 VL 相邻两次被检查的间隔约为 400 拍。
- 授权与准备：授权 9 拍、组帧器准备 11 拍、起始判断 2 拍，共 22 拍。

$400+22=422<512$，留有 90 拍裕量。因此在上述条件下，MAC 在 T 时刻准点输出前导码首字节。

线路争用时，物理链路同一时刻只能发送一帧，组帧器也仅设一个待发槽位。v 到期且消息已排队时，可能因以下三种原因推迟发送：

- 线路占用：排在 v 前面的帧逐个发完，每帧线路占用 $C(n_u)=67+\max(n_u,17)$ 拍。
- 提前授权占槽等待：扫描按 VL 索引轮转，选中首条已到期或将在 512 拍内到期的候选 VL。若扫描先遇到一条尚未到期的 VL u 并发出授权，组帧器的待发槽位将被 u 占用，直到 T_u 时刻才启动发送，期间 v 无法获得授权。每次此类提前授权引入的占槽等待为 $w_u<512$ 拍。
- 轮询扫描延迟：扫描从上次选中位置继续向后轮询，在遍历到 v 之前所选中的 VL 均先于 v 发送；扫描与这些帧的发送重叠进行，扫描范围不超过一圈，连同 v 自身的授权和准备不超过约 422 拍。

记 Q_v 为在 v 之前发送的其他 VL 帧的集合，则

$$J_v \le \Big(\sum_{u\in Q_v}\big(C(n_u)+w_u\big)+422\Big)\times 8\ \text{ns}, \tag{3-3}$$

其中 w_u 只对在自身允许时刻之前被授权的帧非零。服务报文各走自己的 VL，同样按 BAG 调度，已包含在 Q_v 中。

一次争用只造成一次相位后移。下一次允许时刻按实际起始时刻加 BAG 计算（第 5.5 节）。多条 VL 发生发送争用后，后发 VL 的相位后移，此后各自按 BAG 更新允许时刻。要持续保持错开的相位，需满足两个条件：

- 各 VL 的 BAG 互为整数倍。ARINC 664 P7 规定 BAG 取 2^k ms，满足这一点；任取整数拍的 BAG 会使相位相互漂移，周期性地再次发生争用。
- VL 在允许时刻之前已有消息排队。计时到期时若队列为空，`timer_active` 被清除，下一条消息到达即发，相位改由应用的提交时刻决定。

两个条件都满足时，争用只在启动时发生，稳定后同一 VL 相邻两帧的起始间隔恰为 BAG。

以 P3 的 50 µs 作为发送抖动的检查取值，按式 (3-3) 对应 $\sum_{u\in Q_v}(C(n_u)+w_u)\le 6250-422=5828$ 拍。例如：

- 同时到期的其他 VL 至多 3 条最长帧（$w_u=0$）：$3\times 1538=4614$ 拍，满足；

- 前面有 2 条最长帧且都被提前授权：$2 \times (1538 + 512) = 4100$ 拍，满足；
- 77 条最短帧同时到期：$76 \times 84 = 6384$ 拍，不满足。

同时参与争用的 VL 数量超出上述示例时，主机须错开各 VL 首条消息的提交时刻，使任一 VL 之前排队发送的帧满足该条件；在上述两个条件下，错开只需在启动时做一次（第 8.9 节）。

与 ARINC 664 P7 的抖动公式 $40\ \mu s + \sum(20 + L_{\max}) \times 8$ bit/1 Gbit/s 相比，$C(n) \times 8$ ns 恰等于公式中对应帧的线路占用时间；本设计的扫描开销 422 拍（3.4 µs）与至多 8 次提前授权等待（共 4096 拍）之和小于公式中的 40 µs 项。因此对满足 P7 公式、且排在前面的提前授权帧不超过 8 个的配置，式 (3-3) 给出的发送抖动上界不超过按 P7 抖动公式计算的值，也不超过 P7 规定的 500 µs 抖动上限。

3.7.4 存储预算与缓存规划

在存储架构规划中，设计严格解耦了“全量静态链路配置”与“动态在途流量缓冲深度”：

- 静态配置容量：硬件配置表与映射表完全支持 128 条发送 VL 与 4096 条接收 VL（加上接收映射表 65536 个 VLID 全地址空间）的并发静态配置；接收包装器 `afdx_app_map` 同步维护 4096×16 bit 的 VL_idx $\rightarrow$ VLID 反查映射。
- 顶层应用流包装缓冲：`afdx_app_tx` 与 `afdx_app_rx` 各自配备 8×4 KiB（共 32 KiB）分体消息缓存与 32 项描述符队列，单侧即可容纳 4 条 8192 B 最长消息或 22 条 1471 B 单帧消息，吸收顶层连续突发与核心分页申请之间的短时速率差；`afdx_tx_buf` 另设 256 B 回滚 RAM（`afdx_tx_rollback_ram`），对超过 8192 B 的异常超长消息安全回收已分配缓存页并上报 `ev_app[1]`。
- 动态发送载荷缓存：发送核心配置 128 KiB 共享载荷存储池（`afdx_tx_payload_store`，划分为 256 页 $\times$ 512 字节），采用基于单向链表的动态页分配机制。单帧最长报文（1471 字节）占 $\lceil 1471/512 \rceil = 3$ 页，8192 B 最大分片消息占 16 页。在团队声明的 63 VL 满载规格下，63 条单帧大包同时在途占 $63 \times 3 = 189$ 页（73.8%）；在 60 条 1471 B 短包与 1472/3000/8192 B 三条分片长包并发的 63 VL 混合满载工况下，峰值占 $180 + 3 + 6 + 16 = 205$ 页（80.1%），均留有充裕空闲页裕量（77/81 VL 超规格单帧压力下分别占 231/243 页）；当超规格突发导致空闲页耗尽时，硬件执行安全丢弃并置位统计脉冲，确保系统免于死锁。
- 接收端环形缓冲、描述队列与 **16** 路重组缓存：A、B 网各设 4096 字节接收环形缓冲（`rx_ring_a/b`），可安全容纳连续两个最长以太网帧（2×1471 字节）。交付描述队列 `rx_dq` 设为 32×73 bit；分片帧经 IC/RM 去重通过后立即由 `afdx_rx_deliver` 搬移至 `afdx_rx_reassembly` 的专用重组槽位（2-Bank $\times$ 8 槽位 = 16 路并发上下文，每槽位 8 KiB，总计 128 KiB 重组载荷 RAM），使跨多个 BAG 周期到达的长报文分片完全不驻留占用 4 KiB 线速环形缓冲；超时未收齐的残缺上下文在 $250,000,000$ 拍（2 s）后由硬件自动老化回收。

3.7.5 握手等待

表 3-5 握手等待与初始化预算

接口或操作	等待上界与计算依据
入队应答	发送引擎一次只执行一种操作（含分片元数据读写的授权操作约 9–12 拍）。每扫描满 16 次或已有候选时可插入入队操作，入队应答等待不超过 64 拍。
m_ready 反压	核心入口经两槽弹性缓冲隔离后，取页暂停 6 拍；消息末尾写描述符、入队应答、异常时页面回滚释放和事件脉冲存在短时停顿；顶层 AFDX_top 则由 32 KiB afdx_app_tx 缓存吸收核心跨页停顿，在缓存未满时保持 tx_ready=1 连续接收整条应用消息。
应用口等待	app_tx_ready 受服务通道轮询仲裁控制（第 7.1 节）。三个服务均有报文待发时，应用至多等待其各发送一条：RTC 报文 48 字节、SNMP 响应不超过 256 字节、615A 报文不超过 516 字节；经管理发送适配器流式预取后，字节发送以线速每拍 1 字节推进，连同预取启动与入队等待，握手等待上界约为 1100 拍（约 8.8 µs）。
配置写	向配置表写入时，需等待对应收发核心通过 cfg_ok 给出许可。发送引擎最坏连续执行发送起始确认、入队和分片授权约 28 拍；接收引擎单次帧处理日程（含分片重组槽位查询与预留）为 30 拍（相位 0..30），双网连续处理两份描述加 AGE 中断等待上界约 64 拍（第 8.4 节）。
初始化	最大的 rx_map 逻辑容量 65536 项，硬件划分为 32 个子库并行清零（2048 拍），配合全局清表定时器计数及跨级输出寄存，从解除复位至核心 afdx_es 就绪耗时 65539 拍（524.312 µs），顶层 AFDX_top tx_ready 于约第 65,550 拍有效。

服务通道按整条消息轮询，限制了服务连续占用消息通路的时间，表中的约 8.8 µs 是提交口的握手等待预算。入队后，用户与服务报文按各自 VL 的 BAG 参与调度，共享组帧器和发送 MAC；服务 VL 的线路占用与提前授权等待统一计入式 (3-3) 及第 8.9 节的配置预算。

第 4 章　接口设计

4.1　顶层端口

按官方 1026/1030 答疑澄清（R110、R120-3、R124、R126），顶层模块 `AFDX_top` 完整保留选题模板的 8 个核心端口（名称、方向和位宽不变，其中 `tx_data`/`rx_data` 直接承载带 2 字节大端 VLID 前缀的应用层流式消息），并显式引出 A、B 双网千兆 GMII 物理端口、32 位配置总线及硬件统计事件脉冲。顶层共声明 29 个独立端口信号，总位宽为 172 位（占 EQ6HL130-CSG484_H 可用 339 个用户 I/O 的 50.74%）。顶层端口定义见表 4-1；内部协议核心 `afdx_es` 同时保留显式的核心应用端口 `app_tx_*`/`app_rx_*`（第 4.7 节）。

表 4-1　`AFDX_top` 顶层端口定义（29 个独立端口信号，共 172 引脚）

端口	方向	含义
选题模板端口（2 字节 VLID 前缀应用流接口）		
`clk`	入	125 MHz 全局主时钟
`reset`	入	同步复位，高有效
`tx_data[7:0]`、`tx_valid`	入	应用层发送字节流：`tx_valid` 连续为 1 区间构成一条消息（前 2 字节大端 VLID + 1–8192 B 载荷），相邻消息间至少空闲 1 拍
`tx_ready`	出	应用发送就绪：内部表初始化完成且发送包装缓存可接纳新消息时为 1
`rx_data[7:0]`、`rx_valid`	出	应用层接收字节流：`rx_valid` 连续为 1 区间输出一条去重/重组后的完整消息（前 2 字节大端 VLID + 载荷），相邻消息间至少空闲 1 拍
`rx_ready`	入	按官方答疑 R62 口径保留端口兼容，硬件内部不使用（无反压连续交付）
A/B 双冗余网络 GMII 物理接口		
`gmii_txd_a[7:0]`、`gmii_tx_en_a`	出	A 网 GMII 发送字节流与发送使能（含前导码、SFD 与 FCS）
`gmii_txd_b[7:0]`、`gmii_tx_en_b`	出	B 网 GMII 发送字节流与发送使能，与 A 网锁步同拍起始
`gmii_rxd_a[7:0]`、`gmii_rx_dv_a`	入	A 网 GMII 接收字节流与数据有效
`gmii_rxd_b[7:0]`、`gmii_rx_dv_b`	入	B 网 GMII 接收字节流与数据有效
配置与统计事件接口		
`cfg_wr`、`cfg_rd`、`cfg_addr[17:0]`、`cfg_wdata[31:0]`	入	32 位字寻址配置总线写/读使能、地址与写数据
`cfg_rdata[31:0]`、`cfg_rvalid`、`cfg_ready`	出	配置读返回数据、读数据有效（固定 2 拍）与写就绪

接下页

表 4-1 （续）

端口	方向	含义
ev_tx[6:0]	出	核心发送统计脉冲：accept、drop_dis、drop_full、drop_long、drop_nopage、tx_a、tx_b
ev_a[7:0]、ev_b[7:0]	出	A/B 网接收分类与异常脉冲（含 ev_a/b[7] 分片重组拒绝），见表 6-3
ev_rm_drop、ev_deliver	出	冗余管理去重丢弃脉冲、有效消息交付脉冲
ev_app[2:0]	出	顶层应用流包装器丢包脉冲：[0] 发送未映射 VLID 丢弃，[1] 发送超 8192 B 或缓存满回滚丢弃，[2] 接收包装缓存溢出丢弃

顶层参数 CE_DIV、TX_PAGE_AW、TX_OFF_AW、LOOKAHEAD、RX_RING_AW、RX_DQ_AW、RX_CONFIG_BANKS 的默认值分别为 2、8、9、512、12、5、16，透传至核心 afdx_es。

4.1.1 顶层端口映射与网络分层定义

按照 ARINC 664 Part 7 标准分层与官方 1022/1026/1030 答疑澄清（R62、R110、R111、R112、R120-3、R124、R126）：

1. 顶层应用流接口（模板 **8** 端口）：clk、reset、tx_data、tx_valid、tx_ready、rx_data、rx_valid、rx_ready 构成面向机载分区应用的流式收发边界；每条消息以 2 字节大端 VLID 为首，以 valid 连续高电平定界、消息间至少空 1 拍定帧。
2. 双网 **GMII** 物理接口：独立引出 A 网（gmii_txd_a/gmii_tx_en_a、gmii_rxd_a/gmii_rx_dv_a）与 B 网（gmii_txd_b/gmii_tx_en_b、gmii_rxd_b/gmii_rx_dv_b）八位千兆 GMII 端口，传输含 7 字节前导码、1 字节 SFD 与 4 字节 FCS 的完整以太网字节流。
3. 配置同步窥探与核心显式接口：主机通过 cfg_* 配置发送 VL 表与接收映射表时，AFDX_top 硬件自动窥探 tx_cfg 字 2（VLID）与 rx_map 写入事务，同步更新 afdx_app_tx 与 afdx_app_rx 的 VLID 映射，无需额外配置指令；内部 afdx_es 保留显式 app_tx_*/app_rx_* 信号供核心级与服务联合测试直接复用。
4. **MAC** 实现与评测依据（**R111**、**R112**）：MAC 模块采用纯 RTL 自研实现；功能仿真波形、日志、覆盖率记录、门级网表与布局布线后 STA 报告共同构成签核依据。

4.2 GMII 接口

A、B 两网各有一组八位 GMII，在 valid 为 1 的每拍传送一个字节。

- 发送：gmii_tx_en_a/b（核心内为 tx_valid）从前导码首字节至 FCS 末字节连续为 1，帧间至少空闲 12 拍，见图 4-1。GMII 输出直接来自 MAC 的输出寄存器，发送适配延迟为 0。若某条 VL 未使能其中一路网络，则该帧发送期间对应网络的使能与数据全程保持为 0。
- 接收：gmii_rx_dv_a/b（核心内为 rx_valid）的上升沿表示一帧开始，下降沿表示该帧结束。相邻两帧的低电平间隔最短为 1 拍。MAC 先寄存输入，再逐拍查找帧起始定界符（SFD）、累加 CRC 并剥离 FCS。

- 复位：复位期间输出为 0；对于表初始化完成前开始接收的帧，接收核心将整帧忽略（第 3.4 节）。
- 时钟与错误信号：接口采用全双工模式，收发字节均与 `clk` 同步，不含 TX_CLK、RX_CLK、TX_ER、RX_ER、CRS 和 COL。接收核心根据字节流检查前导码、帧长和 FCS；连接外部 PHY 时，外围适配逻辑须处理 PHY 错误标记并丢弃带错帧。收发时钟适配见第 3.4 节。

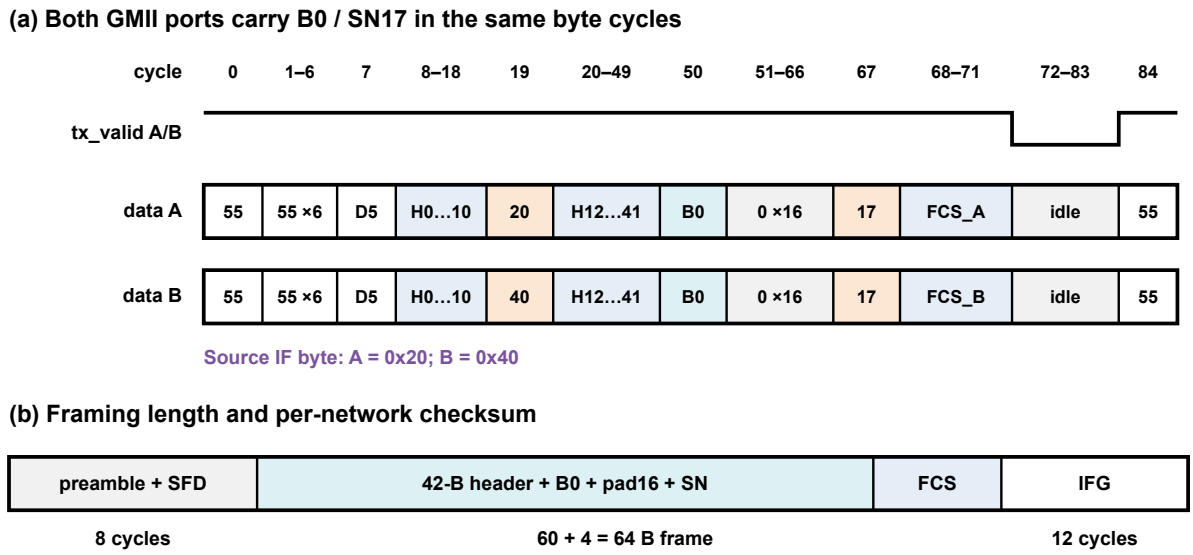


图 4-1 GMII 发送一帧的接口时序（1 字节消息，最短帧）。第 0 拍为前导码首字节，图中按标注的周期段压缩展示；示例载荷为 B0、SN 为 17，两网同拍起始，仅第 11 字节（接口号，A 为 `0x20`、B 为 `0x40`）和 FCS 不同；某网未启用时该网整帧保持为 0

4.3 以太网帧与 IPv4 分片格式

图 4-2 给出了本设计发送的 AFDX 单帧与首分片格式。目的 MAC 和源 MAC 按 ARINC 664 Part 7 的固定格式编码：目的 MAC 为 `03 00 00 00` 加 16 位 VLID，源 MAC 为 `02 00 00` 加 16 位端系统用户 ID 和 1 字节接口号（A 网 `0x20`、B 网 `0x40`）。当应用层消息总长 $M \le L_{\max}$（$L_{\max} \le 1471$ B）时，按不分片单帧发送；当 $L_{\max} < M \le 8192$ B 时，硬件按 IPv4 分片规则自动拆分为多个以太网帧（表 4-2）：

- 不分片帧与首分片（**42** 字节协议头）：包含 14 字节以太网头、20 字节 IPv4 头与 8 字节 UDP 头。IPv4 片偏移为 0，首分片置 `MF=1`（不分片帧 `MF=0`）；UDP 长度字段填写整条应用消息总长加 8（$M+8$），UDP 校验和字段按 RFC 768 填 `0x0000`。非末分片的 IPv4 数据段长度（含首片 8 字节 UDP 头）向下取整为 $\min(L_{\max}+8, 1479)$ 的最大 8 字节整数倍（当 $L_{\max} = 1471$ B 时，首片承载 8 字节 UDP 头加 1464 B 应用数据，共 1472 B IPv4 载荷）；应用数据不足 17 字节时补零至 17 字节。
- 后续分片（**34** 字节协议头）：仅含 14 字节以太网头与 20 字节 IPv4 头，不含 UDP 头。同一应用消息的所有分片共享同一个 16 位 IPv4 标识（Identification），片偏移字段以 8 字节为单位递增（例如 $L_{\max} = 1471$ B 时第 2 片偏移为 $1472/8 = 184$，承载最多 1472 B 应用数据；8192 B 消息共拆分为 6 个以太网帧，末片 `MF=0` 承载 840 B 应用数据）。后续分片若末片应用数据不足 25 字节，则硬件补零至 25 字节，使加 34 字节头、1 字节 SN 与 4 字节 FCS 后的以太网总帧长仍不小于 64 字节。

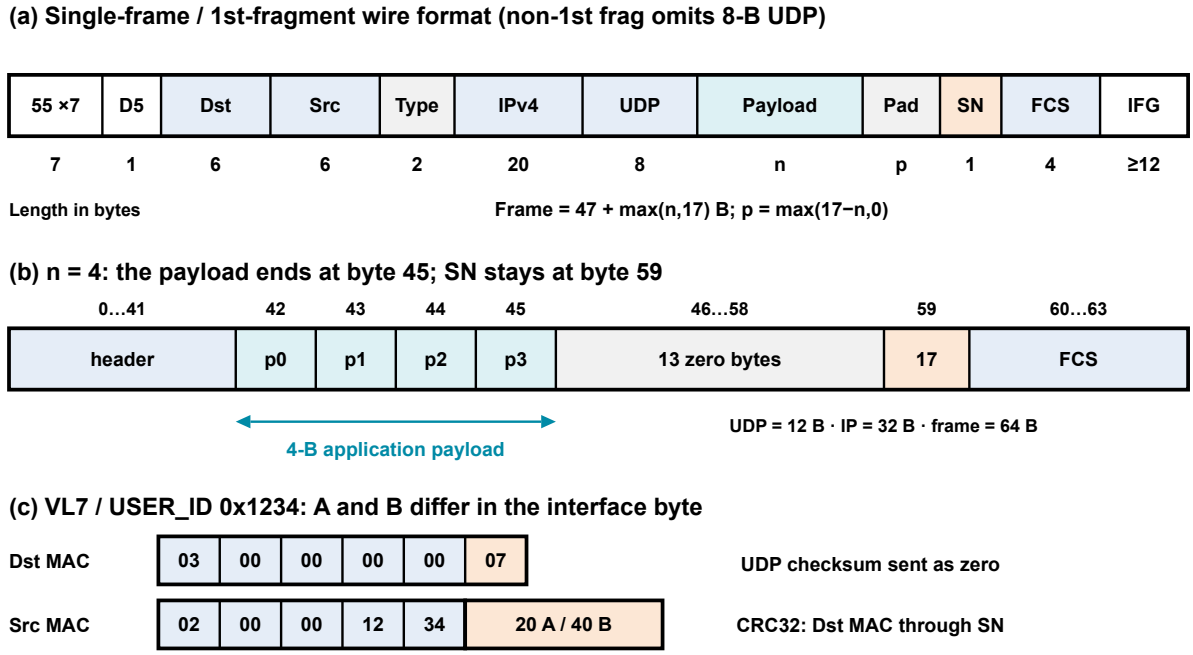


图 4-2 AFDX 单帧与首分片格式及具体字节位置。单帧应用数据长度 $1 \le n \le 1471$ 字节（后续分片无 8 字节 UDP 头，应用数据从第 34 字节起始，至多 1472 字节）；示例 $n = 4$ 时，p0–p3 位于第 42–45 字节，补零 13 字节，SN 位于第 59 字节，含 FCS 的帧长为 64 字节。地址编码采用 VL 7 与 USER_ID=0x1234

表 4-2 AFDX 以太网帧与 IPv4 分片字段（十六进制）

字节位置	长度	内容
0–3	4	`03 00 00 00`，Part 7 规定的目的 MAC 固定部分
4–5	2	VLID，高字节在前
6–8	3	`02 00 00`，源 MAC 固定部分
9–10	2	端系统用户 ID（寄存器 `USER_ID`）
11	1	接口号：A 网 `0x20`，B 网 `0x40`
12–13	2	`08 00`（IPv4）
14–33	20	IPv4 头：`45 00`；总长度（首片/单帧 $n+28$，后续分片 $n+20$）；标识（每条应用消息递增 1，同消息各分片共享）；标志与片偏移 `{2'b00, MF, offset[12:0]}`；TTL `0x01`；协议号 `0x11`（UDP）；逐片硬件计算的 IPv4 头校验和；源 IP；目的 IP
34–41	8/0	仅单帧与首分片存在 UDP 头：源端口、目的端口、总长度 $M+8$、校验和（发送填 `0x0000`）；后续分片无此 8 字节
42（或 34）起	n	本帧承载的应用数据（单帧 $1 \le n \le 1471$，首片 ≤ 1464，后续片 ≤ 1472）
其后	补零	单帧/首片 $n < 17$ 时补 0 至 17 B；后续分片 $n < 25$ 时补 0 至 25 B
再后	1	AFDX 序列号 SN（每发出一个以太网帧/分片独立递增）
最后	4	FCS（CRC32，由 MAC 添加）

由此，单帧/首片含 FCS 的帧长为 $47 + \max(n, 17)$，后续分片含 FCS 的帧长为 $39 + \max(n, 25)$，物理帧长均严格落在 64–1518 字节范围内；A、B 两份帧只有第 11 字节和 FCS 不同。复位后，各 VL 首帧的 SN 均为 0，之后按 1、2、…、255、1、…循环（分片报文的每一个分片均独立消耗一个递增 SN 并遵守所属 VL 的 BAG 间隔）。组播目的

IP 按 Part 7 图 3-38 配置为 `224.224.VLID[15:8].VLID[7:0]`，发送核心直接使用配置表中的目的 IP，因此单播服务（如 615A）也可共用该通路。

端系统以 VL 为独立调度与查表粒度：发送侧支持 1–8192 B 消息的单帧直发或 IPv4 自动分片；接收侧先对每一到达帧执行 FCS/IPv4 校验、基于 VLID 的逐帧 IC 序号检查与双网 RM 去重，不分片帧直接交付上层应用或内部服务，分片帧则进入 16 路并发重组引擎按片偏移顺序拼装为完整 UDP 报文（若发送方提供了非零 UDP 校验和则同时验证全报文校验和）后统一交付。

CRC32 采用反射多项式 `0xEDB88320`，初值为全 1，计算范围覆盖从目的 MAC 到 SN 的全部字节。结果取反后按低字节在前的顺序发送。收发通路均采用组合模块 `afdx_crc32_d8`，以 32 位当前 CRC 值和 8 位数据为输入，输出更新后的 CRC 值。

4.4 应用接口

4.4.1 顶层 2B-VLID 流式边界与核心应用接口

应用层消息交互包含顶层公开流式接口（`AFDX_top` 的 `tx_data`/`rx_data`）与内部核心应用接口（`afdx_es` 的 `app_tx_*`/`app_rx_*`）两层：

- 顶层流式接口（**R120-3/R124/R126**）：发送端在 `tx_ready=1` 时拉高 `tx_valid`，连续送入 2 字节大端 VLID 前缀与 1–8192 B 应用载荷（期间 `tx_valid` 不得间断），消息结束后至少拉低 `tx_valid` 1 拍以标志帧尾。包装器 `afdx_app_tx` 将消息写入 32 KiB 分体缓冲并通过 3 级流水比较器查得 7 位发送 VL 索引：若 VLID 未映射则整包丢弃并脉冲上报 `ev_app[0]`；若消息超过 8192 B 或包装缓冲/描述符队列满则回滚丢弃并上报 `ev_app[1]`。接收端由 `afdx_app_rx` 通过 4096×16 bit 映射 RAM 反查 16 位 VLID，在 `rx_valid` 连续高电平区间依次输出 2 字节大端 VLID 与完整载荷（相邻消息间至少空 1 拍；若 32 KiB 接收包装缓冲溢出则上报 `ev_app[2]`）。
- 核心显式消息接口（**`afdx_es`**）：`app_tx_valid` 与 `app_tx_ready` 同拍为 1 时传送一个载荷字节，`app_tx_last` 标记末字节；7 位 VL 索引 `app_tx_vl` 和可选 UDP 端口在首字节锁存。没有服务报文争用时，`app_tx_ready` 经两槽入口弹性缓冲连接 `afdx_tx_buf`，每拍接收一字节，仅在跨 512 B 页分配（暂停 6 拍）或消息末尾入队确认（暂停约 10 拍）时拉低；顶层 32 KiB 包装缓冲将该跨页停顿对外部流接口屏蔽。三个管理服务同时有待发报文时，核心应用口轮询等待上界约 1100 拍（约 8.8 µs，第 3.7 节）。

核心缓存页用尽、消息超长（> 8192 B）或 VL 队列已满时，发送核心丢弃该消息并计数，自动回收已分配的缓存页，随后继续接收并丢弃该消息的剩余字节。

图 4-3(a) 以核心口无服务争用、长度为 $2 \le n \le 512$ 字节且不跨页的消息为例：接收首字节后，分配缓存页需 6 拍；接收末字节后，写入消息描述并完成入队需 10 拍。`app_tx_ready` 为 0 时，上游预取器保持当前字节及其有效标记，直至完成握手；图中的 C0 是下一条消息的首字节。

4.4.2 接收消息的连续交付

在核心接收接口上，一条消息（单帧或重组完成的长消息）的 n 个字节在连续 n 拍内送出，期间 `app_rx_valid` 保持为 1，`app_rx_last` 与末字节同拍；12 位 VL 索引、端

口和来源网络在整条消息期间保持固定，见图 4-3(b)。顶层 `AFDX_top` 在此基础上前置 2 字节大端 VLID，以连续 $n+2$ 拍 `rx_valid=1` 无反压输出。

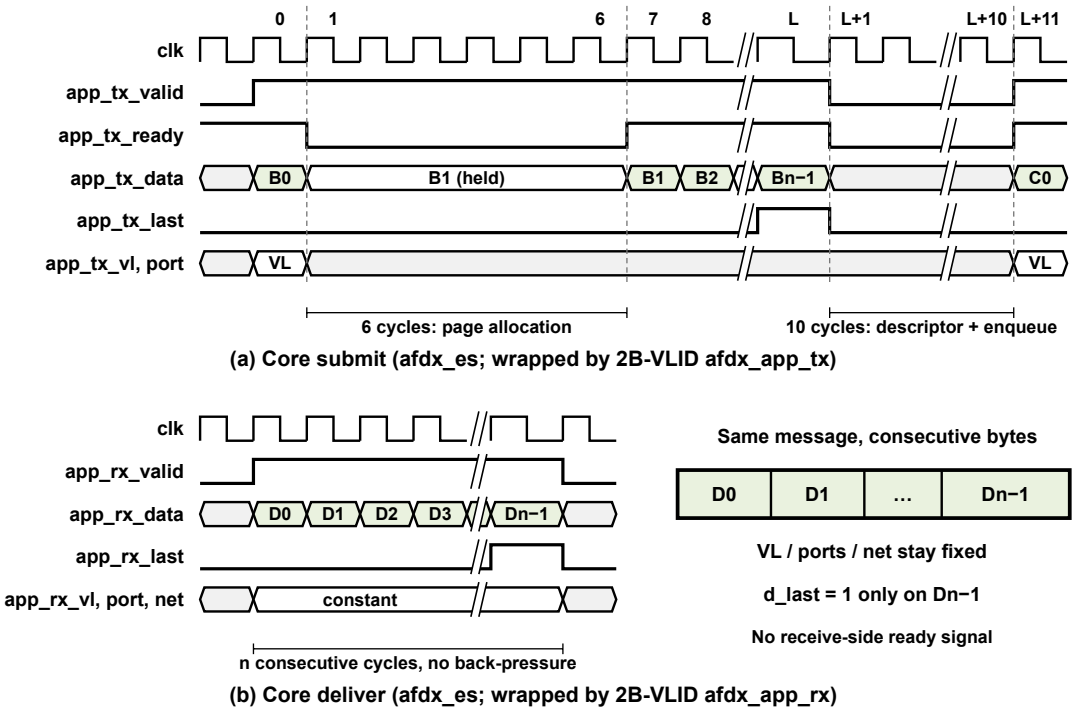


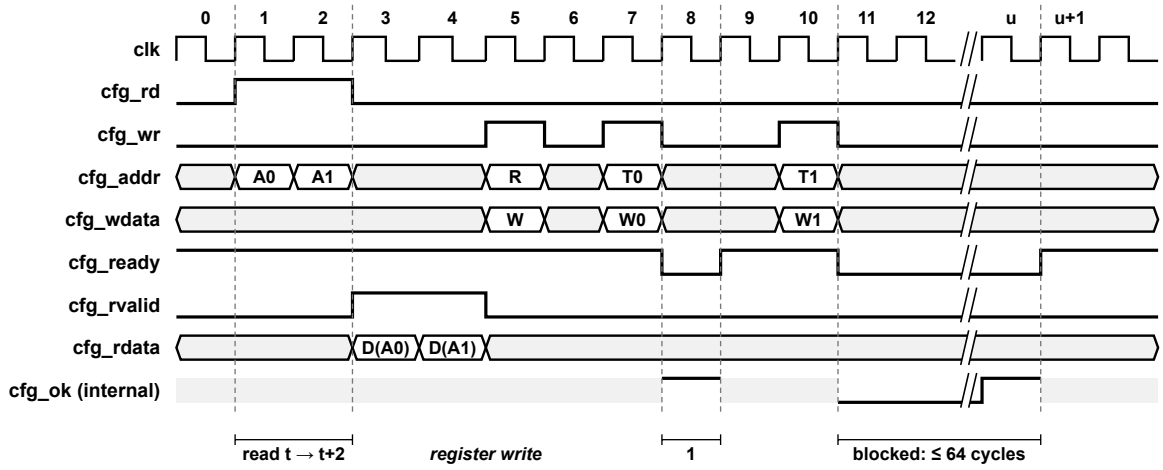
图 4-3 核心应用接口（`afdx_es`）时序：(a) 无争用、空闲引擎及时接纳时的消息提交与握手示例；(b) 接收消息连续交付。顶层 `AFDX_top` 由 `afdx_app_tx`/`afdx_app_rx` 封装为带 2 字节大端 VLID 前缀的流式接口；示例中的 6 拍取页和 10 拍提交间隔不代表所有争用条件下的等待上界

4.5 配置总线

配置总线支持寄存器和三张配置表的写入，以及寄存器和计数器的读取，时序如下（t 为周期号，图 4-4）：

- 读：t 拍 `cfg_rd` 为 1，则 $t+2$ 拍 `cfg_rvalid` 为 1、`cfg_rdata` 有效。复位结束后可以每拍连续读。同拍读写同一寄存器时，读操作返回更新前的值。
- 写寄存器：`cfg_wr` 与 `cfg_ready` 同拍为 1 时，完成写操作。
- 写配置表：$t+1$ 拍起 `cfg_ready` 变 0，待写寄存器保持地址和数据，直到核心在某拍 u 给出 `cfg_ok` 时写入，$u+1$ 拍 `cfg_ready` 恢复；不受阻时 $u=t+1$，最长等待 64 拍（第 8.4 节）。

图 4-4 中，A、R、T 分别对应连续寄存器读、寄存器写和配置表写；寄存器写期间 `cfg_ready` 保持为 1。



RX table write: the same T1 / W1 survives the blocked interval

accepted at 10	pending while cfg_ok = 0	permit at u	ready at u+1
{T1, W1}	{T1, W1, enable=1}	write T1 ← W1	enable=0

图 4-4 配置总线的连续读、寄存器写与接收配置表写时序

4.6 事件接口

ev_* 端口在统计事件发生时输出单周期脉冲，与第 8 章的计数器一一对应，供外部逻辑监视各类事件。对于每条发送消息，发送核心均产生且仅产生 accept 或四类丢弃脉冲中的一种。对于表初始化完成后开始接收的每一帧，接收核心在对应网络上产生一个帧分类脉冲；若判定为正确帧，还会输出一个处理结果脉冲（第 6.6 节）。

4.7 内部接口

一级模块之间及核心内部子模块之间的接口以寄存器输出为主。ready 握手与 cfg_ok 采用组合输出；afdx_svc 的数据选择和接收分发也采用组合直通，仅包含选择与门控逻辑。内部接口主要采用以下四种约定：

- 字节流握手（valid/ready/last）：valid 与 ready 同拍为 1 时传送一个字节；附带字段在首字节锁存并保持到末字节。
- 请求应答（req/ack）：请求持续有效，直到接收到应答脉冲；应答与结果在同一拍输出。
- 单深缓冲握手（**Single-Slot Handshake**）（valid/take 或 valid/ready）：生产者置 valid 并保持字段不变，消费者接收后，生产者才能送入下一项。
- 写允许授权（cfg_ok）：表的读写控制模块通过组合信号给出写允许授权。许可有效时，核心寄存外部写请求，并在下一拍写入 RAM；该拍不再发出冲突的表读请求（第 8.4 节）。

表 4-3 汇总各内部接口，各信号的定义见附录 C。

表 4-3 内部接口汇总

接口	两端	约定	关键时序
包装发送 m_*	afdx_app_tx → afdx_es	字节流	经 8 信用预取器连续供给核心应用口
消息 m_*	afdx_svc → tx_buf	字节流	经两槽弹性缓冲解耦；跨页取页低 6 拍，消息末尾低约 10 拍
入队 enq_*	tx_buf → tx_engine	请求应答	携带 14 位消息长度，应答不晚于请求后 64 拍
授权 grant_*	tx_engine → tx_builder	单深缓冲握手	含分片偏移、标识与首/后续片标志；当前帧 tx_sof 起可接受下一授权
发送起始/分片完成	tx_builder → tx_engine	单拍脉冲	回报首前导字节时刻 t_{start} 及当前分片读末指针 fragment_done_*
帧字节 s_*	tx_builder ↔ mac_tx	字节流	经一级双缓冲流寄存器解耦；MAC 在前导码末拍接收第 0 字节
帧描述 desc_*	rx_parse → rx_engine	单深缓冲握手	f_end 后第 3 拍给出；含分片偏移与 MF，单帧判定日程 30 拍
重组预约 query/reserve	rx_engine ↔ rx_reassembly	查询应答	帧准入第 5/6 拍发起查询，19 拍返回槽位可接纳结果，第 28/29 拍原子提交
交付队列 rx_dq	rx_engine → rx_deliver	32×73 队列	队列非空后的第 6 拍输出首个 frag_valid
重组与交付 d_*	rx_deliver → rx_reassembly → afdx_svc	字节流	单帧直通，分片帧按顺序写入槽位并在末片校验通过后连续输出
服务发送 svc_tx_*	三个服务 → afdx_svc	字节流	按整条消息轮询
配置表写	afdx_cfg → 收发核心	写允许授权	等待不超过 64 拍（第 8.4 节）

第 5 章　发送通路

本章沿数据流向介绍发送通路的顶层应用流包装器 afdx_app_tx 以及发送核心 afdx_tx_core 的五个核心模块：分页缓存 afdx_tx_buf（含入口弹性级与回滚 RAM）与空闲页池 afdx_tx_free、分片调度引擎 afdx_tx_engine、组帧器 afdx_tx_builder 和双网锁步 MAC afdx_mac_tx。

5.1　顶层发送流包装器

afdx_app_tx 将顶层 AFDX_top 的带 2 字节 VLID 前缀应用字节流（tx_data/tx_valid/tx_ready）转换为核心 afdx_es 的显式消息流（m_valid/m_ready/m_data/m_last/m_vl）：

- **VLID 窥探与 3 级流水比较映射**：主机向配置总线写入 tx_cfg 字 2 时，包装器同步锁存对应 7 位发送索引的 16 位 VLID。收到应用消息前 2 字节后，硬件经 3 级流水比较器（第 1 级 128 路相等比较 $\rightarrow$ 第 2 级 16×8 分组归约 $\rightarrow$ 第 3 级最终编码）映射出内部 7 位 VL 索引 m_vl；若未命中已使能 VLID，则在消息结束拍丢弃整包并输出 ev_app[0]。
- **32 KiB 分体缓存与 8 信用预取**：消息载荷按页写入 afdx_app_payload（8 个逻辑 4 KiB Bank，每 Bank 拆分为 $4 \times 1024 \times 9$ bit 子片并加入写数据 LUT 门控隔离，读返回流水 4 拍），已接纳消息的起止指针与 VL 索引存入 32 项描述符队列；下游 afdx_app_read 维护 8 信用预取缓冲，无缝屏蔽 4 拍读延迟并连续驱动核心入口。消息长度超过 8192 B 或包装缓存/描述符满时，写指针回滚至包首并输出 ev_app[1]。

5.2　发送核心与配置表

图 5-1 是发送核心的框图。应用和服务提交的消息先由 afdx_tx_buf 分页写入 128 KiB 载荷存储池 afdx_tx_payload_store，再加入发送队列。afdx_tx_engine 按 BAG 与单帧最大载荷 $L_{\max}$ 选出下一待发单帧或 IPv4 分片，向 afdx_tx_builder 发出授权。组帧器随后预读对应片偏移处的载荷、拼接 42 字节（首片/单帧）或 34 字节（后续分片）协议头并硬件计算 IPv4 首部校验和，逐字节送入 afdx_mac_tx，由 MAC 锁步发往两网。

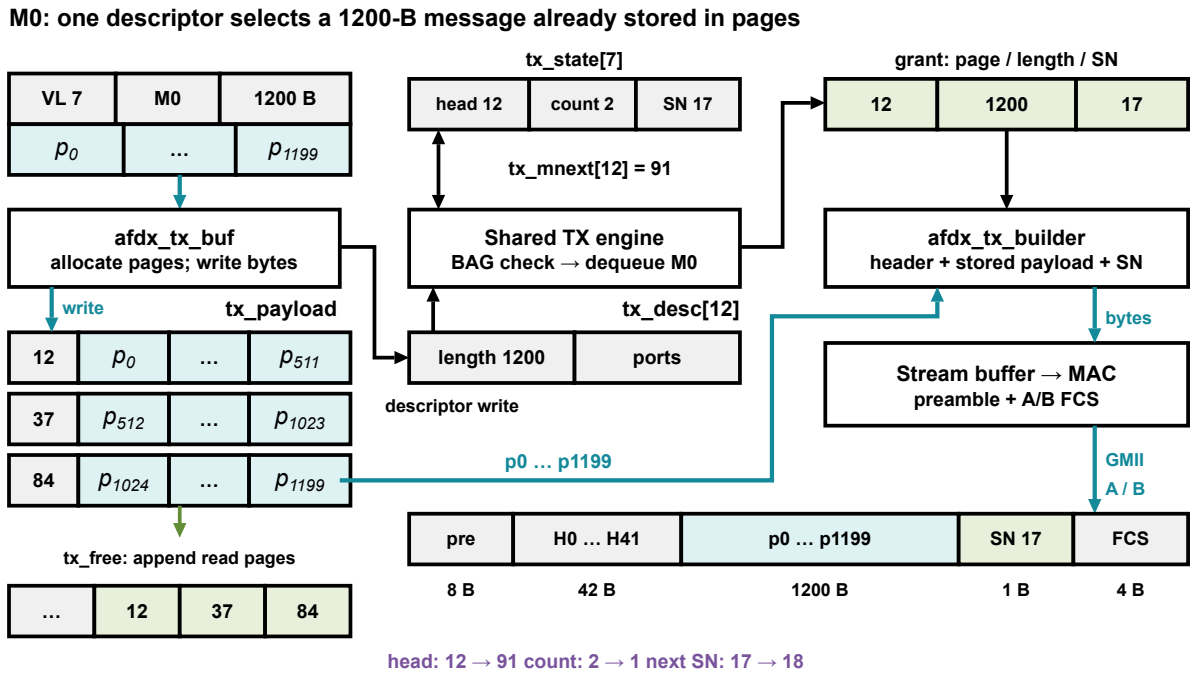


图 5-1 以 VL 7 的消息 M0 为例，1200 字节载荷存入页 12、37、84，描述符由首页号 12 索引。调度引擎读取描述符、队列状态与分片进度，授权组帧器读取载荷；整包末片出队后，队首沿 tx_mnext[12] 前移到 91。下方展示相同载荷在双网帧中的位置，以及读完后归还的页号

发送核心共设 10 组主存储表（表 5-1），其中载荷池采用带本地 27 位写捆绑与 LUT 写数据隔离的专用同步 RAM 模块 `afdx_tx_payload_store`（`afdx_tx_buf` 内另配 16×8 bit 单消息回滚页号表 `afdx_tx_rollback_ram`），其余各表均由 `afdx_ram_sdp` 实现，每张表的写口与读口均由单一模块独占。

表 5-1 发送核心配置、状态与分片表

表	深度 × 位宽	写口	读口	内容
`tx_cfg`	1024×32	配置总线	`tx_engine`	每条 VL 8 个字，地址 {VL[6:0], 字号[2:0]}
`tx_state`	128×63	`tx_engine`	`tx_engine`	`timer_active`、`next_time`、`sn`、`q_count`、`q_head`、`q_tail`
`tx_mnext`	256×8	`tx_engine`	`tx_engine`	同一 VL 队列中的下一条消息（首页号）
`tx_desc`	256×47	`tx_buf`	`tx_engine`	消息描述：14 位总长（1–8192 B）、`port_en`、源/目的端口
`tx_pnext`	256×8	`tx_buf`	`tx_builder`	消息内的下一页
`tx_payload`	131072×8	`tx_buf`	`tx_builder`	128 KiB 载荷池（`afdx_tx_payload_store`），地址 {页号 [7:0], 偏移 [8:0]}
`tx_free`	256×8	`tx_free`	`tx_free`	空闲页号 FIFO
`fragment_progress`	128×27	`tx_engine`	`tx_engine`	在途分片消息剩余字节数（14 位）与下一片 IP 偏移（13 位）

接下页

表 5-1 （续）

表	深度 × 位宽	写口	读口	内容
fragment_pointer	128 × 17	tx_engine	tx_engine	下一待发分片的起始页号（8 位）与页内字节偏移（9 位）
fragment_meta	128 × 142	tx_engine	tx_engine	首片锁存的 IP/UDP 头信息、单帧预算与共享 16 位 IP 标识（5 组 SDP）

tx_cfg 中每条 VL 占 8 个字，其中前 6 个配置字有效（表 8-2）。将配置字合并为一张 1024×32 的单表并由引擎在授权过程中逐拍顺序读取（表 5-5），可减少拆分为六张浅表时的块 RAM 容量浪费。afdx_tx_core 例化 7 张表和 5 个子模块，统一管控 tx_cfg 的写口与复位初始化：复位后完成 tx_cfg（1024 拍）、tx_state（128 拍）清零与空闲页池装满（257 拍）后置位 init_done，启动引擎和分页缓存；运行期间仅在 cfg_we 与引擎的 cfg_ok 同拍为 1 时寄存写地址与数据，并于下一拍写入配置表。

5.3 空闲页池

afdx_tx_free 管理载荷缓存的 256 个页号；每页从分配到归还期间均由一条消息独占。空闲页池采用 FIFO 结构，由一块 256×8 的 RAM、读写指针和计数器组成；复位后用 257 拍依次写入页号 0–255，装满后允许取页。

页池提供一个取页口和两个页面回收口，均带握手：

- 取页口 pop_req/pop_ready/pop_ack/pop_ok/pop_page 连接 tx_buf，采用请求应答握手；
- 页面回收口 1 ret1_* 连接 tx_builder，每读完一页便归还该页；
- 页面回收口 2 ret2_* 连接 tx_buf，用于回收被丢弃消息占用的缓存页。

取页由四步状态机完成，见表 5-2。从请求被接受到 pop_ack 共 4 拍，其中 2 拍用于同步 RAM 读出和寄存。

表 5-2 空闲页池取页状态

状态	动作	转移
POP_IDLE	pop_ready= 1；接受请求时若 count= 0 立即应答 pop_ok= 0，否则按 rd_ptr 发起 RAM 读取	有页 → POP_READ
POP_READ	等待同步 RAM 读出	→ POP_CAPTURE
POP_CAPTURE	寄存读出的页号	→ POP_REPLY
POP_REPLY	给出 pop_ack、pop_ok= 1 和页号	→ POP_IDLE

每个页面回收口先把页号锁存至各自的暂存寄存器（ret*_ready 表示该寄存器为空），再经共用写口逐个写入 FIFO 尾部；两个寄存器均满时页面回收口 1 优先。页面申请与释放同拍发生时 count 不变。

FIFO 深度等于页数，任一页号在池中至多出现一次。写口每拍可写入一个页号，两个页面回收口各自最快每 2 拍还一页。

5.4 分页缓存

afdx_tx_buf 在入口内置两槽位 49 bit 弹性寄存器级（afdx_tx_ingress_elastic），切断上游服务仲裁器与分页状态机之间的跨模块反压扇出；随后逐字节接收 1–8192 B 消息，按页写入 tx_payload，在 tx_pnext 中把同一消息的页串成单向链表，同时将本消息分配的每一页页号顺序记入 16×8 bit 回滚表（afdx_tx_rollback_ram），最后写 47 位 tx_desc 并向引擎发出入队请求。载荷缓存共 256 页、每页 512 字节，单条 8192 B 最长消息至多占用 16 页。

载荷写口采用本地寄存与写隔离：写使能、地址和数据统一锁存后送入 afdx_tx_payload_store，在 RAM 入口由本地寄存器与写数据隔离 LUT 驱动 M4K 块。常规末字节在 ENQUEUE 拍与描述符一同写入同步 RAM。发送核心启用 EARLY_ENQUEUE=1，在 DESC 拍提前给出入队请求；引擎仍须采样请求并读表，随后才可授权发送。

分页缓存的接口包括消息接口 m_*（表 C-1）；取页口和页面回收口 2；tx_payload、tx_pnext、tx_desc 的写口；入队请求应答口（表 C-2）；五个统计脉冲 ev_accept、ev_drop_dis、ev_drop_full、ev_drop_long、ev_drop_nopage。

5.4.1 消息接收与页分配

分页缓存的页链与 VL 消息队列见图 5-2，接收状态机各状态的动作见表 5-3。m_ready 经两槽入口弹性缓冲向上游呈现平滑流控。

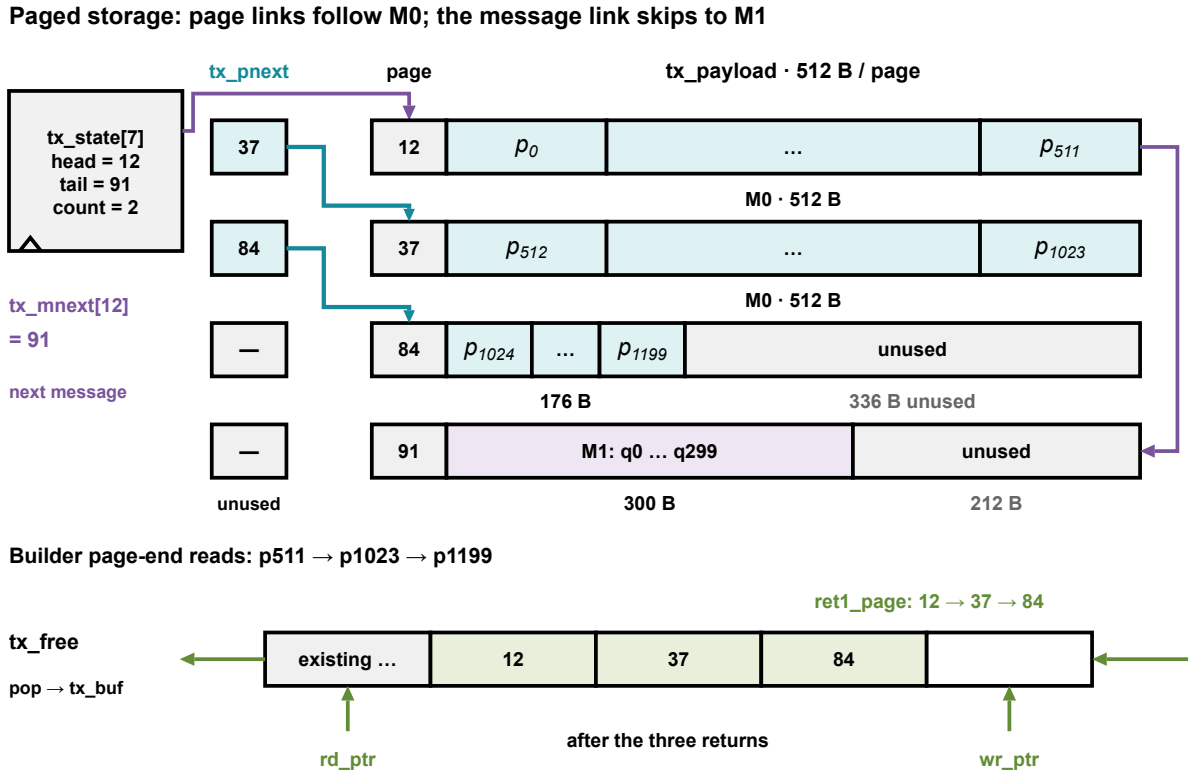


图 5-2 载荷 RAM 的稀疏页视图。M0 的 1200 字节分布于页 12、37、84，末页的 176 字节已用空间与 336 字节未用空间分开显示；青色连线按同索引的 tx_pnext 值访问下一页。紫色消息链从首页 12 跳到 M1 的首页 91，后者保存另一条 300 字节消息。下方为无并发取页时三次回收后的空闲 FIFO，读指针指向原有空闲页，写指针已越过新增的页号 12、37、84；终止页链的依据是消息长度

表 5-3 分页缓存各状态

状态	动作	转移
IDLE	接收首字节并锁存至 held_data，锁存 VL 与端口，长度记 1	→ ALLOC
ALLOC	置 pop_req	→ WAIT_PAGE
WAIT_PAGE	等 pop_ack；得页后记入 afdx_tx_rollback_ram，首页记为 enq_head，否则写 tx_pnext[上一页] = 新页；把暂存字节写到新页偏移 0	无页 → RETURN；暂存字节是末字节 → DESC；否则 → RECV
RECV	每拍收一字节写入当前页，偏移加 1；偏移到 511 时置 boundary	已满 8192 字节仍有数据 → RETURN；跨页字节暂存 → ALLOC；末字节 → DESC
DESC	写入描述符表 tx_desc[enq_head] = {14 位长度, port_en, 端口}	→ ENQUEUE
ENQUEUE	置 enq_valid	→ REPLY
REPLY	等 enq_ack，记下 enq_result	接纳 → EVENT；拒绝 → RETURN
RETURN	从 afdx_tx_rollback_ram 依次读出已分配页号，经页面回收口 2 全部归还	消息已结束 → EVENT；否则 → DRAIN
DRAIN	继续接收剩余字节并丢弃（不写入缓存）	末字节 → EVENT
EVENT	按原因产生一个统计脉冲	→ IDLE

图 5-2 中，消息 M0 的前两页各保存 512 字节，末页保存 176 字节；读取以消息总长与各分片长度为界，末页剩余空间不参与发送。tx_pnext[12]= 37 指向本消息的下一页，而 tx_mnext[12]= 91 指向同一 VL 的下一条消息；M0 全部分片发送完毕后，q_head 前移到 91。

写满一页后，将下一字节暂存于 held_data，取到新页后写入其偏移 0。跨页字节到达时才发起取页请求。8192 B 最大分片消息占 16 页，afdx_tx_rollback_ram 深度 256 足以覆盖任意页长参数下的完整页回滚。

5.4.2 时序与异常处理

以首字节被接收为第 0 拍：第 1 拍发取页请求，第 2 拍被空闲页池接受，第 6 拍 pop_ack 到达并写入首字节，第 7 拍回到 RECV。每次取页期间内部暂停 6 拍。接收消息末字节后，DESC 和 ENQUEUE 各占 1 拍，引擎在约 6 拍后返回入队应答，EVENT 再占 1 拍。

超长（第 8193 个字节到来）、取页失败与入队被拒（如单帧超 $L_{\max}$ 或队列已满）三种情况均转入 RETURN 态：通过 afdx_tx_rollback_ram 先归还本消息已占用的全部缓存页（最多 16 页），再继续接收并丢弃剩余字节。模块在输出该消息的统计脉冲前完成全部缓存页的零泄漏回收。reason 记录首次出现的原因，每条消息在 EVENT 状态产生一个统计脉冲。

5.5 调度引擎

afdx_tx_engine 是 tx_state 和 tx_mnext 的唯一读写控制模块，负责判定消息能否入队、按 BAG 选择待发帧、向组帧器授权，并根据实际起始时刻更新下一次允许发送的时刻。

5.5.1 队列状态与操作仲裁

tx_state 为每条 VL 保存一项 63 位状态，见表 5-4。每条 VL 的消息队列是一条单向链表：q_head、q_tail 存首尾消息的首页号，tx_mnext[首页号] 指向下一条消息，因此 128 条 VL 共用 256 项链表。

表 5-4 tx_state 字段

位	字段	含义
62	timer_active	next_time 有效；为 0 表示可立即发送
61:30	next_time	下一帧最早起始的 now 值
29:22	sn	下一帧的序号，复位为 0，之后 1–255 循环
21:16	q_count	排队消息数，不超过配置的 qmax（至多 63）
15:8	q_head	队首消息首页号
7:0	q_tail	队尾消息首页号

引擎一次只执行一种操作，图 5-3 展示优先选择、共享计数器和状态表写口的具体示例。在 IDLE 状态，引擎按“发送起始确认（START）> 入队（ENQ）> 授权（GRANT）> 扫描”的优先级选择操作，选中后由 phase 控制各步骤逐拍推进，完成后返回 IDLE（表 5-5，表中拍数不含 IDLE 响应的 1 拍）。

入队操作在入队请求有效且满足“已有候选”或“连续扫描满 16 条 VL”时触发（want_enq）；授权操作在已有候选、组帧器待发槽位就绪且无待确认起始帧时触发（want_grant）。无上述操作时，IDLE 每 2 拍扫描检查一条 VL。

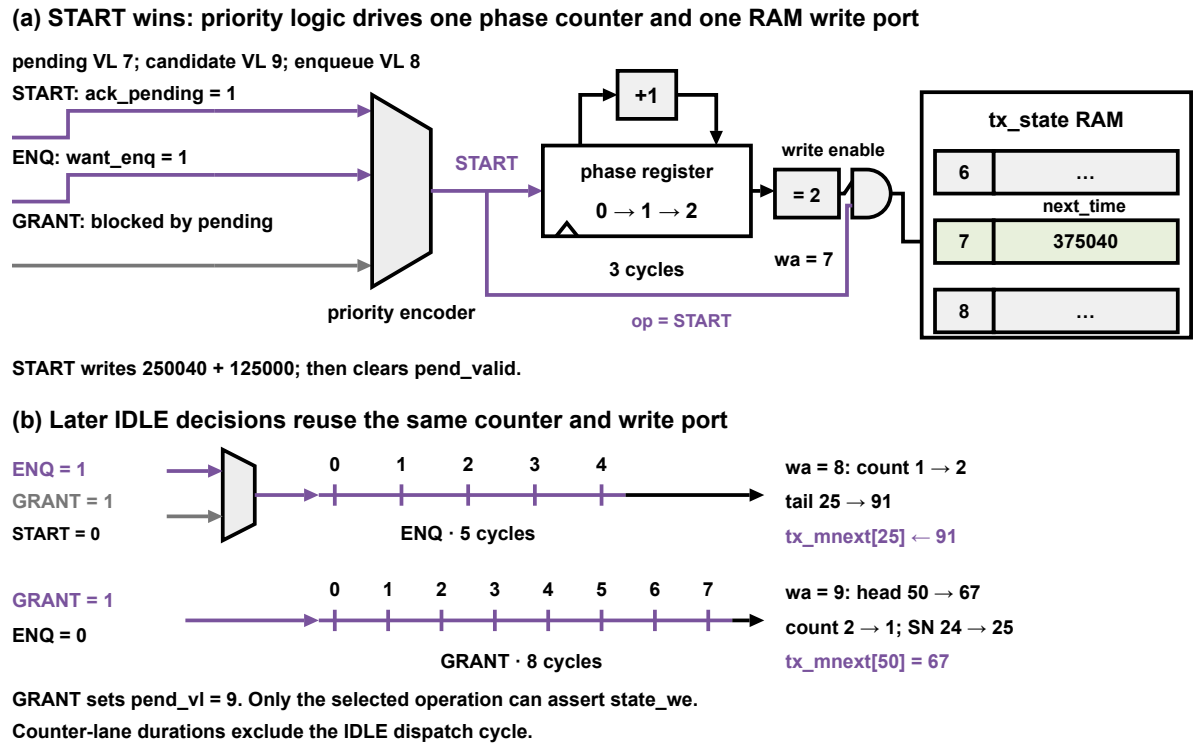


图 5-3 优先选择、带加一反馈的 phase 寄存器与单一状态表写口。(a) START 和 ENQ 同时请求，待确认起始帧使 GRANT 暂不可用；选中 START 后在 phase 2 写回 VL 7 的新期限并清除 pending。(b) 后续 ENQ 与 GRANT 继续复用计数器和写口，分别在末阶段写入 VL 8、9 的状态；刻度表示时间上的 phase 值，拍数不含 IDLE 响应拍，入队示例假定检查通过

表 5-5 调度引擎的操作

操作	触发	逐拍动作	拍数
START	组帧器报告 t_{start}（先锁存到 `ack_pending`）	0：读 `tx_state[pend_vl]`；1：保存；2：写 `timer_active`$=1$、`next_time`$=t_{\text{start}}+$`bag`，清 `pend_valid`	3
ENQ	`enq_valid`，且已有候选或扫描配额已满	0：读 `tx_cfg` 字 0 和 `tx_state`；1：保存；2：$d=$`now`$-$`next_time`；3：$d+512$ 并判定使能/长度/队满；4：接纳则写回 `q_count`$+1$、新队尾，非空队列另写 `tx_mnext`[旧队尾]，条件满足时定为候选	5
GRANT	有候选且 `grant_ready`	0–4：依次读配置字 1–5 及分片上下文表；2：读 `tx_desc`、`tx_mnext`[队首]；5–7：按 $L_{\max}$ 计算本片 IP 数据长、应用载荷长、起止偏移与剩余字节；8：发出 `grant_valid`，写回 `sn` 前进与分片进度（仅末片 `work_last` 时 `q_count`-1 并推进 `q_head`），记 `pend_vl`	9
扫描	无上述请求且无候选	每 2 拍发起一次 VL 状态读取，经 5 级流水判定	2/VL

5.5.2 入队、IPv4 自动分片与扫描

入队判定按顺序检查：VL 未启用或两网都未启用时返回结果码 1；当 $n \le 1471$ B 但大于配置的 `max_payload`（$L_{\max}$），或当 $1471 < n \le 8192$ B 但 $L_{\max} < 8$ B（无法构成 8 字节对齐的非末分片）时返回结果码 3（`drop_long`）；`q_count` 不小于 `qmax` 时返回 2；其余情况接受入队并返回 0。

对于 $n > L_{\max}$ 的长消息，引擎通过 fragment_progress、fragment_pointer 与 fragment_meta 三张 128 项表按 VL 独立维护在途分片上下文：

- 单片预算与 **8** 字节对齐：单帧最大 IPv4 载荷预算为 $B_{\text{ip}} = \min(L_{\max} + 8, 1479)$；若剩余待发 IPv4 载荷（首片含 8 字节 UDP 头，即 $n+8$）大于 B_{ip}，本片标记 grant_more=1（MF=1），IPv4 数据长度取 $\lfloor B_{\text{ip}}/8 \rfloor \times 8$（如 $L_{\max} = 1471$ B 时为 1472 B，首片含 1464 B 应用数据，中间片含 1472 B 应用数据），片偏移增量恰为 ip_data$/8$；末片（MF=0）直接发送全部剩余字节。
- 共享 **IP** 标识与逐片 **BAG/SN** 调度：首片授权时分配并锁存 16 位递增 IP 标识 next_ident 至 fragment_meta，同一消息后续分片复用该标识；每发送一片均独立递增该 VL 的 AFDX sn，并在组帧器读完本片字节后经 fragment_done 将下一片起始 {页号, 页内偏移} 写回 fragment_pointer，等待下一个 BAG 窗口到期后继续调度下一片，直至末片发出后才弹出队列头 q_head。在当前分片读完回写指针前，位图 scan_busy_banks 阻止该 VL 被提前重复选中。

扫描时每 2 拍发起一次 tx_state 读取，依次检查各条 VL，再通过 5 级流水作出判定。图 5-4 展示两条 VL 的判定与选中消息的发送示例。记 d =now−next_time，判据为：

- q_count$\neq 0$、(timer_active$= 0$ 或 d+LOOKAHEAD≥ 0)，且既不是 pend_vl 也无未回写的在途分片指针：定为候选，扫描停在下一条；
- q_count$= 0$、timer_active$= 1$ 且 $d \geq 0$：写回 timer_active$= 0$，防止 now 回绕约 34 s 后把旧时刻误判为将来；
- 判定完成时 scan_commit 指针前移至下一条 VL；任一操作插入时流水线作废，自 scan_commit 重新发出，并按更新后的状态重新判定。

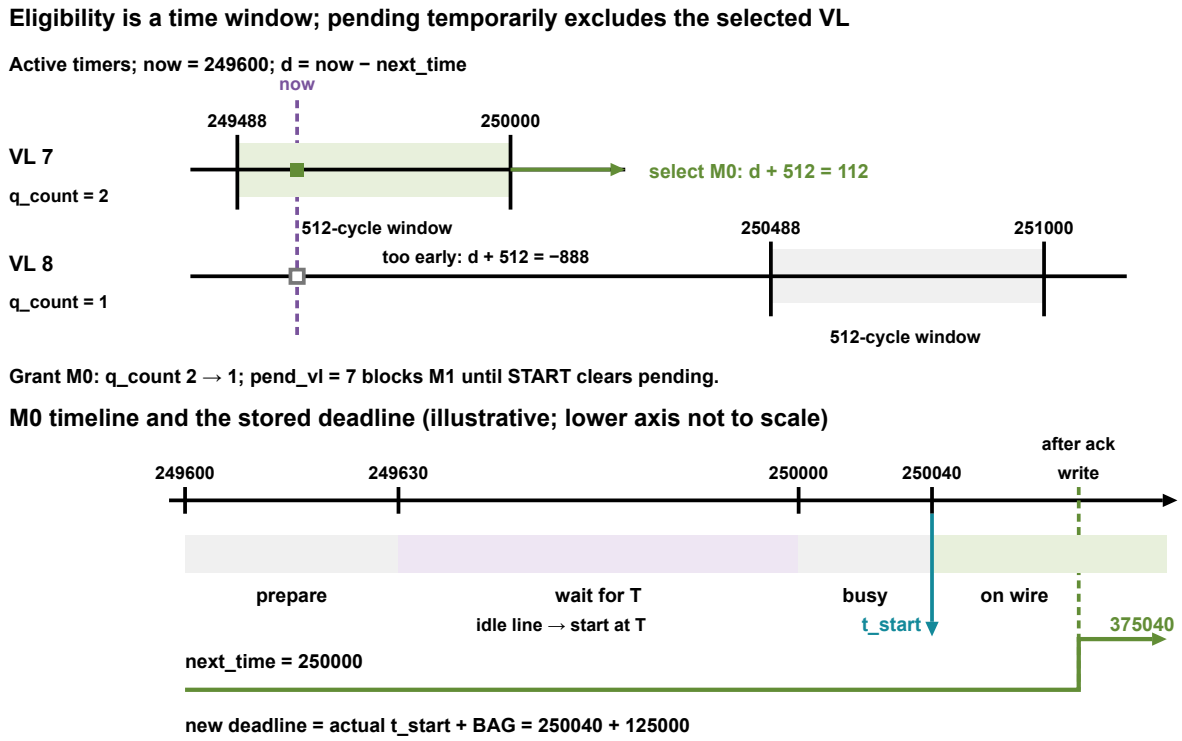


图 5-4 两条 VL 的资格窗口与 M0 的期限状态。定时器有效、now= 249600 时，VL 7 已进入其 512 拍提前准备窗口，VL 8 尚未进入；VL 7 授权后以 pending 阻止队内下一条消息重复入选。下方时间轴为不按比例的机制算例：M0 在 $T = 250000$ 前准备就绪，但因线路争用在 250040 起始；新期限按实际起始加 125000 拍 BAG 得到 375040，并在收到发送起始确认、执行 START 写回后才更新，阶梯不与起始拍重合。无定时约束时跳过 T 检查

扫描从上次选中 VL 的下一条继续，轮流检查全部 128 条 VL。入队操作仅在已有候选或扫描满 16 条（SCAN_QUOTA）时插入，使持续入队期间仍能推进扫描；无候选时，任一 VL 相邻两次被检查的间隔约 400 拍（第 3.7 节）。

5.5.3 授权状态与发送时刻

一帧从授权到真正起始之间，该 VL 的 next_time 仍是旧值。pend_vl 保存该帧的 VL，扫描和入队操作均不会再次将其选为候选；发送起始确认（TX Start Confirm）写回新的 next_time 时清除该标记。组帧器的单帧待发槽位对应一个 pend_vl 寄存器。

128 条 VL 轮询一遍需 256 拍，长于最短帧的 84 拍时隙；若到期后再启动扫描，发送时刻将随轮询位置浮动。引擎在到期前 LOOKAHEAD= 512 拍内选出候选并完成授权，组帧器提前完成长度和校验和计算，使 MAC 在 now 到达允许时刻 T 的当拍输出第一个前导字节。扫描回访约需 400 拍，再加上授权和准备所需的 22 拍，合计小于 512 拍，见第 3.7 节和图 3-2。

next_time 以实际起始时刻 t_{start} 加 bag 计算。线路争用使某帧晚发时，下一帧的允许时刻随之顺延，同一 VL 相邻两帧的起始间隔始终不小于 BAG。

授权阶段将当前 sn 锁存至组帧器，并将下一序列号写回状态表：复位后各 VL 首帧序列号为 0，随后按 1–255 循环递增。

5.6 组帧器

`afdx_tx_builder` 根据授权信息组装完整帧内容（目的 MAC 到 SN），在允许时刻送入 MAC，并向引擎报告实际起始时刻。

组帧器内部最多同时处理两帧：一帧在 `stream_active` 状态向 MAC 送字节，另一帧在 `pending` 指示的待发槽位中准备并等待。`grant_ready`= ¬`pending`，`pending` 在本帧的 `tx_sof` 有效时清除。因此，当前帧起始后即可接受下一帧授权，并在当前帧发送期间完成下一帧的准备。

5.6.1 帧头准备与载荷发送

组帧器的结构见图 5-5：帧头通过 336 位移位寄存器输出，载荷通过预读流水和 8 项字节 FIFO 输出，补零和 SN 则由计数器控制输出，三路数据经同一个选择器送入 MAC。

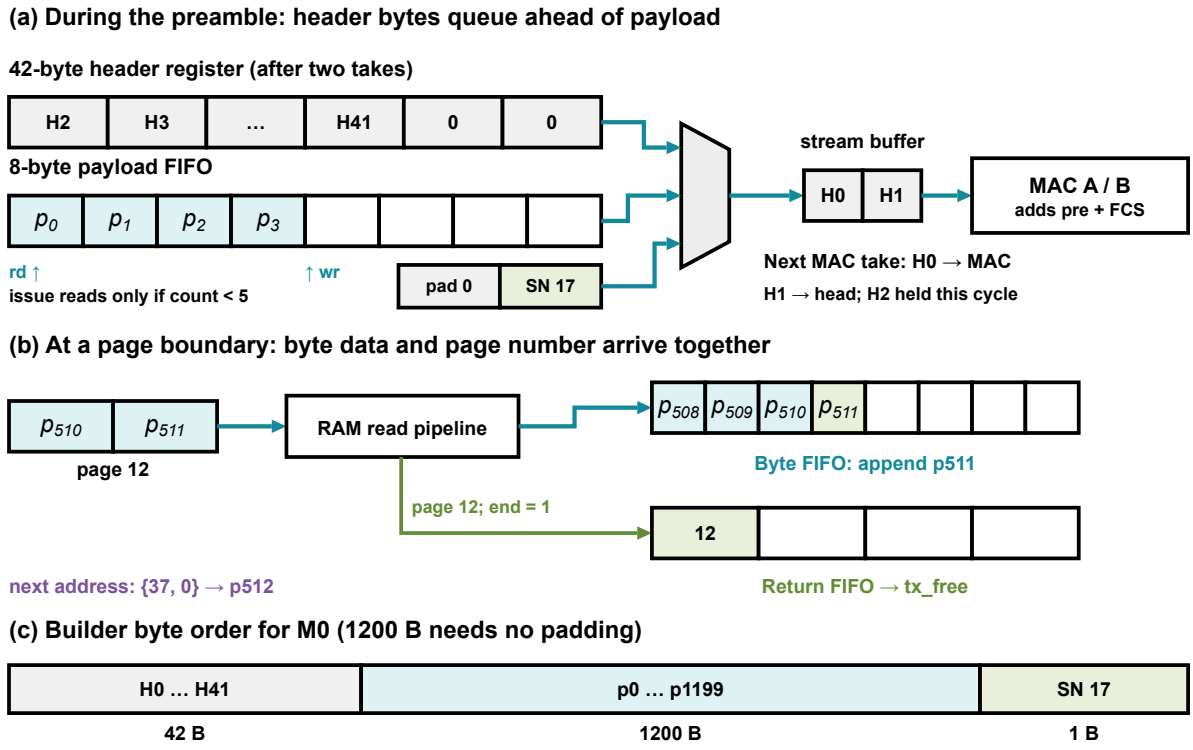


图 5-5 以 M0 的具体字节展示组帧器机制。(a) 前两个帧头字节已进入流缓冲，帧头寄存器已移位两次，载荷保留在 8 项 FIFO 中。下次 MAC 握手取走队首 H0，H1 前移；由于上游 ready 已寄存，该拍 H2 继续保持。(b) 页 12 的末字节 p_{511} 进入字节 FIFO 时，页号 12 同时进入独立的 4 项页面回收 FIFO；下一个页地址为 {37,0}，对应 p_{512}。FIFO 内容为逻辑队列顺序示例。(c) 1200 字节消息无需补零，帧头、载荷、SN 依次输出

组帧器在接受授权后锁存全部字段（含分片标志 `p_first`/`p_more`、片偏移 `p_ip_offset`、整包消息长度 `p_message_len`、共享 IP 标识 `p_ident` 与 `USER_ID`），随后经表 5-6 所示的 12 个准备步（`prep=0..11`）计算本帧 20 字节 IPv4 头校验和：每拍预锁存下一个 16 位加数（`sum_operand`），使单步加法器位宽不超过 16 位，彻底消除跨字段选择器的长进位链。

启动发送时，必须已置位 `prepared` 且上一帧已送完，同时满足以下任一条件：本帧无定时约束（`grant_timed`= 0，即该 VL 此前队列为空、允许立即发送）、允许时刻已过，或距离允许时刻至多 8 拍。

表 5-6 组帧器准备步骤（12 拍 IPv4 首部校验和流水）

prep	动作
0	IP 总长 $= n +$ (p_first?28 : 20)，UDP 总长 $= M + 8$；累加器置固定常数和初值（首片/单帧 0x462d，后续分片 0x4625），预取 p_len
1–7	依次以 16 位加法累加本片长度 p_len、IP 标识 p_ident、分片字段 {2'b00, MF, offset}、源 IP 高/低 16 位、目的 IP 高/低 16 位
8–9	两级进位折叠（first_fold → fold_low/fold_high）将高 4 位进位折回 16 位
10	按位取反锁存 16 位 IPv4 头校验和 checksum
11	待 header_template 稳定采样 checksum 后置位 prepared

为隔离组帧状态机与双网 MAC 发送器之间的跨模块路径，afdx_tx_core 在 afdx_tx_builder 与双路 afdx_mac_tx 之间插入了一级流缓冲（stream buffer 双缓冲解耦结构），并将 now 采样与到期作差拆分为多级本地寄存器（now_input → now_q → time_difference → due_∗ → time_due）。为精确补偿计时流水、载荷读返回与输出流缓冲引入的寄存拍数，afdx_tx_builder 的提前启动偏移配置为 8 拍（参数 START_LEAD_REGISTERED = 32'hfffffff8，即 -8）。由此，在线路空闲时，MAC 输出首个前导码字节的时刻与理论到期时刻严格对齐在 now$= T$ 当拍。

起始拍将协议头模板并行装入分组移位寄存器（14 组 $\times 24$ bit，避免宽总线全局使能扇出），A、B 两路共用，仅将 B 网的第 11 字节（网络接口标识）替换为 0x40。每当 MAC 取走一个字节，组帧器便按 part 推进输出：单帧或首分片先发送 42 字节以太网/IPv4/UDP 头（UDP 校验和固定置 0x0000），后续分片发送 34 字节以太网/IPv4 头；再发送 n 字节本片载荷；单帧/首片 $n < 17$ 时补零至 17 字节，后续分片 $n < 25$ 时补零至 25 字节；最后发送带 s_last 的 SN。tx_sof 所在拍的 now 作为 t_{start} 回报引擎。

从起始拍开始，只要 FIFO 中不足 5 字节，组帧器便自授权给定的起始 {页号, 页内偏移} 起每拍发起一次 tx_payload 读取；读请求寄存、同步 RAM 和本地返回寄存共 3 拍后写入 8 字节 FIFO。跨页时自动沿 tx_pnext 预取后继页号。读完一页（且非仍需保留描述符的首页 active_head）时，页号进入 4 项回收 FIFO 归还空闲页池；整条消息的首页 active_head 则保留至末分片（active_more=0）读完时统一归还，确保多片调度期间 tx_desc[head] 始终有效。

5.6.2 时序与异常处理

- **FIFO 容量**：发起读取时，FIFO 中至多有 4 字节，另有至多 3 个尚未返回的读请求，最多 $4 + 3 + 1 = 8$ 字节，等于 FIFO 深度。
- 载荷预读与供给时序：MAC 在起始后第 8 拍才取首字节（字节 0），协议头又占 42 拍（后续分片为 34 拍），第一个载荷字节在起始后第 50 拍（或第 42 拍）才被取走，而 FIFO 在起始后第 4 拍就开始填充；此后每拍补入一字节、取出一字节。下一页号在跨页前 500 拍以上已读出。
- 准备时间：下一帧或分片的帧头与校验和准备需 13 拍（锁存 1 拍加 prep=0..11 共 12 拍），可与当前帧的发送完全重叠；最短帧时隙为 84 拍。

在无发送争用且当前 VL 已满足 BAG 间隔时，从应用消息末字节提交完成到 GMII 首前导码字节 tx_sof 固定为 32 拍（0.256 µs，原为 31 拍 / 0.248 µs）；若按 1022 答疑 R95

规则计至以太网 SFD（`8'hD5`，前导码第 8 字节）结束，则发送处理时延 L_{tx} 典型为 39 拍（0.312 µs），上界为 100 拍；基准条件与逐拍分解见第 3.7 节图 3-2。

若 MAC 输出 `tx_underrun`，本帧记为错误帧，不产生 `ev_tx_a`/`ev_tx_b`；其余帧则在 `tx_eof` 有效时，分别为本帧启用的网络产生一个统计脉冲。

5.7 双网发送 MAC

`afdx_mac_tx` 用一个状态机驱动 A、B 两套 CRC 和输出寄存器，两网同拍输出。发送节拍由 MAC 控制，上游按其握手信号提供字节。图 5-6(a) 展示两网字节分别进入 CRC 累加器、形成 FCS 及相关状态转换，(b) 展示第 6.2 节接收 MAC 的四字节尾部寄存与 FCS 剥离机制。

相比每网独立例化一套带用户侧 FIFO 的通用三速以太网 MAC IP（相同器件下的资源对照见测试报告），共用单状态机和字节位置控制既能减少重复控制逻辑，也从结构上保证 A、B 两网副本在 MAC 输出寄存器端同拍起始。相应地，两网共用发送时隙，单网 VL 也按整帧占用该时隙，带宽按全部发送 VL 合并预算（第 8.9 节）。

发送状态机仅在 `byte_en` 有效时推进，帧间隔计数按 `clk` 递减；当前设计中 `byte_en` 在复位后恒为 1，发送时序按每拍一字节组织。各状态的动作见表 5-8，发送一帧的字节时间见表 5-7，L 为取走的字节数。MAC 在起始拍锁存 `s_en_a`、`s_en_b`；未启用的网络在整帧期间 `tx_valid` 与 `tx_data` 保持为 0；两网均未启用时状态机仍按相同节拍推进，使各帧时序与网络使能无关。上游持续有帧待发时，帧间隔恰为 12 拍。

数据阶段若上游未能及时提供字节（下溢 Underrun），MAC 输出 `tx_underrun` 脉冲，发出 4 字节不取反的错误 FCS，然后继续接收并丢弃上游这一帧的剩余字节。

复位期间线路输出全为 0，复位结束后先等待 12 拍帧间隔，再允许发送首帧。

表 5-7 MAC 发送的字节时间

字节时间	线上内容	本拍动作
起始前一拍	空闲	判断起始，锁存两网使能
0–6	`0x55`	
7	`0xD5`	取走首字节（字节 0）
8 至 $8+L-1$	字节 0 至 $L-1$	取走下一字节，取走带 `s_last` 的字节后停止
其后 4 拍	FCS	A、B 两网各自更新 CRC32
再后至少 12 拍	空闲	第 12 拍可判断下一帧起始

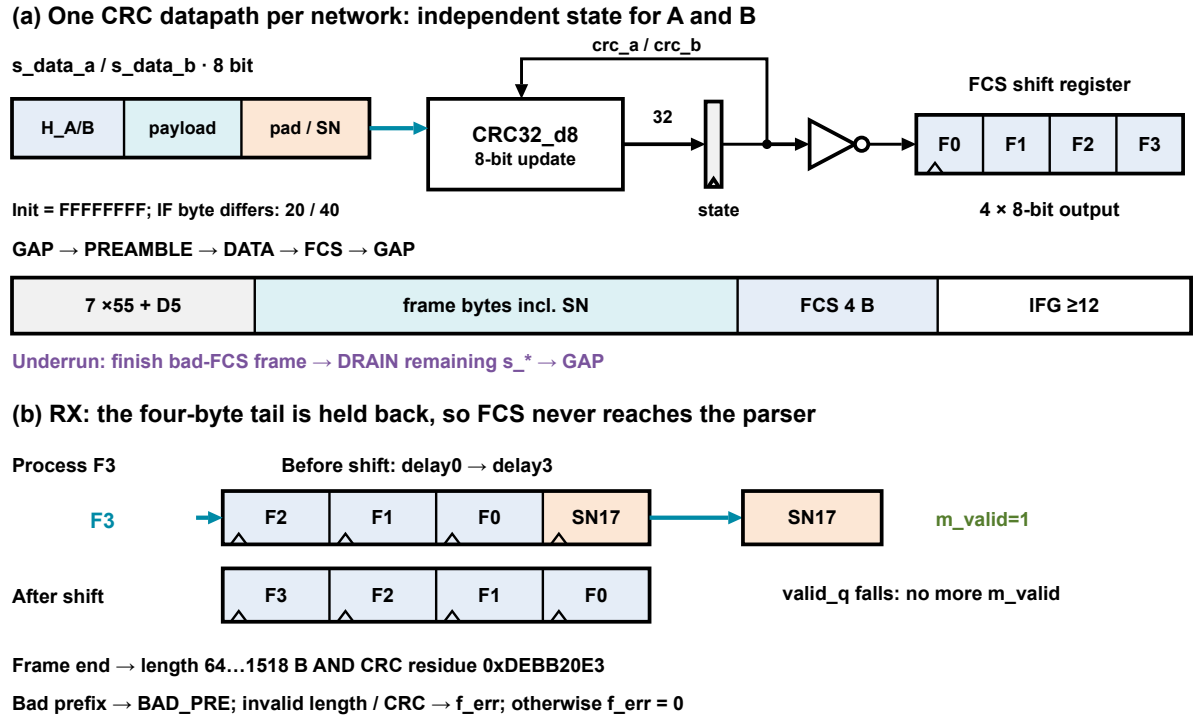


图 5-6 双网 MAC 的字节处理及状态转换机制。(a) A、B 两网帧头中的接口标识不同，分别更新独立 CRC 并生成 FCS；下方给出正常发送与下溢（Underrun）后的转换路径。(b) 接收侧保留最近四个字节：处理最后一个 FCS 字节时交付此前保存的 SN，帧结束后保留的四个 FCS 字节不再输出到解析器

表 5-8 发送 MAC 各状态

状态	动作	转移
GAP	输出 0；`gap_left` 递减	`gap_left`= 0 且 `s_valid` → PREAMBLE，同拍锁存两网使能、输出 `tx_sof`
PREAMBLE	输出 7 个 `0x55` 和 `0xD5`；末拍 `s_ready`= 1 取走首字节（字节 0）	取到字节 → DATA；未备好 → FCS（下溢 Underrun）
DATA	每拍输出一字节并更新两网 CRC	末字节已出 → FCS；未备好 → FCS（下溢 Underrun）
FCS	输出 4 字节 FCS，低字节先发；第 4 拍后撤 `valid`、输出 `tx_eof`，`gap_left`= 11	正常 → GAP；下溢（Underrun）→ DRAIN
DRAIN	`s_ready`= 1，继续接收并丢弃上游剩余字节	收到 `s_last` → GAP

第 6 章　接收通路

接收通路由接收核心 `afdx_rx_core`（含双网 MAC、帧与分片解析器、共享接收引擎、环形缓存交付模块与 16 路并发顺序分片重组引擎 `afdx_rx_reassembly`）以及顶层接收应用流包装器 `afdx_app_rx` 组成。MAC 在接收过程中逐字节校验 CRC，解析器同步检查以太网/IPv4/UDP（或后续 IPv4 分片）协议头并将载荷预写入本网环形缓冲。对于通过解析器检查的帧，接收引擎依次执行多 Bank VL 查表、配置检查、完整性检查（IC）、冗余管理（RM）以及分片重组槽位预约，再通知解析器提交数据或回退写指针；已提交的单帧由交付模块直通送出，分片帧则搬移至专用重组槽位完成顺序拼接与全报文校验，最终经 `afdx_app_rx` 加上 2 字节大端 VLID 前缀连续输出至 `rx_data`。

6.1　接收核心与配置表

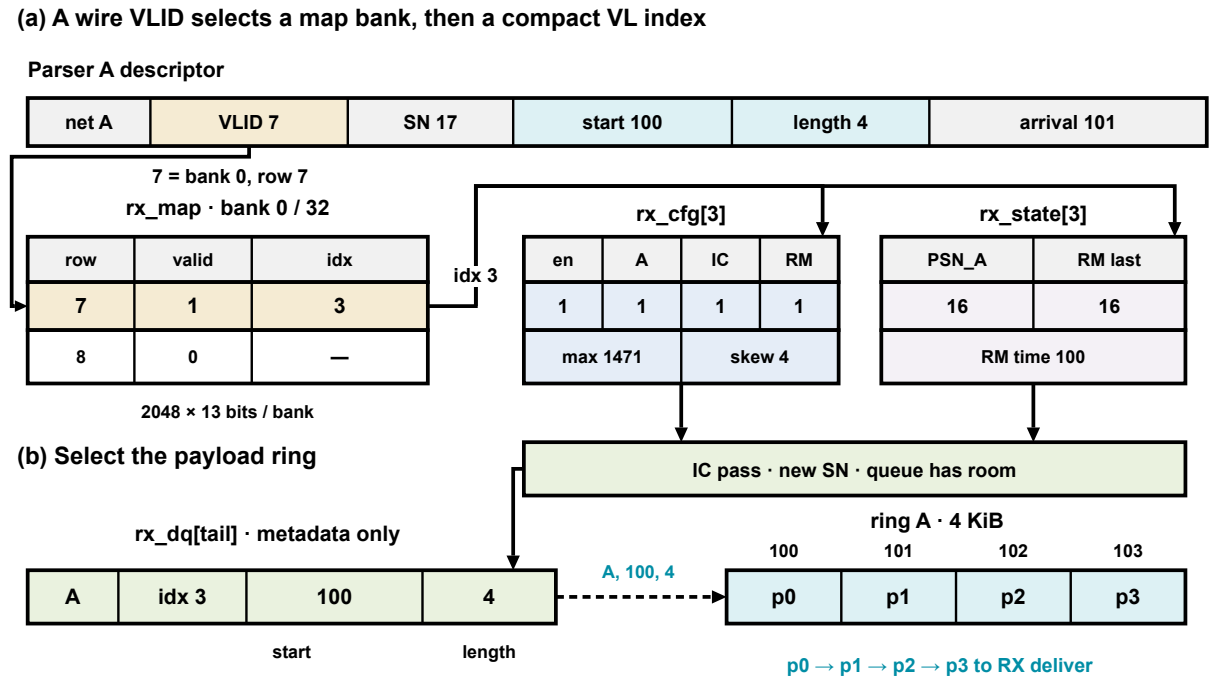


图 6-1　接收查表与载荷定位示例。A 网的 VLID 7 选择映射表 bank 0 的第 7 行，得到索引 3，随后读取该项配置和状态。接纳后，交付队列保存元数据（分片帧同时记录预约重组槽位号），通过网络、起点和长度定位 ring A 中的载荷字节；图中仅列出本例使用的字段

接收侧的片上存储表见表 6-1。为消除大容量存储表在大扇出布线与短路径写保持上的时序瓶颈，设计在物理结构上实施了三项精细拆分：

1. **`rx_map` 32-Bank** 双写分片与三级或树归并：逻辑规模 65536×13 bit 物理划分为 32 个 2048×13 bit 子库，每个子库进一步按写口拆分为 7 bit 与 6 bit 两块 `afdx_ram_sdp`（共 64 个实例），将单个写控制寄存器的物理 RAM 负载减半；写侧共享 16 路局部配置总线副本（`RX_CONFIG_BANKS=16`），读侧经“2-Bank → 4-Group → 最终 4-Group”三级寄存或树（OR-tree）归并，在 `FRAME` 第 5 拍稳定锁存映射结果。
2. **`rx_cfg` 与 `rx_state`** 窄位分片封装：`rx_cfg`（4096×32）拆分为 8 个 4 位 `afdx_rx_table_slice`，`rx_state`（4096×43）拆分为 10 个 4 位加 1 个 3 位 `afdx_rx_table_`

slice（内部调用 afdx_rx_reassembly_ram，采用“E0 读命令寄存 → E1 同步 RAM → E2 本地返回寄存”窄返回域结构）。

3. 扩展交付队列与双 **Bank** 重组存储：rx_dq 扩展为 32×73 bit（在原 68 位基础上新增 1 位分片标志与 4 位预约重组槽位号）；afdx_rx_reassembly 划分为 2 个独立 Bank（各 8 槽位），每 Bank 含 4 张专用元数据 RAM 与 8×4 条 8192×2 bit 载荷通道 RAM（总计 128 KiB）。

表 6-1 接收侧核心与分片重组存储表

表	深度 × 位宽	写口	读口	内容
rx_map	65536×13	配置总线	rx_engine	valid[12]、idx[11:0]，32 Bank ×2 写分片（64 块 SDP）
rx_cfg	4096×32	配置总线	rx_engine	8 组 4 位分片：en、net_a/b、ic_en、rm_en、skew_max、max_payload
rx_state	4096×43	rx_engine	rx_engine	11 组窄分片：每网 PSN 及有效位；rm_last、rm_time 及有效位
rx_ring_a/b	4096×8	本网 rx_parse	rx_deliver	A/B 网各 4 KiB 环形缓冲，按到达顺序暂存单帧/分片载荷
rx_dq	32×73	rx_engine	rx_deliver	已准入帧描述：分片标志、4 位重组槽位、来源网、环形起点、长度、idx、UDP 端口
reasm_meta	$2 \times 8 \times 258$	reassembly	reassembly	每 Bank 4 张 8 项表：key(92b)、header(92b)、admission(28b)、progress(46b)
reasm_payload	$16 \times 8192 \times 8$	reassembly	reassembly	16 槽位 ×4 通道 8192×2 bit 重组载荷池（128 KiB，带 128 个写隔离 LUT）
afdx_app_map	4096×16	配置窥探	afdx_app_rx	顶层接收包装器 VL_idx → VLID 反查表（另配 32 KiB 接收包装缓冲）

rx_cfg 中的 skew_max 以 now_c 的计时单位（32.768 µs）计。afdx_rx_core 例化 A、B 两路接收 MAC 与解析器、共用接收引擎、交付模块及 5 张表，其中 rx_map、rx_cfg 的写口由核心统一管控。复位后，核心用 65536 拍逐项清零 rx_map（前 4096 拍同步清零 rx_cfg），引擎另行清零 rx_state；全部完成后置位 init_done。运行期间仅在 cfg_ok 为 1 时锁存配置写请求并于下一拍写入配置表。核心将各模块的统计脉冲按位相或，寄存一拍后输出。

6.2 接收 MAC

afdx_mac_rx 把 GMII 输入寄存一拍后，在 rx_valid 为 1 的每一拍处理一个字节，FCS 剥离机制见图 5-6(b)。

- 帧起始定界（**SFD** 检测）：前导字节 0x55 的数量不限，检测到 0xD5 后以紧随的下一字节作为帧首字节。若收到其他值，或在检测到 0xD5 之前 rx_valid 已回到 0，则判为前导码错误。复位后若 rx_valid 已为 1，MAC 先等待其回到 0，再从下一帧开始接收。

- 剥离 **FCS**：每接收一字节均更新 CRC 和帧长；输出比输入晚 4 字节，收到第 $k+4$ 字节时输出第 k 字节，最后 4 字节（FCS）不送后级。
- 帧结束：寄存后的 `rx_valid` 首次为 0 时，MAC 判定帧接收结束，输出 `f_end` 和 `f_err`。错误判定依次检查前导码错误、帧长不在 64–1518 范围内、CRC 余式不等于 `0xDEBB20E3`，均无异常则判为正确帧。MAC 向后级最多输出 1514 字节；若帧长超限，则按帧长错误结束本帧。
- 帧间：`f_end` 之后立即回到空闲，可分辨间隔为 1 个低电平周期的两帧。

以 GMII 上 FCS 末字节到达时刻为第 0 拍：第 1 拍该字节锁存至输入寄存器，第 2 拍寄存后的 `rx_valid` 拉低，第 3 拍输出 `f_end`、`f_err` 和 `f_len`。剥离 FCS 后的最后一个有效字节在第 2 拍从 `m_data` 输出，比第 3 拍的 `f_end` 早 1 拍，解析器据此完成校验和累加。

6.3 帧头解析与预写入缓存

`afdx_rx_parse` 由字节位置计数器 `position` 驱动，随输入字节流式检查帧头；另设两个辅助状态机分别负责长度计算与帧尾分类，结构见图 6-2。

6.3.1 流式检查与长度计算

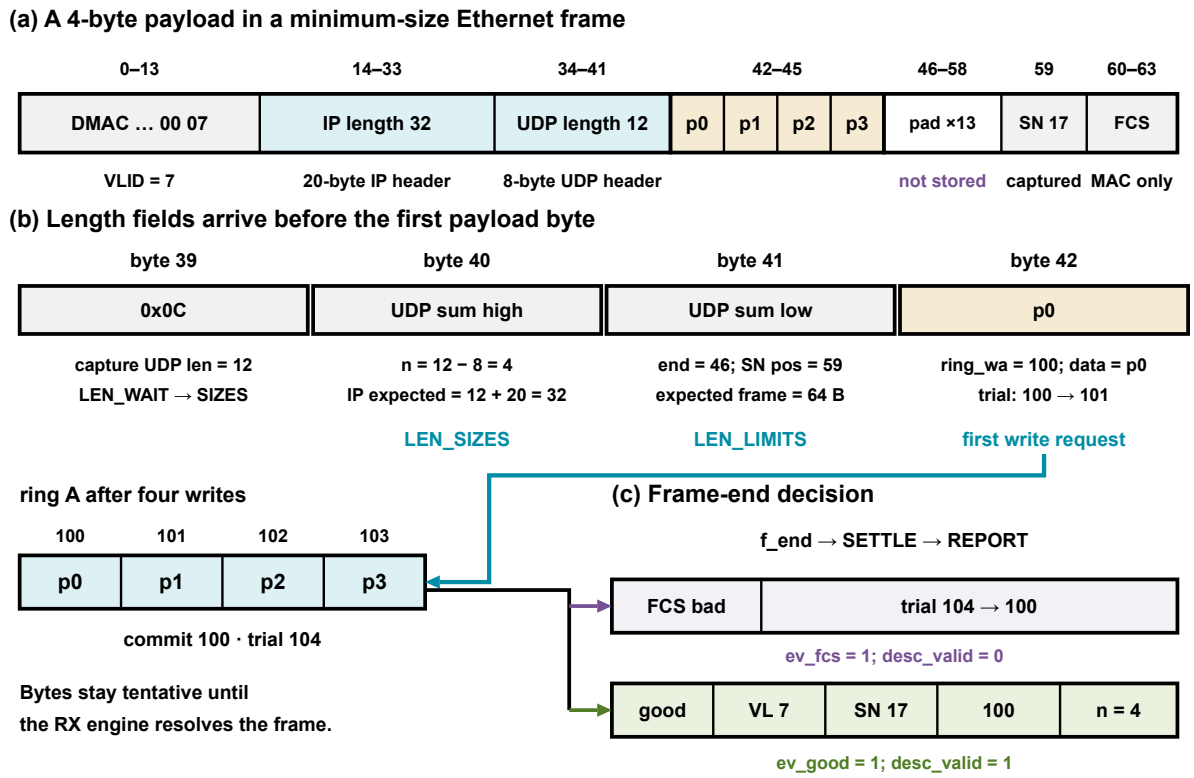


图 6-2 四字节载荷的解析示例：(a) UDP 长度 12、IP 总长 32，对应四个载荷字节、13 个补零字节和位置 59 的 SN；(b) 第 39 字节捕获长度，第 40、41 字节所在拍计算边界，第 42 字节发起首次载荷写请求；(c) 同一预写结果在帧尾按校验结果发布描述或回退。正确分支假定其余帧头检查均通过

帧头必须满足以下条件，任一条件不满足均判为“帧头错”：

- 第 0–3 字节为 `03 00 00 00`；第 12–13 字节为 `08 00`；第 14 字节为 `0x45`；第 23 字节为 `0x11`；IP 头 10 个 16 位字的反码和为 `0xFFFF`；IPv4 标志保留位为 0；
- 不分片单帧（**`MF=0`** 且片偏移 $= 0$）：记 n 为 UDP 长度减 8，要求 $1 \le n \le 1471$，IP 总长等于 $n + 28$，含 FCS 帧长等于 $47 + \max(n, 17)$；UDP 校验和非 0 时，伪头、UDP 头与 n 字节数据的反码和须为 `0xFFFF`；
- **IPv4** 首分片（**`MF=1`** 且片偏移 $= 0$）：第 34–41 字节为 8 字节 UDP 头，记整包应用消息总长 $M =$ UDP 长度 $- 8$（要求 $1 \le M \le 8192$），本片 IPv4 数据长 $d_{\text{ip}} =$ IP 总长 $- 20$ 须为非零 8 字节整数倍（本片应用载荷 $n = d_{\text{ip}} - 8$），含 FCS 帧长等于 $47 + \max(n, 17)$；
- **IPv4** 后续分片（片偏移 $\neq 0$）：无 UDP 头，本片应用载荷 $n =$ IP 总长 $- 20$（$1 \le n \le 1472$）；若 `MF=1`（中间分片），n 须为 8 字节整数倍；片偏移字节数加 n 不得越界，且含 FCS 帧长须等于 $39 + \max(n, 25)$。

补零字节的值不检查。解析器同时提取 VLID、IP 源/目的地址、16 位 IP 标识、分片标志与片偏移、UDP 头部字段（首片/单帧）、本帧有效载荷长度 n 和帧尾 SN。

单帧与首分片自第 42 字节起预写入本网环形缓冲，后续分片自第 34 字节起预写入本网环形缓冲。IP 校验和与单帧 UDP 校验和分别使用流水累加器每两字节累加一次；进位折叠采用两级寄存器流水，到帧尾后第 1 拍（FIN_SETTLE）稳定。

6.3.2 载荷预写入与提交

从第 42 字节开始，解析器将载荷边接收边写入本网环形缓冲，待最终判定后再决定是否提交。环形缓冲采用三个指针（图 6-3）：读指针 `rd` 由交付模块维护；已提交写指针 `wr_commit` 和预写入指针 `wr_trial` 由本网解析器维护。

计算剩余空间时以 `wr_commit` 为准，不计入正在接收的帧。第 42 字节到来时 n 已知，若剩余空间小于 n，解析器便不写入本帧载荷，并将其记为溢出。

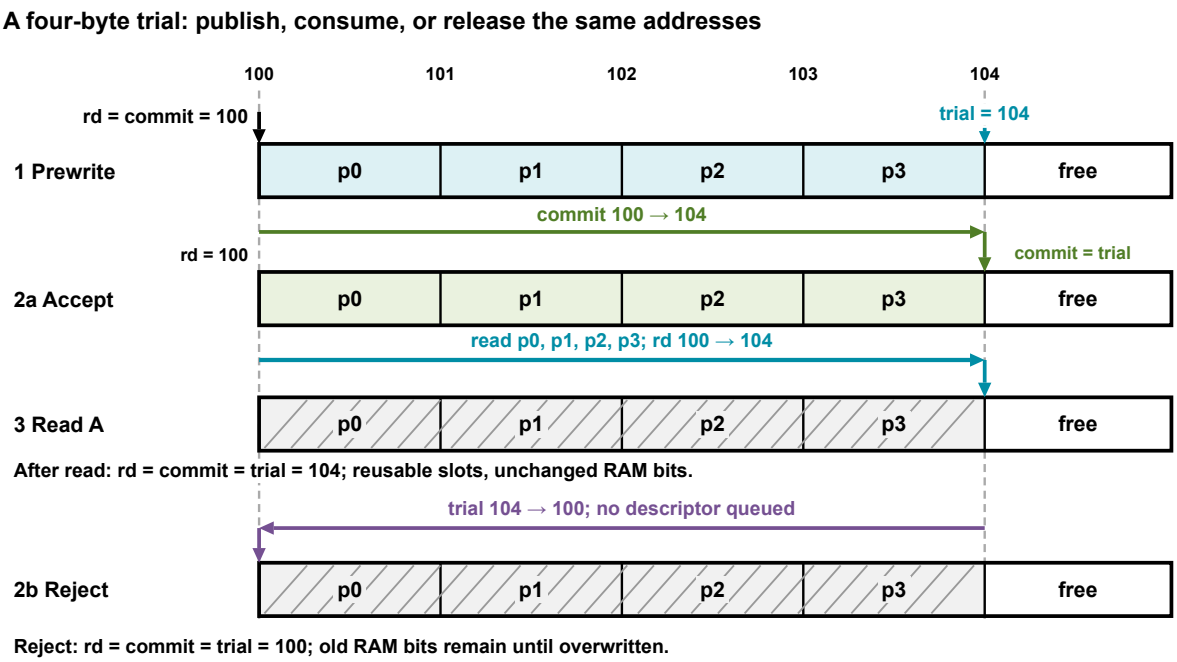


图 6-3 四字节载荷在同一地址坐标中的预写、接纳、读取与拒绝。接纳使 commit 从 100 前进至 104，逐字节读取再使 rd 前进至 104；拒绝是从预写状态出发的另一分支，使 trial 从 104 回退至 100。斜线表示空间可复用；读取和拒绝均不清除 RAM 中的旧位

6.3.3 帧尾分类与描述交接

每次 f_end 有效时，解析器均对该帧进行一次分类。finish_phase 在 f_end 后第 2 拍进入 FIN_REPORT 并作出判定，类别脉冲和帧描述均经寄存后输出，在第 3 拍有效。分类时，若 f_err 表示 FCS 错误或帧长错误（含前导码错误），则直接采用该结果；否则依次判定帧头错误、溢出或正确。

解析器在帧尾完成错误帧的分类与计数，并将预写入指针 wr_trial 回退至已提交写指针 wr_commit；正确帧则生成帧描述字交由接收引擎处理（表 C-4）。两网在同一拍结束的正确帧，其帧描述也在同一拍给出。

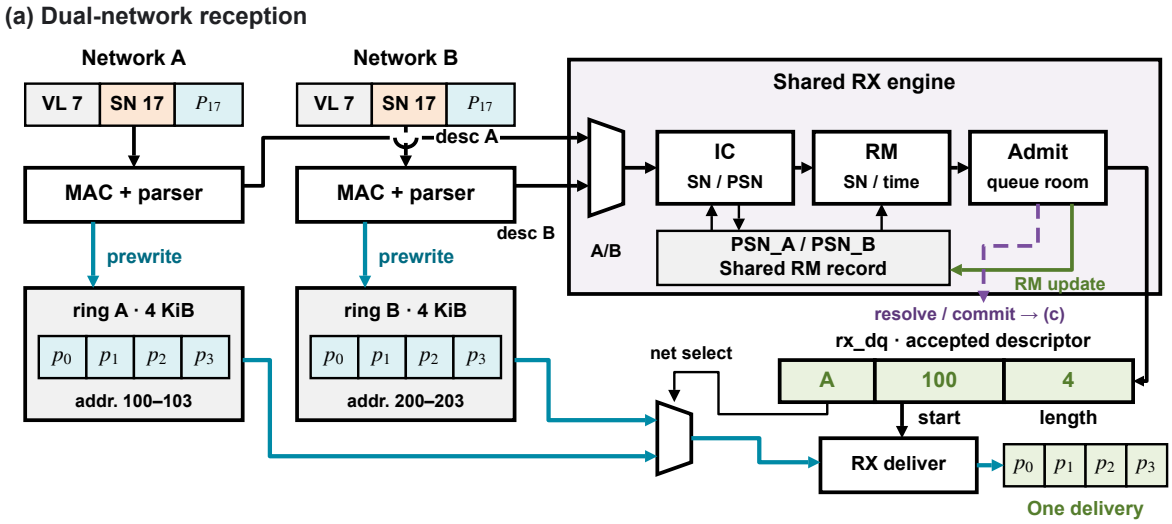
同一网相邻两帧的 f_end 至少相隔 73 拍（最短帧 64 字节加前导码 8 字节和 1 拍间隔）。引擎完整处理一份帧描述（含分片重组查询与原子预约）日程为 30 拍（未知 VL 与配置不符路径提前结束），两网各一份描述的服务时间合计至多约 62 拍，仍短于同一网相邻描述的最小到达间隔（73 拍）。若数据帧的首个前导字节在表初始化完成前到达，解析器将忽略整帧，不生成帧描述，也不计入统计。

6.4 完整性检查与冗余管理

afdx_rx_engine 是 rx_state 的唯一读写控制模块，一次处理一份帧描述或执行一步冗余记录过期清理，两网描述同拍到达时先处理 A 网。

6.4.1 帧描述处理流程

双网副本从预写入到一次交付的机制见图 6-4。引擎先查询 32-Bank rx_map，再并行读取窄分片 rx_cfg/rx_state 并向 afdx_rx_reassembly 发起槽位准入查询，随后依次执行配置检查（单帧/首片 $n \le$ max_payload，后续分片 $n \le$ max_payload $+8$）、逐帧完整性检查（IC）和双网冗余管理（RM），待重组槽位查询返回后原子提交 rx_state、rx_dq 与重组槽位预约。任一步未通过，均按对应原因计数并通知解析器回退预写入指针。



(b) Same VL / SN: process A, then B

Net	SN	IC: local PSN=16	RM record	RM result	commit	Queue write
A	17	Pass	16 → 17	New frame	1	{A, 100, 4}
B	17	Pass	17 → 17	Duplicate	0	—

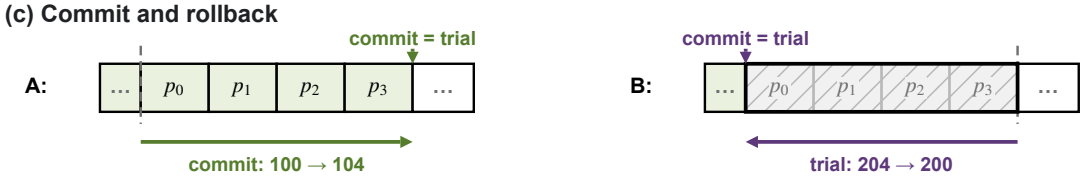


图 6-4　双网副本的一次交付机制。示例中 VL 7 的两份 SN 17 副本同时待处理，各网 PSN 和初始 RM 记录均为 16；配置有效、IC/RM 已启用、到达间隔在冗余窗口内，且交付队列有空位。A 网先通过并入队，B 网通过独立 IC 后被共享 RM 判为重复，回退其预写指针；地址仅为示例

引擎有 INIT、IDLE、FRAME、AGE 四个操作态。为适配 32-Bank 三级归并、窄分片状态表返回及双 Bank 分片重组 19 拍查询往返，FRAME 采用固定 30 拍日程（phase 0–30，每拍至多一级算术或比较逻辑，见图 6-5 和表 6-2）。未知 VL（第 7 拍跳至 30）、配置拒绝（第 12 拍跳至 30）与 IC 序号拒绝（第 14 拍跳至 29 写回 PSN）均支持提前结束；AGE 任一步收到新描述时立即让出并返回 IDLE。

表 6-2　接收引擎 FRAME 各阶段日程（30 拍流水）

phase	动作
（IDLE）/ 0	按 A 网优先取走帧描述（take），向 32-Bank rx_map 发出并行读请求
1–5	等待 32-Bank 三级或树读回流水，第 5 拍锁存映射结果 selected_map
6–7	有效映射触发 rx_cfg/rx_state 窄分片读取，并向 afdx_rx_reassembly 发出寄存查询（Q0=7）；无效映射在第 7 拍直接跳至 30 报告未知 VL
8–10	等待 afdx_rx_table_slice 本地返回（第 10 拍锁存配置字与本网 PSN 状态）
11–12	检查网络使能与载荷上限（后续分片允许 max_payload + 8），计算到达时间差绝对值、PSN 一/二步后继与模 255 序号距离；配置不符在第 12 拍跳至 30
13–14	第 13 拍完成完整性检查（IC）判定；第 14 拍若 IC 失败则跳至 29 仅更新 PSN，否则完成冗余管理（RM）判定 deliver_allowed

接下页

表 6-2 （续）

phase	动作
15–26	等待分片重组模块完成双 Bank 8 槽位匹配/空闲槽位搜索与准入校验（第 26 拍锁存 result_valid、reserve_ok 与 reserve_slot；单帧恒为可放行）
27–28	综合 deliver_allowed、rx_dq 余量与 reserve_ok 生成原子准入决策，若接纳分片帧则向目标重组 Bank 发出 reserve 预约命令
29	原子写回 rx_state（PSN 与 RM 记录）及 32 × 73 bit 交付队列 rx_dq
30	输出 resolve 与 commit 脉冲，产生对应统计事件（ev_deliver/ev_rm_drop/ev_a/b），返回 IDLE

6.4.2 配置与完整性判定

rx_map 映射无效时，引擎将该帧计为“未知 VL”；VL 未启用、本网未启用，或单帧/首片 $n >$ max_payload（后续分片因省去 8 字节 UDP 头允许 $n \leq$ max_payload $+ 8$）时，计为“配置不允许”。

完整性检查（ic_en$= 1$ 时）每网独立进行。记本网上一帧的序号为 PSN，满足以下任一条件即可通过检查：表初始化后本网首次收到该 VL；SN 为 0（对端复位）；SN 为 PSN 在 1–255 环上递增 1 步或 2 步的后继序号，后者允许丢一帧。PSN 为 0 时，后继序号分别为 1 和 2。

图 6-5 中，本网 PSN、共享 RM 记录和交付队列各有独立的写使能。

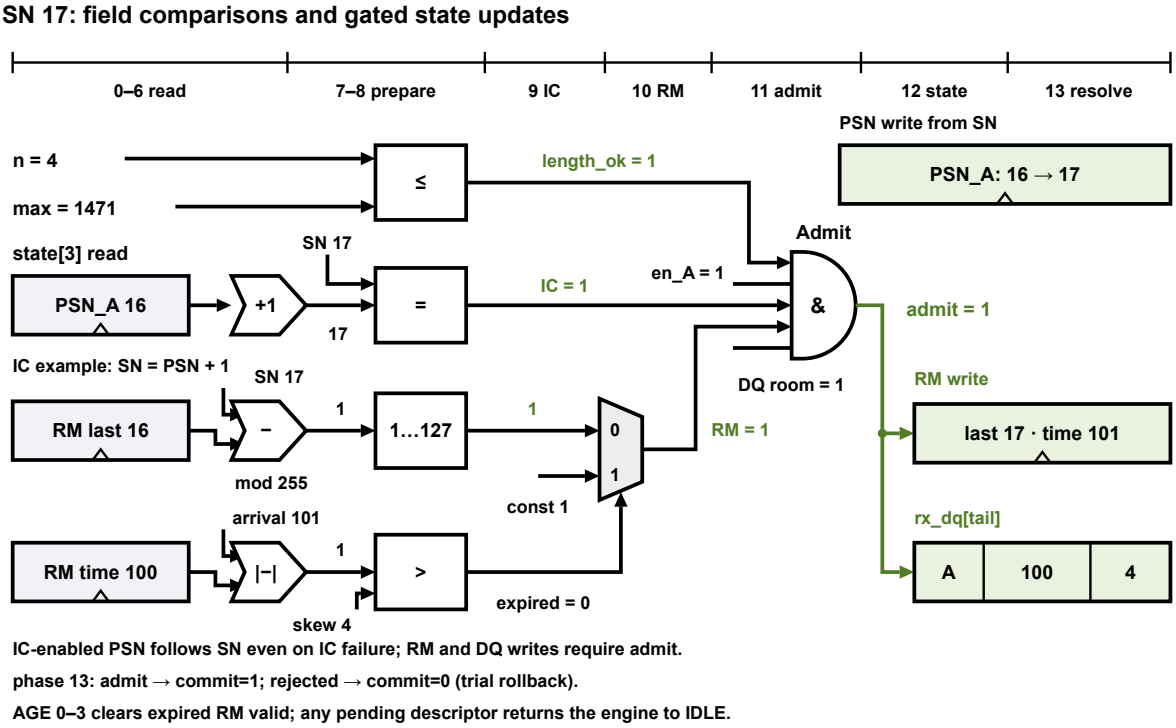


图 6-5 一份 SN 17 描述的比较与写使能通路。IC/RM 均启用且状态有效，本例 SN 恰好匹配 PSN 的一步后继；RM 以模 255 序号距离和到达时间差判新，窗口过期时由 MUX 直接放行。门控结果使能 RM 记录和交付队列写入；本网 PSN 的更新独立于最终接纳结果。上方标尺给出核心判定逻辑段

启用完整性检查时，通过映射与配置检查的帧均以当前 SN 更新本网 PSN，包括序号检查失败的帧；后续序号连续时可恢复同步。未通过序号检查的帧计为“完整性检查丢弃”。

引擎先对每份副本独立进行完整性检查，通过后再进行冗余管理，符合 ARINC 664 P7 的处理顺序。

6.4.3 冗余窗口与过期清理

冗余管理（`rm_en`$=1$ 时）按处理顺序交付首个有效副本；两网描述同拍待处理时先处理 A 网。副本到达间隔在 `skew_max` 内时，同一单帧或分片只交付一次，判据如下：

- 尚无记录，或到达时刻与 `rm_time` 之差超过 `skew_max`：交付；当两份副本的处理顺序与到达顺序相反时（如引擎忙期间 B 网先完成解析、A 网后完成解析，引擎转入空闲时按 A 网优先仲裁先处理了 A 网），对 16 位时间差取绝对值。
- 否则 SN 等于 `rm_last`：另一网的重复帧，丢弃并计“冗余管理丢弃”。
- 否则 SN 为 0 或 `rm_last` 为 0：交付。
- 否则在 1–255 环上，SN 领先 `rm_last` 1–127 步则交付，其余丢弃。

对于判定可交付的帧，若 `rx_dq` 已满或分片重组准入拒绝（如缺首片、片偏移不连续、IP/UDP 长度矛盾或 16 个槽位全满），则计入本网 `ev_a/b[7]` 异常/溢出脉冲并回退环形缓冲预写入指针；否则提交环形缓冲中的载荷，将 73 位交付项写入 `rx_dq`，并更新 `rm_last` 和 `rm_time`。因此，逐帧 **FCS/IPv4** 校验、独立 **IC** 与双网 **RM** 去重均在进入分片重组前完成，同一分片的冗余副本不会重复进入重组槽位。

`now_c` 每变化一次，引擎在空闲时通过 AGE 状态的 5 个步骤检查下一条 VL；有帧描述到来时立即返回 IDLE 处理帧，之后重试该项过期清理检查。判据为：若记录已超过 `0x4000` 个计时单位（约 0.537 s），清除 `rm_time_valid`。

帧描述处理优先于过期清理，清理被打断后会重新检查同一项。4096 项约需 0.134 s 检查一遍，记录在约 0.671 s 内失效，早于 16 位时间差的半圈时间 1.074 s。

6.5 交付与 16 路并发顺序分片重组

接收后端由环形缓存读出模块 `afdx_rx_deliver`、双存储体分片重组引擎 `afdx_rx_reassembly` 与顶层接收流包装器 `afdx_app_rx` 串联构成：

1. 环形缓存读出（**`afdx_rx_deliver`**）：按入队顺序从 32×73 bit 块 RAM 队列 `rx_dq` 取出帧描述（图 6-6），在下游 `afdx_rx_reassembly` 就绪时以每拍 1 字节从 `rx_ring_a` 或 `rx_ring_b` 连续读出载荷并立即推进读指针释放 4 KiB 环形缓冲空间。
2. 双存储体 **16** 路顺序分片重组（**`afdx_rx_reassembly`**）：按接收 VL 索引最低位划分为 2 个独立存储体（`afdx_rx_reassembly_bank` $\times 2$），每体含 8 个并发重组上下文槽位（全芯片共 16 路并发上下文）。每个存储体维护 4 张 8 项专用同步 RAM 元数据表（`key` 8×92：{VL, 源/目的 IP, 16 位 IP 标识}；`header` 8×92：首片锁存的 UDP 端口、期望总长 M 与校验和种子；`admission` 8×28：准入期望偏移；`progress` 8×46：写进度、运行 UDP 校验和与 2 s 老化时间戳），并配备 8 个载荷槽位（`afdx_rx_reassembly_payload_slot`，每槽位由 4 条 8192×2 bit 通道 RAM 交

织构成 8 KiB 容量，全芯片共 128 KiB，所有 256 个 RAM 数据输入端均经 128 个 `lut_mask="8888"` 组合隔离 LUT 门控以消除短路径保持时间违例）。

3. 严格顺序拼接、校验与老化清理：不分片单帧直接透传输出至 `d_valid/d_data/d_last`。对于 IPv4 分片帧，首分片（`offset=0, MF=1`）分配空闲槽位（若同 VL 同标识已有残缺旧上下文则先覆盖重置），后续分片严格要求当前片偏移等于槽位期望偏移（`offset == expected`，拒绝缺首片、乱序、重叠或总长不符的分片）；载荷逐拍写入对应槽位并流式累加 UDP 伪头与数据反码和。当末分片（`MF=0`）写完且累计长度恰等于首片声明的 M、同时（若首片携带非零 UDP 校验和）全报文校验和等于 `0xFFFF` 时，状态机按每拍 1 字节连续输出完整重组消息并释放槽位；若末片总长或非零 UDP 校验和不符，或驻留超过 `TIMEOUT_CYCLES = 250,000,000` 拍（125 MHz 下为 2.0 s）仍未收齐，硬件立即废弃该槽位并在对应来源网上输出 `reassembly_reject`（映射至 `ev_a/b[7]`）。
4. 顶层接收应用流包装器（**`afdx_app_rx`**）：核心交付的非服务应用消息经顶层一级寄存器（`delivery_*`）打拍后写入 32 KiB 分体缓冲（结构同 `afdx_app_payload`），同时以 12 位接收 VL 索引查询 `afdx_app_map`（4096 × 16 bit）还原 16 位大端 VLID，在 `rx_valid` 连续高电平区间前置 2 字节 VLID 后无反压输出整条消息（消息间至少空 1 拍）。

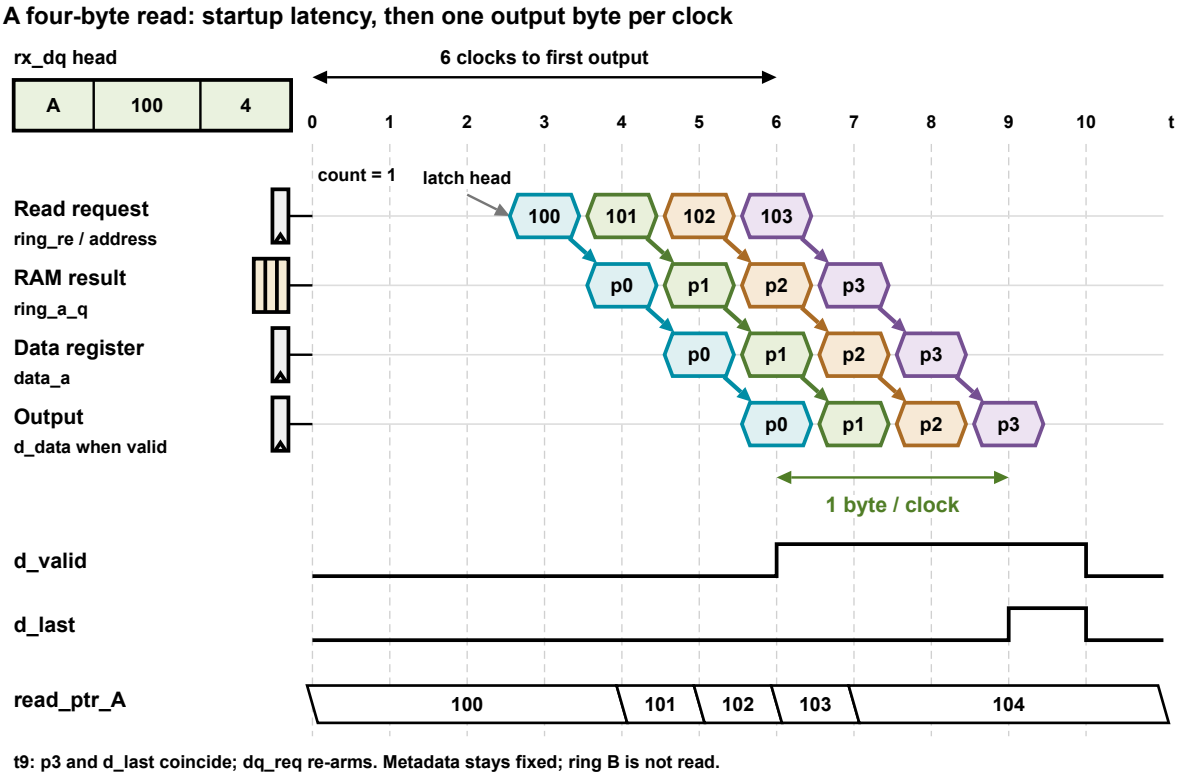


图 6-6 四字节消息的交付时空图。每条斜线跟踪同一字节从读请求经同步 RAM、数据寄存到输出的逐拍传播；总线值段标出各拍的读地址或字节，行首图元表示真实 RAM 或寄存器。队列计数在第 0 拍变为 1，第 6 拍出现首字节，随后每拍连续输出一个字节；有效、末字节和读指针波形共用同一时间轴，区分启动延迟与稳态吞吐

核心单帧无争用时，从帧尾 FCS 结束到核心交付口首个 `d_valid` 典型延迟仍保持低延迟流水，设计上界为 200 拍；多片长消息则在末分片写入完成并校验通过后连续每拍 1 字节输出整包（最长 8192 B 连续输出 8192 拍）。

6.6 统计分类

对于表初始化完成后开始接收的每一帧（一段 GMII rx_dv 连续为 1 的字节流），解析器均在帧尾离开线路后的 32 拍内，在对应网络上输出且仅输出一个类别脉冲。对于分类为正确的帧，接收引擎还输出一个处理结果脉冲；重组引擎在老化超时或末片全包校验拒绝时在对应来源网上补充输出拒绝脉冲。每网的 ev 位定义见表 6-3。

表 6-3 接收统计脉冲 ev_a、ev_b

位	名称	产生模块	含义
0	正确帧	rx_parse	通过 MAC 和单帧/分片帧头检查
1	FCS 错误	mac_rx/rx_parse	CRC32 校验不符
2	帧长错误	mac_rx/rx_parse	帧长越界（<64 或 >1518 B）或前导码错误
3	帧头错误	rx_parse	固定字段、IP/UDP 长度、8 字节对齐或校验和错误
4	未知 VL	rx_engine	rx_map 未映射
5	配置不允许	rx_engine	VL 或本网未启用，或单帧超过 max_payload（后续分片超过 max_payload $+8$）
6	完整性检查丢弃	rx_engine	SN 不连续
7	溢出 / 重组拒绝	三处相或（0x80）	环形缓冲满、rx_dq 满，或分片重组准入拒绝（缺首片、乱序、重叠、槽位满、末片全包长度/UDP 校验和错误、2 s 老化超时）

位 7（掩码 0x80）由解析器环形缓冲溢出、接收引擎 rx_dq/重组准入拒绝以及 afdx_rx_reassembly 后端校验/超时拒绝三路脉冲相或产生。两网合计另有 ev_rm_drop（冗余管理丢弃）、ev_deliver（引擎准入交付）及顶层包装器 ev_app[2]（接收包装缓冲溢出）。

第 7 章　服务功能

RTC、SNMP 和 615A 均通过 UDP 报文通信，经服务通道 `afdx_svc` 与应用共享收发核心。三类服务报文均绑定独立配置的发送 VL，与用户消息统一参与 BAG 调度、完整性检查和冗余管理。

7.1　服务通道

`afdx_svc` 由组合选择逻辑和状态寄存器组成：发送侧为 `busy`、`owner`、`turn` 以及授权向量寄存器 `served[3:0]`，接收侧为 `rx_active`、`rx_claimed`，结构见图 7-1。

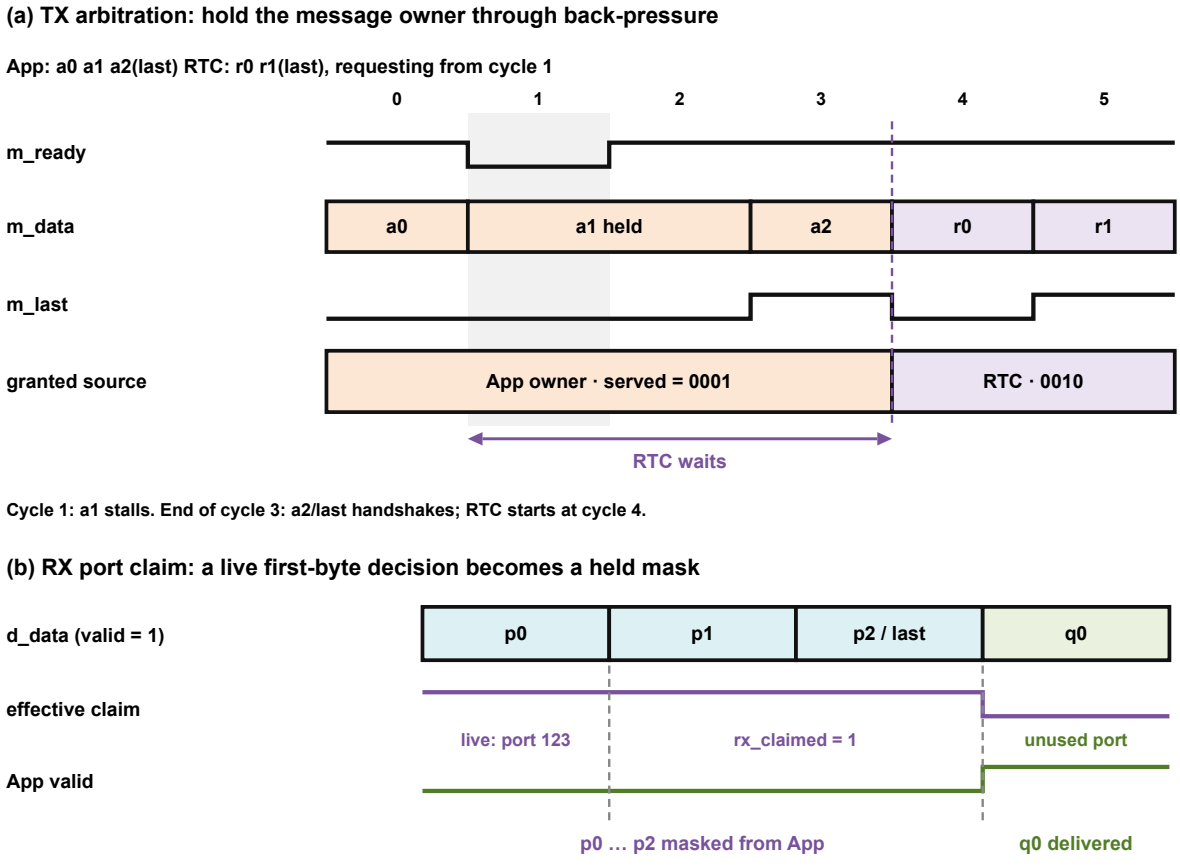


图 7-1　服务通道示例：(a) 应用消息未结束时保持授权，RTC 消息等待；(b) UDP 端口 123 在首字节认领并屏蔽应用交付

发送仲裁器以整条消息为单位，依次轮询应用、RTC、SNMP 和 615A 四个来源。选中的来源独占消息通路，直至末字节完成握手，随后从下一个来源开始轮询。服务发送口均空闲时，应用与发送核心之间的数据和 ready 信号直连。

为切断四个服务源发送状态机到发送核心 `afdx_tx_core` 注入多路器之间的跨模块组合长路径，`afdx_svc` 对仲裁授权向量进行了一级寄存（`served[3:0]`），实现仲裁决策与多路数据选通的时序解耦。发送仲裁器由状态寄存器构成：`busy` 指示当前消息传输忙，`owner` 锁存当前来源，`turn` 指示下一轮轮询起点；末字节握手时，仲裁器清零 `busy`，

并将 turn 更新为下一个来源。因此，任一来源提出请求后，至多等待其余三个来源各传完一条消息。

接收核心的交付数据同时广播给应用和三个服务（图 7-1(b)）。三个服务使用互不冲突的 UDP 端口，均不对交付口施加反压。首字节到达当拍，各服务根据 d_dport 经组合逻辑译码产生端口匹配信号，一方面直接门控首字节的应用口 valid，另一方面锁存至 rx_claimed 并在后续字节期间（rx_active= 1）持续保持。因此，匹配服务端口的消息由对应服务接收，应用口的 valid 和 last 全程保持为 0；其余消息原样交付应用，并在顶层 AFDX_top 入口经专用边界寄存器组（delivery_∗）打拍隔离后再驱动接收包装器 afdx_app_rx 的片上 RAM。

7.2 RTC 时钟同步

afdx_rtc 维护由 32 位秒寄存器和 32 位纳秒寄存器构成的本地实时时钟，每拍递增 8 ns，纳秒值满 10^9 时向秒进位。秒计数以 2000-01-01 00:00:00 UTC 为起点；发送报文中的 NTP 秒值在此基础上加 3155673600。时钟同步遵循 RFC 4330 标准 SNTPv4 客户端/服务端协议，交互时序与四时间戳原理见图 7-2。

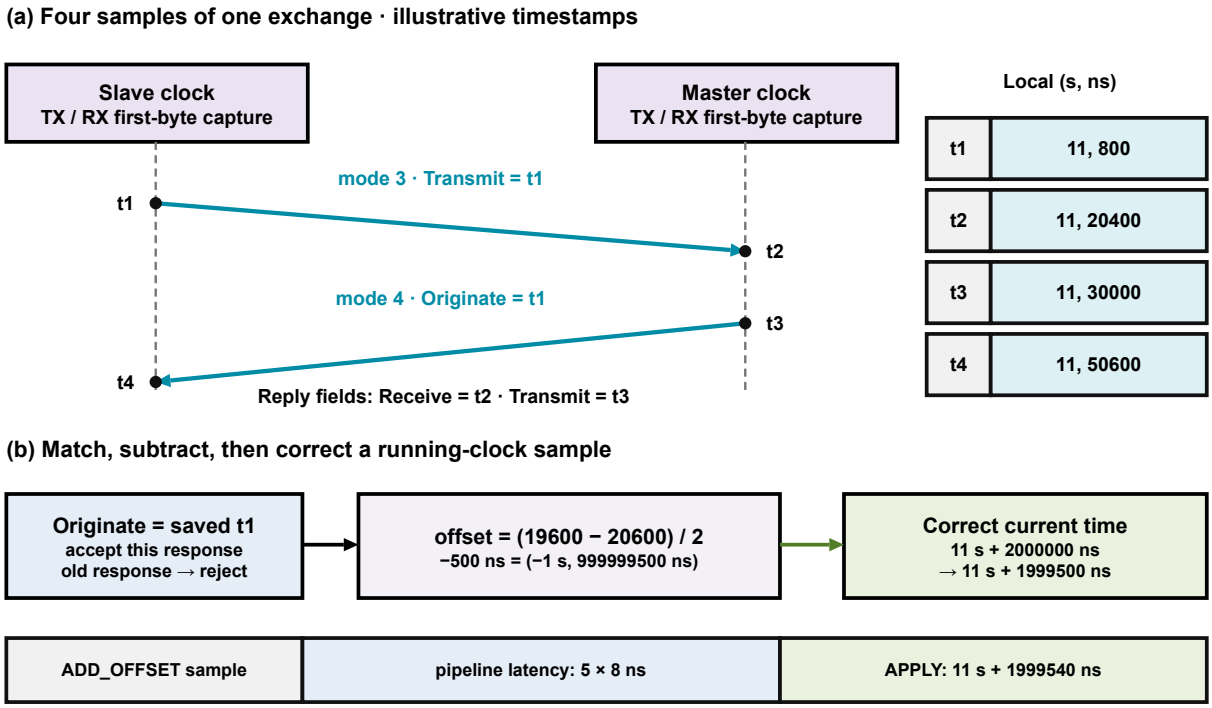


图 7-2 SNTP 四时间戳机制与 −500 ns 偏差校正；时间值用于原理说明

7.2.1 时间戳采样与收发时序

afdx_rtc 包含本地实时时钟、48 字节接收移位寄存器、发送字节状态机、时钟同步计算状态机和三个 afdx_ntp_fraction 换算器（发送时间戳编码、接收或参考时间戳编码、响应时间戳解码），算术流水与进位选择数据通路见图 7-3。

同步从端（Client）按配置周期 PERIOD 定期发起同步请求，记录发送时刻 t_1；时钟源端（Server）在收到请求时采样 t_2，发送应答时采样 t_3，并将请求的 Transmit 时间戳逐位

回显至应答的 Originate 字段；从端收到应答时记录本地时间戳 t_4。从端仅接纳 Originate 与本次请求严格相等的应答，随后计算链路双向时钟偏差：

$$\text{offset} = \frac{(t_2 - t_1) + (t_3 - t_4)}{2} \tag{7-1}$$

并将计算出的偏差累加至当前本地时钟。在一个同步周期内未收到有效应答时，下一周期以新的 Transmit 时间戳发起重试，直至同步完成。

发送时间戳在报文首字节与发送核心握手当拍由硬件精准采样，接收时间戳在核心交付首字节时瞬时锁存。时间戳在握手后通过专用算术流水换算，并在对应字段输出至物理链路前完成准备。

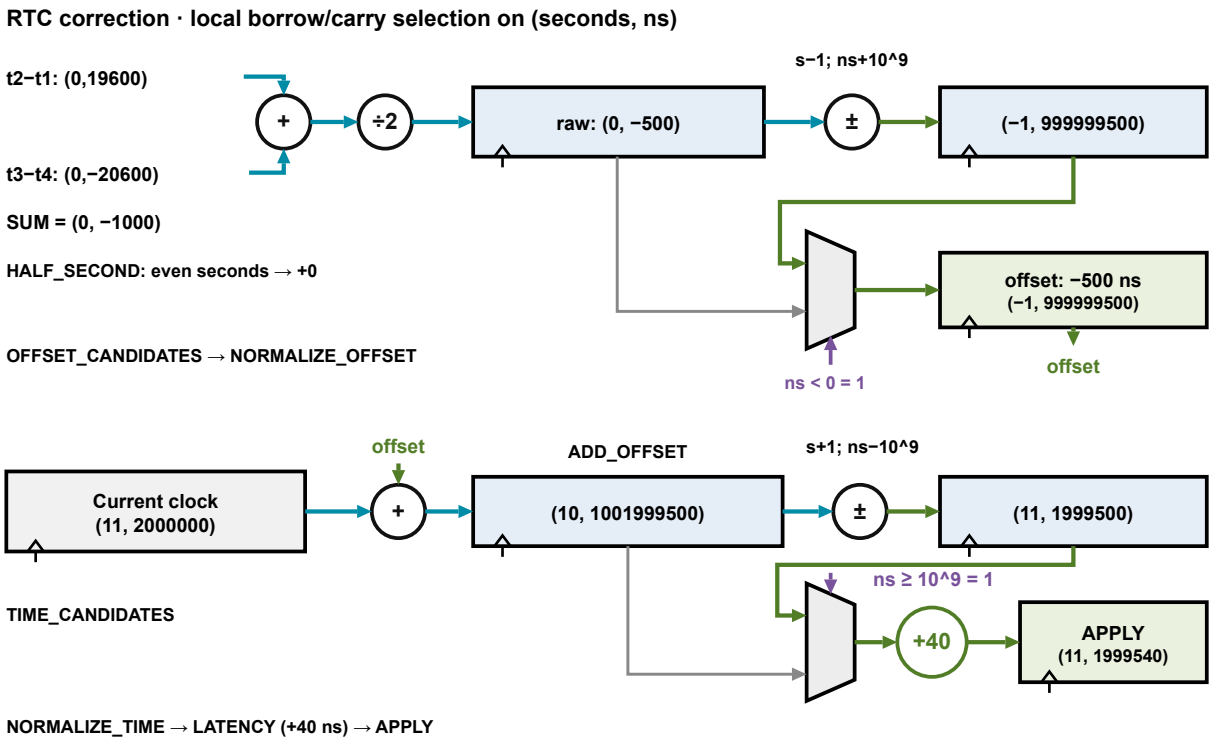


图 7-3 RTC 时钟同步算术数据通路：−500 ns 偏差的秒/纳秒借位候选、MUX 选择、时钟进位与 40 ns 流水补偿

服务端有待响应的请求或客户端同步定时器到期时，发送状态机立即启动。在无反压条件下，每个字节发送耗时 4 拍：TX_GROUPS 先在两组字段中各选一个 32 位字，TX_WORD 选定目标字，TX_BYTE 选出字节并置位 `tx_valid`，TX_HANDSHAKE 等待 `tx_ready`。发送单个 48 字节报文至少需 192 拍。Transmit 字段位于第 40–47 字节，距首字节握手至少相隔 160 拍，而硬件 NTP 小数换算仅耗时 65 拍，确保时间戳完全在飞（On-the-Fly）编码装配。服务端的 Receive 字段 t_2 在请求首字节到达时即启动换算，应答报文在整条请求校验通过后无缝启动回送。

7.2.2 时间换算与偏差校正

`afdx_ntp_fraction` 负责 32 位 NTP 二进制小数与本地纳秒整数之间的定点精确换算。纳秒编码为 NTP 小数时，采用 32 轮、每轮 2 拍的逐位非恢复余数除法计算 $\lfloor \text{ns} \times 2^{32}/10^9 \rfloor$；反向换算采用逐位相加与定点右移，并在末轮执行四舍五入。运算每拍至多执行一级 32 位加减，整数纳秒经往返换算严格实现零误差闭环。

秒差和纳秒差分别处理，每拍至多执行一级 32 位运算，步骤见图 7-3：先依次解码响应中的 t_2、t_3 小数部分（各 65 拍），再用 11 拍完成两段时间差、求和、除以 2 与写回，其中将奇数秒对应的半秒折算为纳秒，并通过 OFFSET_CANDIDATES、TIME_CANDIDATES 两个候选计算态拆开进位计算与结果选择。在 ADD_OFFSET 步中，将偏差校正量加到当前时钟值上；从该步到 APPLY 步写回共经过 5 拍，因此在 LATENCY 步由硬件精确补偿 40 ns。时钟同步计算期间，本地硬件 RTC 正常连续累加推进。

Client 正在等待响应且发送状态机空闲时，以响应的版本为 VN$=4$、模式为 mode$=4$ 且 Originate 与本次请求的 Transmit 逐位相等作为有效响应的充要判据，匹配后即启动时钟同步计算；迟到的旧响应因 Originate 不符而被自然过滤。计算期间或 `rx_done` 有效的周期不发起新请求。

7.2.3 报文字段与寄存器

请求与响应均为 48 字节，所有多字节字段高字节先发，见表 7-1。双方 UDP 源、目的端口均为 `UDP_PORT`（默认 123）。

表 7-1 SNTPv4 报文字段

字节	请求（Client）	响应（Server）
0	`0x23`：LI=0、VN=4、mode=3	`0x24`：LI=0、VN=4、mode=4
1	stratum=0	stratum=1
2–3	Poll=4、Precision=-26	同左（约 14.9 ns）
4–11	Root Delay、Root Dispersion 均为 0	同左
12–15	0	Reference ID=`LOCL`
16–23	0	最近一次配置装入的时刻
24–31	Originate=0	请求 Transmit 的逐位复制
32–39	Receive=0	t_2
40–47	Transmit=t_1	t_3

RTC 寄存器位于 `0x80`–`0x88`，见附录表 B-1。设置时间时，先写 `SEC`、`NS` 两个暂存字，再通过 `CTRL` 的装入位使两者同时生效；读 `SEC` 时同时锁存纳秒值，随后读 `NS` 得到同一时刻的值。偏差以 `OFFSET_SEC`$\times 10^9+$`OFFSET_NS` 纳秒表示。

7.3 SNMP 网络管理

SNMP 代理接收 UDP 目的端口为 161 的管理报文，响应以请求的源端口为目的端口，经配置的发送 VL 送出。系统采用模块化三层结构：顶层包装 `afdx_snmp`、协议核心引擎 `afdx_snmp_engine` 以及管理面共享发送输出缓冲 `afdx_mgmt_tx`。图 7-4 是其处理流程。

为了在 125 MHz 严苛时钟下实现深层 BER 编解码状态机的物理时序收敛，`afdx_snmp` 在顶层实现 `CE_DIV=2` 局部使能翻转树，并通过多路复制驱动将 `afdx_snmp_engine` 内部 47 个状态及其数据通路约束为 16 ns（2 周期）多周期路径；与此同时，高速 UDP 报文接收捕获、请求/响应 RAM 接口、配置总线访问以及 `afdx_mgmt_tx` 发送缓冲保持 125 MHz 单周期全速运行，既保障了 16 ns 宽松的协议解码建立时间，又避免了跨时钟域亚稳态风险与报文漏收。

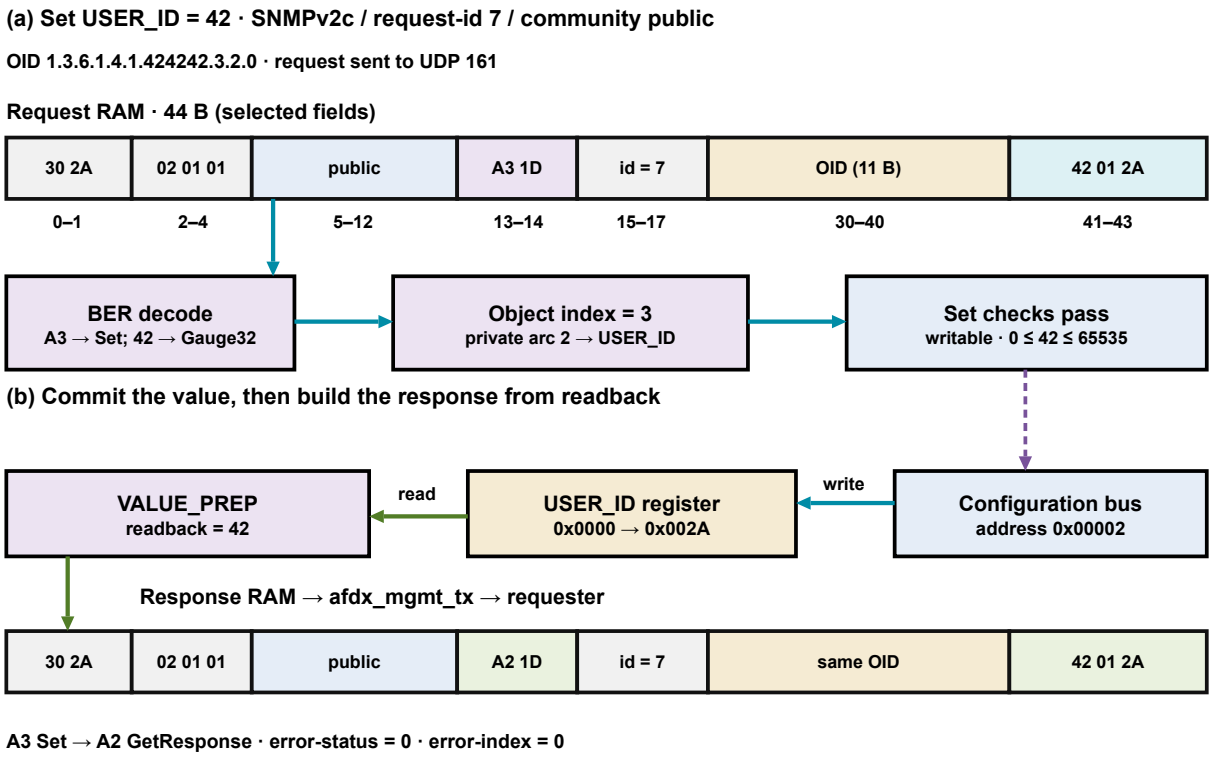


图 7-4 SNMP Set 示例：将 USER_ID 写为 42，从请求 RAM 解码，经对象查表与配置读回构造响应

7.3.1 协议对象与操作

MIB 包含 35 个只读对象与 1 个读写对象（USER_ID），共 36 个。对象分为 sysDescr、sysUpTime 两个系统对象，以及演示用私有子树 1.3.6.1.4.1.424242.3 下的 34 个对象，覆盖版本号、USER_ID、RTC 秒和同步次数、全部 27 个收发统计计数器、SNMP 请求数、Community（community）错误数和 Trap 发送数。对象值直接取自硬件寄存器。

代理支持以下协议版本和操作：

- 版本：SNMPv1（version=0）和 v2c（version=1）。
- **Get / GetNext**：Get 精确匹配标量 OID；GetNext 按子标识符逐个比较，返回严格大于请求 OID 的第一个对象，可从 1.3 遍历全部对象。对象或实例未命中时，v1 响应 noSuchName，v2c Get 统一响应 noSuchObject；v2c GetNext 遍历至末尾时响应 endOfMibView。
- **Set**：USER_ID 为读写配置对象，类型为 Gauge32，取值 0–65535。代理先检查 OID、类型和取值范围，再经配置总线写入并读回，以读回值组装响应。只读对象、类型不符和越界分别按 v1/v2c 的错误码响应，配置不变。
- **Trap**：向 UDP 端口 162 发送 SNMPv2-Trap（PDU 0xA7），携带 sysUpTime.0 与 snmpTrapOID.0 两个 VarBind。当 TRAP_ENABLE 已置位时，ENABLE 由 0 变 1 触发 coldStart，Community 校验不符触发 authenticationFailure。

每个请求只处理一个 VarBind。若请求的 Community 不符，代理静默丢弃而不发送响应，并递增 BAD_COMMUNITY 计数器。

7.3.2 请求解析与响应组装

代理的主状态机有 47 个状态，按功能分为 7 组。图 7-5 以 Set `USER_ID` 为 42 的同一请求，展示 OID 解码与设置状态的具体数据变化。请求先完整写入 256×8 的请求 RAM，解析与查找通过公共读取状态序列访问 RAM：原状态分支将返回位置写入 `return_state`，读取序列用 4 拍依次检查越界、发出读请求、等待同步读出并锁存数据，随后转回原状态分支。`Fetch` 状态组由解析、`Lookup`、`Value` 和 `Encode` 共用；每个解析状态只处理一个已就绪的字节，单拍逻辑只有一级比较。Set 未命中时经 Value 组的 `SET_ERROR` 编码错误响应，异常处理完成后返回接收状态。

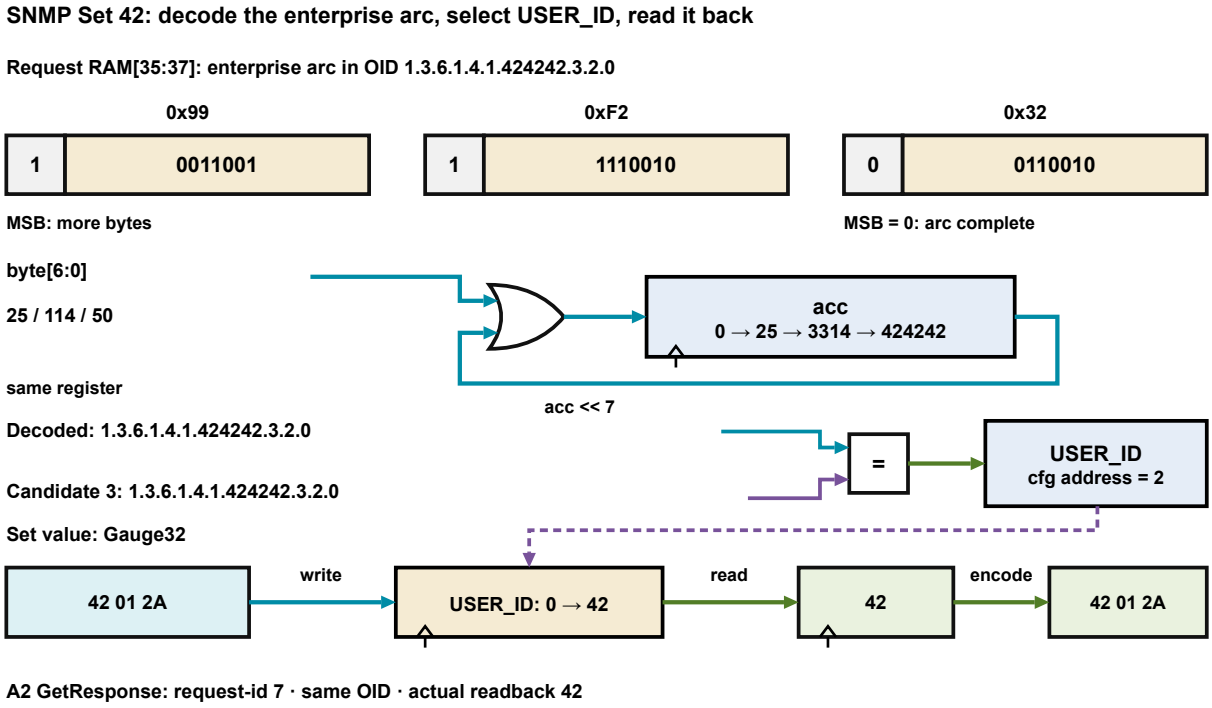


图 7-5 SNMP OID 的续字节位、7 位移位累加、对象比较及 Set 写后读回数据通路

代理按 BER 格式依次解析 SEQUENCE、version、`community` 和 PDU；PDU 内依次是 `request-id`、`error-status`、`error-index` 和 VarBindList。长度字段支持短形式（Short-form Length）与长形式（Long-form Length，`0x81`、`0x82`）。响应在 256×8 的响应 RAM 中逐字节组装，原样保留 `request-id` 和 `community`，再经服务发送口输出。

查找对象时，代理按对象表顺序依次检查 36 个候选，对每个候选按子标识符逐段比较：多字节子标识符按 BER 的 7 位分组累加还原成整数后比较数值，使 GetNext 按 OID 的数值顺序选择对象。

在 LEN1–LEN4 四拍中，代理自内向外计算值、VarBind、VarBindList、PDU 和整条消息的长度，随后在 BUILD 状态按“类型、长度、内容”逐字段写入响应 RAM。其中，`community`、OID 和回显值在 COPY 状态下从请求 RAM 逐字节复制。响应组装完成后，发送状态机逐字节读出数据，每字节用 4 拍送入服务发送口。

代理一次处理一个请求，只在 IDLE 状态接收新请求。处理期间到达的请求仍按端口匹配结果分流到代理，并计入请求数，但不产生响应，由管理端超时重发。超过 256 字节的请求整条丢弃。

7.3.3 配置访问与寄存器

代理通过与主机共享的配置读通路访问寄存器。主机访问不同地址时，优先处理主机请求；主机与代理读取同一地址时，将两者在同一拍合并。只有主机无读写请求且 `cfg_ready` 为 1 时，代理才获得写访问权。

SNMP 寄存器位于 `0x90–0xA3`，包括启用、响应 VL、community 长度和内容、请求计数、Trap 开关与计数及 sysDescr，见附录表 B-1。

7.4 ARINC 615A 软件数据加载

ARINC 615A 目标端采用严谨的三层微架构：顶层封装 `afdx_615a`、协议状态机核心 `afdx_615a_engine` 与共享发送输出缓冲 `afdx_mgmt_tx`。外部机载数据加载机（Data Loader）经 TFTP 发起单文件上传请求（LUR），目标端自动拉取头文件（LUH）和目标数据文件，完成硬件 CRC 校验后将数据存入片上 8 KiB 高速加载区，并通过状态文件（LUS）实时报告传输进度与结果；目标端同时支持介质定义下载（Media-Defined Download, MDD），将加载区内容作为文件 `MEMORY.BIN` 回传加载机。协议文件版本默认采用 `A4`，向下兼容 `A3`；目标端硬件标识为 `DEMC3_1`。

`afdx_615a` 顶层例化了专用时钟使能树（`CE_DIV=2`），将包含约 70 个状态的核心状态机 `afdx_615a_engine` 隔离在 16 ns 多周期路径中，使复杂的协议会话与校验逻辑完全免于 8 ns 单周期关键路径约束；对外报文捕获与发送输出缓冲保持 125 MHz 全速运行。

目标端重点实现单文件上传与介质定义下载（Media-Defined Download, MDD）两项核心操作；其余操作（如信息查询操作 LCI 与操作员定义卸载 LNO）在初始化阶段直接返回拒绝状态码。底层传输遵循 RFC 1350 标准 TFTP 协议，采用固定 512 字节块长：请求报文按操作码及以零字节定界的文件名、模式字段解析；会话期间的数据与 ERROR 报文均按目的 TID 关联对应传输通道。

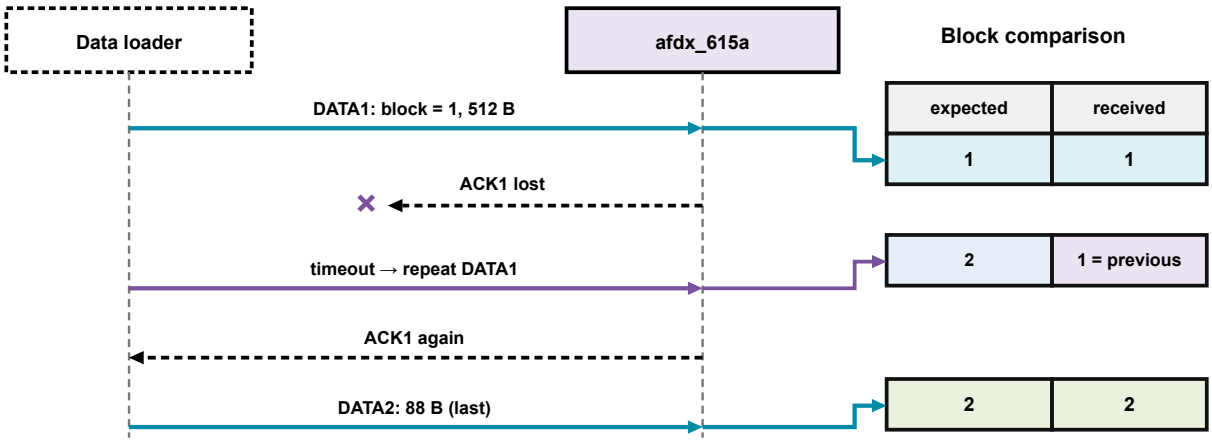
7.4.1 报文分派与异常处理

`afdx_615a` 的主状态机包含报文分派、文件处理与状态报告等状态组，图 7-6 展示重复数据块的关键分派路径。收到的报文先整条存入 1 KiB 的接收缓冲，由分派状态 DISPATCH 按操作码、端口（TID）和块号分派给六类处理过程。文件处理包含 List、Header、Data、Download 和 Status 过程；结束处理完成后返回等待状态。

各处理过程共用字节读取、整数拼接与报文输出状态序列。读取字节时，用 `return_state` 保存返回位置；读取整数时，另用 `num_return` 保存字段处理的返回位置。输出报文时，先在发送 RAM 中逐字节组包，发送后根据返回状态寄存器 `after_tx` 转移，通常返回等待报文的 `WAIT_PACKET` 状态。这些公共序列使各处理分支共用相同的同步读与发送时序。

数据或文件检查失败时，状态机锁存错误码 `reason` 并转入 `FAILURE` 状态，依次发送 TFTP ERROR 和失败状态文件；状态文件传输失败（含重试耗尽）时，结束本次操作。请求名称或模式错误时只返回 ERROR；LCI/LNO 在初始化阶段拒绝。

(a) Lost ACK1: resend the block, never append it twice



(b) DATA_DISPATCH selects copying or acknowledgment only

Arrival	FSM action	RAM write	loaded	CRC (hex)
DATA1	DATA_COPY	[0:511] ← 512 B	0 → 512	56EE
repeat DATA1	ACK → WAIT_PACKET	none	512 → 512	56EE
DATA2	COPY → file check	[512:599] ← 88 B	512 → 600	B166

DATA1 progress status completes before ACK1; final length / CRC checks precede ACK2.

图 7-6　615A 的 ACK1 丢失示例：重复 DATA1 仅重发确认，加载长度与 CRC 均不再次更新

接收缓冲一次保存一个报文。当前报文未取走时，新到报文仍按端口匹配结果分流到服务，但不写入缓冲；对端未收到响应时，按超时机制重发。

7.4.2　TFTP 传输与重传

TFTP 使用 RRQ、WRQ、DATA、ACK、ERROR 五种报文，采用 octet 模式，块长为 512 字节，块号从 1 开始；WRQ 以 ACK0 建立传输。请求目的端口为 `REQUEST_PORT`（默认 59），建立传输后双方使用各自的传输标识符（TID），目标 TID 从 `0x8100` 递增。主文件与状态文件分别保存 TID 和块号。

发送报文保存在 RAM 中，直至收到响应；超时（默认 2 s）后，原样重发上一报文。重传上限默认为 1 次；重传次数耗尽后，判定传输失败，错误原因码为 5。收到重复 DATA 时，只重发 ACK，不重复写入 RAM。

7.4.3　上传与文件校验

上传流程见图 7-7(a)：

1. 加载端读取 `DEMC3_1.LUI`，目标端返回接受状态 `0001`。
2. 目标端写入 `DEMC3_1.LUS` 报告初始状态。
3. 加载端写入上传请求文件 `DEMC3_1.LUR`（含头文件名和件号）；协议版本不支持时拒绝请求，错误原因码为 1。
4. 目标端发送 LUS `0002` 后读取请求文件中的 LUH 头文件；头文件校验通过后读取其指定的数据文件。

5. 数据块按序写入加载区并流式累计 CRC-16。首个满块写入后暂缓 ACK，先经独立会话发送一次进度 LUS，再确认该数据块。
6. 末块（短块或空块）到达后，仅当长度和 CRC-16 均正确时才予以确认，并发送 LUS `0003`（进度 100%）；否则发送 TFTP ERROR 和失败状态 LUS `1003`。

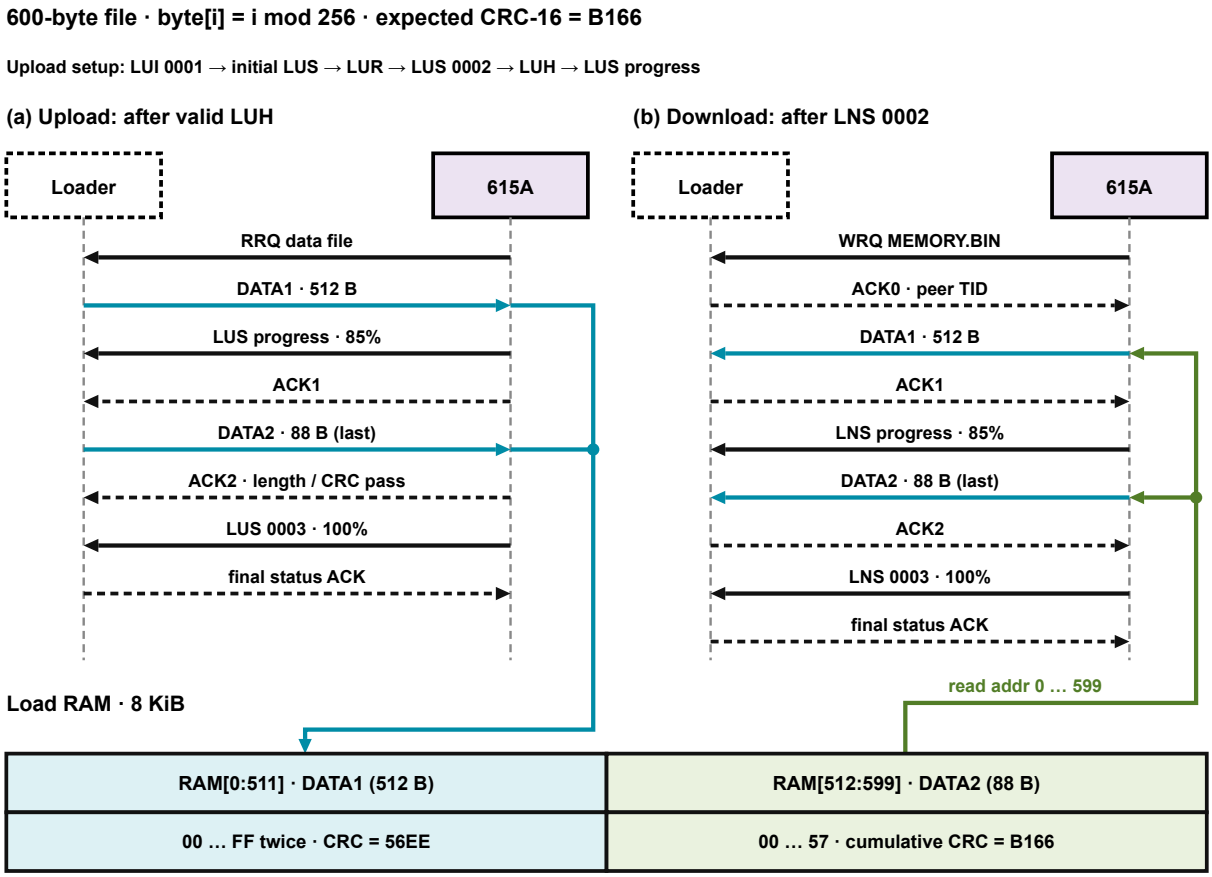


图 7-7 615A 的 600 字节文件示例：上传写入加载 RAM，下载从同一 RAM 读出；图示两块 DATA、ACK 与进度及最终状态文件的先后，省略状态文件内部的常规握手

两种传输的末块均少于 512 字节；文件长度为 512 字节的整倍数时，补发零长度 DATA 块。上传在 LUH 校验完成后先报告进度，再请求数据文件。

LUH 解析支持 ARINC 665-1 的 `8002` 格式版本：目标端依次提取头文件字数、数据文件列表指针、数据文件数、文件名、数据文件字数和两级 CRC-16 字段，对头文件实际长度、格式版本、单文件列表数量、头文件 CRC-16 以及目标数据文件的字节长度和流式 CRC-16 执行完整校验。

CRC-16 的多项式为 `0x1021`，最高位优先、初值 `0xFFFF`、结果不取反，以 8 位并行的组合函数每拍处理一字节。

每生成一份新 LUS，计数加 1，重传时不增加计数。进度按已写入字节数占预期总字节数的整数百分比计算，成功前上限为 99。计算时先由移位相加完成乘 100，再以预期总字节数为减数逐拍迭代求商，最后通过减 10 迭代分离十位与个位，转换为十进制 ASCII 字符。

7.4.4 介质定义下载（Media-Defined Download, MDD）

介质定义下载流程见图 7-7(b)。加载机读取 `DEMC3_1.LND`，目标端响应后发送初始状态文件 LNS。随后，加载机发送下载请求文件 LNR，目标端校验并仅接纳其中的 `MEMORY.BIN` 请求。目标端发送 LNS `0002` 后，按 512 字节块采用停等机制主动回传最近一次成功上传的数据内容；首个满块获确认后上报传输进度，末块获确认后发送最终状态文件 LNS `0003`。下载操作仅以只读方式访问加载区，不影响既有加载数据状态。

文件上传与下载操作严格互斥，会话在最终状态文件（LUS/LNS）获得 ACK 确认后方判定为成功；新一轮上传启动时自动撤销前次的有效长度。片内规划 8 KiB 高速 RAM 作为加载数据暂存区，既支持主机经由配置总线按字节读回，亦可用于介质定义下载（MDD）文件回传。

7.4.5 存储与寄存器

协议 RAM 共有 6 块，均以 `afdx_ram_sdp` 例化：接收报文（1 KiB）、发送与重传报文（1 KiB）、LUR（512 B）、LUH（2 KiB）、加载数据（8 KiB），以及第 6 块专门用于存储拒绝原因文本的 `denial_ram`（32 项 × 8 bit = 256 bit）。在生成拒绝响应时，状态机直接从 `denial_ram` 逐字节流式读出预置的拒绝说明字符串，替代了原有的宽位组合多路选择逻辑，大幅削减了物理级数。

寄存器 `0xB0`–`0xBB` 见表 B-1。`STATUS` 报告空闲、进行中、成功、失败及原因码：0 无错误；1 元数据不支持或无效（版本、文件名、数量）；2 容量或长度超限；3 CRC 错；4 对端 TFTP ERROR 或模式错误；5 重传耗尽。主机通过 `MEMORY_ADDRESS` 选择加载区字节，再从 `MEMORY_DATA` 读回。

第 8 章　配置、复位与统计

afdx_cfg 负责配置总线的地址译码、配置寄存器和 27 个统计计数器的维护、读数据多路选择，并在配置表写入时与收发核心执行许可握手。模块输出全部来自寄存器，结构见图 8-1。

8.1　地址映射

配置地址为 18 位字地址，映射见表 8-1。各条 VL 通过自己的 en 位控制启用。对表外地址的写请求予以忽略，读请求返回 0。

表 8-1　配置地址映射

地址	读写	内容
0x00000	只读	VERSION，0xC3110000
0x00001	只读	STATUS，位 0 为 tx_ready
0x00002	读写	USER_ID[15:0]，填入源 MAC 第 9–10 字节
0x00040–0x00046	只读	发送计数器：accept、drop_dis、drop_full、drop_long、drop_nopage、tx_a、tx_b
0x00048–0x0004F	只读	A 网接收计数器，对应 ev_a 位 0–7
0x00050–0x00057	只读	B 网接收计数器，对应 ev_b 位 0–7
0x00058、0x00059	只读	rm_drop、deliver
0x0005A、0x0005B	只读	a_gdrop、b_gdrop（保留，恒为 0）
0x00080–0x000BB	—	RTC、SNMP、615A 寄存器（附录表 B-1）
0x01000 + VL×8 + 字号	只写	tx_cfg
0x10000 + VLID	只写	rx_map，数据取位 12:0
0x20000 + idx	只写	rx_cfg，idx < 4096

三张表按地址高位译码：地址位 17:10 为 0000_0100 时选中 tx_cfg，位 17:16 为 01 时选中 rx_map，位 17:12 为 10_0000 时选中 rx_cfg。

8.2　配置表格式

三张配置表的字段见表 8-2。复位后三张表全部清零，即全部 VL 未启用、全部 VLID 未映射。

表 8-2　配置表字段

表与字	位	字段	含义
tx_cfg 字 0	0	en	VL 启用
	1、2	net_a、net_b	从 A、B 网发送
	8:3	qmax	队列最多排队的消息数，0–63

接下页

表 8-2　（续）

表与字	位	字段	含义
	19:9	max_payload	单帧最大有效载荷 $L_{\max}$，1–1471 字节（应用消息 $M > L_{\max}$ 且 $L_{\max} \geq 8$ 时触发 IPv4 分片，消息总长上限 8192 B）
tx_cfg 字 1	31:0	bag	BAG，单位为拍，1 ms = 125000，不超过 2^{30}
tx_cfg 字 2	15:0	VLID	目的 MAC 低 16 位（顶层 AFDX_top 自动窥探同步至 afdx_app_tx）
tx_cfg 字 3、4	31:0	源 IP、目的 IP	IPv4 地址
tx_cfg 字 5	31:16、15:0	源端口、目的端口	消息不带端口时使用的默认 UDP 端口
tx_cfg 字 6、7	—	保留	写 0
rx_map[VLID]	12	valid	该 VLID 已映射（顶层 AFDX_top 自动窥探同步至 afdx_app_rx）
	11:0	idx	接收 VL 索引，0–4095
rx_cfg[idx]	0	en	VL 启用
	1、2	net_a、net_b	接收 A、B 网的帧
	3、4	ic_en、rm_en	启用完整性检查、冗余管理
	16:5	skew_max	两网最大到达时间差，单位 32.768 µs
	27:17	max_payload	单帧最大载荷 $L_{\max}$，1–1471 字节（后续 IPv4 分片允许 $\leq L_{\max} + 8$）

8.3　配置读写与仲裁

读数据的多路选择分两拍完成，读延迟固定为 2 拍：第 1 拍按地址低 3 位从四组数据源（全局寄存器、发送计数器、A 网计数器、B 网计数器）及三项服务寄存器中并行预选出候选字并锁存，同时按地址位 17:3 锁存组号；第 2 拍按组号从各路寄存值中选出目标字送至 cfg_rdata，并拉高 cfg_rvalid，每拍至多经过一级 8 选 1 逻辑。

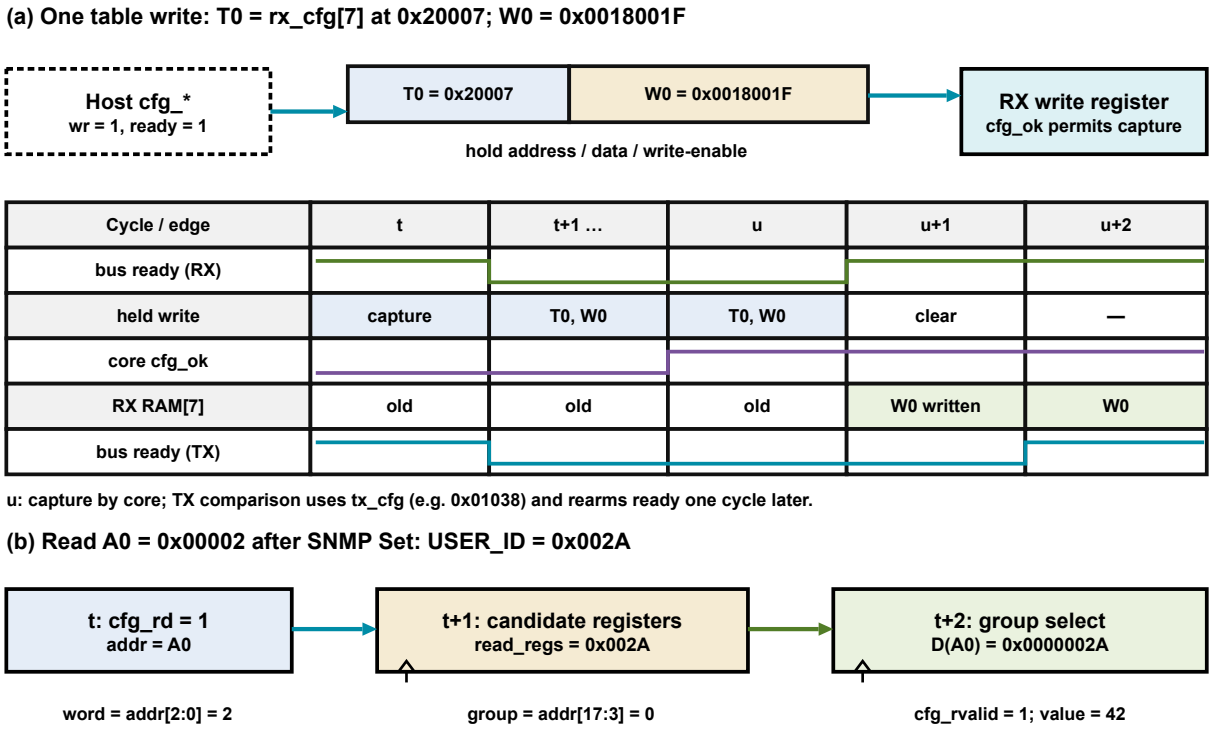


图 8-1　配置事务示例：(a) 暂存对 `rx_cfg[7]` 的写入，等待许可后写 RAM，并对比 RX/TX 的就绪重武装；(b) 读回 `USER_ID`=42 的两拍候选锁存与选择

`cfg_ready` 为 1 时，总线在当拍接受 `cfg_wr` 写请求：普通寄存器和服务寄存器直接写入；配置表写请求则先装入暂存寄存器并拉低 `cfg_ready`，等待对应核心给出 `cfg_ok`（第 4.7 节）。`tx_ready` 为 0 时，配置表写暂存寄存器保持清零，`cfg_ready` 为 0，不接受表初始化期间的写请求。

SNMP 代理读写管理对象使用同一条总线。顶层的仲裁规则是：主机在本拍写入，或主机与代理读取不同地址时，优先处理主机请求；主机读与代理读同一地址时合并为一次读，两者各自在 2 拍后取得数据；代理写只在主机无读写且 `cfg_ready` 为 1 时授予总线访问权。主机与代理的读数据分别由各自的 2 拍有效标记对齐。

8.4　配置写与查表的冲突仲裁

配置表的读口归收发引擎，写口归配置总线。配置写通过 `cfg_ok` 与引擎读请求互斥，避免同步 RAM 同拍同址读写冲突。各引擎的许可条件为：

$$\text{发送侧：}\quad \text{cfg_ok} = \text{enable} \wedge (\text{op} = \text{IDLE}) \wedge \neg\text{want_enq} \wedge \neg\text{want_grant} \tag{8-1}$$

$$\text{接收侧：}\quad \text{cfg_ok} = \text{enable} \wedge (\text{op} \in \{\text{IDLE}, \text{AGE}\}) \wedge \neg\text{desc_a_valid} \wedge \neg\text{desc_b_valid} \tag{8-2}$$

`cfg_ok` 由引擎自身的状态寄存器与同级模块的寄存输出（入队请求、帧描述有效、`grant_ready`）组合生成。`afdx_cfg` 接收一次写表请求后，保持写使能（`tx_cfg_we`、`rx_map_we` 或 `rx_cfg_we`）及地址、数据；当对应核心的 `cfg_ok` 为 1 时，引擎本拍不置位对应配置表的读使能寄存器，核心模块在该拍把写请求锁存到写口寄存器并于下一拍写入

RAM，afdx_cfg 撤销写请求。接收侧在许可后的下一拍恢复 cfg_ready；发送侧通过 tx_cfg_ok_q 与 tx_cfg_rearm 再延后一拍恢复就绪，保证本地应答与真实写请求完成对齐。时序为：

$$\begin{aligned} &\text{cfg_ready}(t+1)=0, \qquad u=\min\{\,\tau\ge t+1:\text{cfg_ok}(\tau)=1\,\},\\ &\text{cfg_ready}_{\text{RX}}(u+1)=1, \qquad \text{cfg_ready}_{\text{TX}}(u+2)=1. \end{aligned} \tag{8-3}$$

其中 t 为总线接受写请求的时刻，$u-t$ 为写表等待时间，上界为 64 拍（第 3.7 节）。

8.5 复位与初始化

复位与表初始化过程见第 3.4 节。配置模块复位后，各寄存器恢复默认值，27 个计数器清零。tx_ready 为 0 期间，配置表写暂存寄存器保持清零，cfg_ready 保持为 0；此时不接受配置写请求，读访问照常进行。

8.6 统计计数器

27 个计数器均为 32 位，计满后回绕至 0，仅在复位时清零。为隔离发送核心与接收核心长距离跨模块引入的时序关键路径，afdx_cfg 对来自 TX/RX 通路的 5 路事件统计脉冲（ev_tx、ev_a、ev_b、ev_rm_drop、ev_deliver）插入了两级流水寄存（_p 与 _q）后再驱动 32 位加法器，降低了跨芯片远距离脉冲直接驱动大位宽累加器的布线延迟。

计入事件输入两级流水打拍（_p 与 _q）及读数据通路的两级流水结构，在 t 拍发出配置总线读请求，$t+2$ 拍返回对应事件脉冲截至 $t-3$ 拍在核心源头产生的累计次数（对应配置模块内部计数值在 $t-1$ 拍及以前的累加状态）。计数器分为：

- 发送（**7** 个）：接纳的消息数；因 VL 未启用、队列满、超长、无空闲页而丢弃的消息数；A、B 网发出帧数。
- 接收，每网 **8** 个：正确帧、FCS 错误、帧长错误、帧头错误、未知 VL、配置不允许、完整性检查丢弃、溢出。
- 接收，两网合计 **2** 个：冗余管理丢弃、交付。
- 保留 **2** 个：a_gdrop、b_gdrop，地址保留，恒为 0。

8.7 异常处理汇总

表 8-3 汇总全设计对各类异常的检测位置、处理方式和统计。发送侧拒收消息后，先回收已分配的页，再继续接收并丢弃剩余字节。

表 8-3 异常处理汇总

异常	检测位置	处理	统计
发送与顶层发送包装			
顶层发送 VLID 未映射	afdx_app_tx	整条消息丢弃，不进入核心	ev_app[0]
顶层发送超 8192 B 或包装缓存满	afdx_app_tx	回滚包装写指针，丢弃本条消息	ev_app[1]

接下页

表 8-3　（续）

异常	检测位置	处理	统计
VL 或两网均未启用	`tx_engine`	拒绝入队，回滚回收已用缓存页	`drop_dis`
队列已满	`tx_engine`	同上	`drop_full`
核心消息超过 8192 B，或分片时 $L_{\max} < 8$，或单帧超过 $L_{\max}$（当配置不分片）	`tx_engine`、`tx_buf`	从回滚 RAM 回收全部已分配页，继续接收并丢弃剩余字节	`drop_long`
无空闲页	`tx_buf`	同上	`drop_nopage`
MAC 发送下溢（TX Underrun）	`mac_tx`	发送未取反的 FCS，使对端判为错误帧，继续接收并丢弃本帧剩余字节	不计 `tx_a`/`tx_b`
接收、分片重组与顶层接收包装			
前导码错误、帧长越界	`mac_rx`	整帧丢弃	帧长错误
FCS 错误	`mac_rx`	整帧丢弃	FCS 错误
固定字段、长度、非 8B 对齐分片或校验和错误	`rx_parse`	回退预写入指针	帧头错误
环形缓冲剩余空间不足	`rx_parse`	不写入本帧	溢出（`ev_a/b[7]`）
VLID 未映射	`rx_engine`	回退预写入指针	未知 VL
VL 或本网未启用、单帧超 $L_{\max}$（后续片超 $L_{\max}+8$）	`rx_engine`	回退预写入指针	配置不允许
SN 不连续	`rx_engine`	回退预写入指针，PSN 仍更新为本次 SN	完整性检查丢弃
另一网已交付的副本	`rx_engine`	回退预写入指针	`rm_drop`
交付队列已满，或分片缺首片/乱序/重叠/16 槽全满	`rx_engine`、`rx_reasm`	回退预写入指针，拒绝进入重组槽位	溢出/拒绝（`ev_a/b[7]`）
末片总长不符、非零全包 UDP 校验错或 2 s 老化超时	`rx_reassembly`	废弃重组槽位并释放上下文	溢出/拒绝（`ev_a/b[7]`）
顶层接收包装缓存溢出	`afdx_app_rx`	整包丢弃，保护已入队消息边界	`ev_app[2]`
表初始化完成前开始的帧	`rx_parse`	整帧忽略	不计
服务与配置			
SNMP community 不符	`afdx_snmp`	不响应；Trap 启用时发 `authenticationFailure`	`BAD_COMMUNITY`
SNMP 请求超过 256 字节或代理忙	`afdx_snmp`	端口匹配后丢弃，由管理端超时重发	忙时计入 `REQUESTS`

接下页

表 8-3 （续）

异常	检测位置	处理	统计
615A 数据/文件检查错、状态传输失败、请求错误	afdx_615a	数据/文件错：ERROR 及失败状态文件；状态传输失败（含重试耗尽）：直接结束；名称/模式错：仅 ERROR；LCI/LNO：初始化拒绝	STATUS 原因码 1–5（第 7.4 节）
SNTP 响应版本、模式或 Originate 不符	afdx_rtc	丢弃，下一周期重发请求	—
表初始化期间写配置	afdx_cfg	cfg_ready= 0，不接受	—
未映射的地址	afdx_cfg	忽略写请求，读出 0	—

8.8 主机编程流程

8.8.1 初始化与启用

主机经配置总线按下列顺序使端系统进入工作状态。每次写表都要等 cfg_ready 为 1（第 4 章）。

1. 初始化：释放复位后轮询 STATUS，等待位 0（tx_ready）变为 1，约 524 µs。
2. 端系统标识：写 USER_ID。
3. 发送 **VL**：对每条发送 VL 先写字 1–5（BAG、VLID、IP、默认端口），最后写字 0，同时设置 en、网络选择、qmax 和 max_payload。
4. 接收 **VL**：对每条接收 VL 先写 rx_cfg[idx]，再写 rx_map[VLID]= $\{1, \text{idx}\}$，使映射生效时配置已经就绪。
5. 服务：为每项服务配置一条发送 VL（目的 IP、端口与服务端口一致），写服务的 TX_VL、端口和 community 等寄存器，最后写 ENABLE。
6. 开始收发：应用开始提交消息。VL 较多时按第 3.7.3 节错开各 VL 首条消息的提交时刻。

8.8.2 运行中更新与服务配置

tx_cfg 的 en 仅在入队时检查；已排队的消息继续参与授权，授权阶段逐拍读取字 0–5。修改一条发送 VL 时，主机应先停止向它提交消息，再把字 0 的 en 写 0，并保持该字其余字段不变；待该 VL 已提交的消息全部发出后，修改其余字，最后重新启用。

通过 cfg_ok 保证配置写与表读互斥，多字配置按字依次生效。接收侧的 rx_cfg 和 rx_map 各自通过单拍整字写入更新；两表的写入分别生效，历史状态在配置更新后保留。

应用与服务的配置还须满足以下条件：

- 应用须等待表示收发核心均已就绪的 tx_ready 有效后，再提交消息。app_tx_ready 由应用仲裁选中条件与发送核心的 m_ready 相与生成。
- 各服务的 UDP 端口须互不冲突；RTC 的模式与时间，以及 SNMP 的 ENABLE、community 和 TX_VL，仅在对应服务没有在途消息时修改。

- 615A 忙时，忽略对启用、发送 VL、请求端口、超时、重传上限和文件版本的控制写请求；写 `0xB4` 后至少等待 4 拍，且只在非忙状态下读 `0xB5`。

8.9 配置约束与网络规划规格

主机须检查各项配置的组合是否满足带宽、抖动、缓存和分片重组并发度要求。结合官方 1030 答疑 R138-3/R139-3，本设计声明的端系统满载网络规划规格为单端 63 条发送 VL / 63 条接收 VL（BAG $= 1$ ms，$L_{\max} = 1471$ B，双网冗余全双工，物理线路占用率 77.5%，留有约 22.5% 突发排空与管理服务裕量）。具体约束及超限处理如下：

- 带宽：A、B 两网由同一个 MAC 状态机同拍驱动，只发 A 网或只发 B 网的帧同样占用一个发送时隙，所以约束针对全部发送 VL，不分网络：$\sum_v C(n_v)/\mathrm{BAG}_v \le 1$，其中 n_v 为单帧最大载荷 $L_{\max}$，$C(n_v)$ 为第 3.7 节式 (3-1) 的线路占用拍数，服务 VL 计入求和。长包分片消息在每个 BAG 周期仅调度 1 个分片帧，单周期带宽占用与单帧 VL 一致。
- 抖动：BAG 取 2^k ms；周期业务宜错开各 VL 首条消息相位，在 63 VL 满载规格下即使发生旋转三元组背靠背突发（B2），同 VL 最大端到端极差抖动仍保持在 96.752 µs（< 100 µs），而超规格 81 VL 对齐突发将因排队逼近 1 ms 周期而超出百微秒门限。
- 发送与重组缓存：发送核心共 256 页（128 KiB），单帧 1471 B 占 3 页，8192 B 最大长包占 16 页；主机规划在途驻留页数须满足 $\sum_v \mathrm{qmax}_v \times \lceil M_v/512 \rceil + 9 \le 256$（例如 60 条短包加 1472/3000/8192 B 三条长包共占 205 页，安全满足；多条并发 8192 B 长包则须受 256 页总预算与顶层 32 KiB 包装缓冲约束）。接收侧配备 16 个并发重组槽位（奇偶 VL 索引各 8 个槽位），同一时刻处于跨 BAG 分片重组中的活跃长包 VL 数在奇偶各 Bank 内均不得超过 8 条，且同 VL 新长包须在前一长包末分片发出后再提交。
- 交付：交付口每拍一字节。若某条 VL 关闭冗余管理，且同时接收两网数据，则每条消息交付两次。所有接收 VL 的载荷交付总速率须低于 1 字节每拍；两网独立满速且都要交付时，所需交付速率超过该上限，超出部分按溢出计数。
- 顺序与冗余窗口：跨 VL 的发送顺序由各 VL 的 BAG 与候选选择共同决定；接收分片必须在同一 VL 内按递增片偏移顺序到达，乱序分片将被拒绝。接收副本的到达间隔超过 `skew_max` 时，冗余记录按过期处理。
- 取值范围：`bag` $\le 2^{30}$ 拍；`qmax` ≤ 63；`max_payload` 为 1–1471（启用分片时 ≥ 8）；应用消息总长 $1 \le M \le 8192$ B；`skew_max` 小于过期清理门限 `0x4000` 个计时单位（12 位字段自然满足）。

第 9 章　时序与存储设计

9.1　时延预算与流水账本

端到端时延按测量边界分为核心分段口径与顶层全消息流式口径：

- 核心分段口径（**1022** 答疑 **R97**）：针对 `afdx_es` 核心口，从发送端应用成功提交消息末字节（m_{last} 握手）起算，至接收端完成末字节交付（d_{last} 有效）结束，严格分解为发送处理段 L_{tx}、网络链路传输段 L_{gmii} 与接收处理段 L_{rx}。
- 顶层全消息流式口径（**1026** 答疑 **R119 / 1030** 答疑 **R139**）：针对 `AFDX_top` 带 2 字节大端 VLID 前缀的流式接口，从发送端首字节（`tx_valid && tx_ready`）输入起算，至接收端末字节（`rx_valid`）输出结束，包含顶层发送包装缓存整包收集、线路传输（单帧或多片跨 BAG 调度）以及接收重组与顶层包装转发的全过程耗时。

在收发通路无其他消息争用、当前 VL 已满足 BAG 最小发送间隔的基准条件下，表 9-1 列出 1471 字节不分片满载帧（对应最大以太网帧 1518 字节）在核心各子模块流水中的开销构成：

1. 发送处理流水：`afdx_tx_buf` 经两槽入口弹性级与载荷写口本地寄存写入数据，写 47 位描述符并向调度引擎发起入队请求；`afdx_tx_engine` 入队判定需 6 拍、单帧/分片授权需 9 拍；`afdx_tx_builder` 采用 12 拍（`0..11`）准备序列计算 IPv4 首部校验和并提前 8 拍（`START_LEAD_REGISTERED = -8`）锁存流缓冲，配合双网 MAC 输出。核心基线实测从 m_{last} 到 GMII 首个前导码字节 `tx_sof` 为 32–34 拍（0.256 µs 典型）；按 R95 规则计至 SFD（`8'hD5`）结束，发送处理时延 L_{tx} 典型为 39 拍（0.312 µs）。
2. 物理链路传输：前导码首字节至 FCS 末字节共 $8 + 1518 = 1526$ 字节，在线路上占 1525 拍（扣除与发送首拍重叠的 1 拍），折合 12.200 µs。直连仿真模型下 SFD 间的链路延迟 $L_{\text{gmii}} = 0.000$ µs。
3. 接收处理流水：接收 MAC 检测帧尾需 3 拍；`afdx_rx_parse` 帧尾分类需 3 拍；接收引擎在基线单帧模式下执行查表、IC/RM 判定与入队需 14 拍（引入 32-Bank 三级归并、窄分片状态表及双 Bank 分片重组准入查询后，`FRAME` 固定日程扩展为 30 拍，即多 16 拍 / 0.128 µs）；交付模块读取 `rx_dq` 并自环形缓冲经单帧直通路径读出首字节需 6–8 拍。基线单帧接收段首字节时延 L_{rx} 为 12.360 µs，末字节时延 $L_{\text{rx,last}}$ 为 24.120 µs。
4. 逐字节交付与顶层流式转发：核心交付口每拍交付 1 字节，1471 字节载荷首至末字节需 $n - 1 = 1470$ 拍（11.760 µs）。叠加顶层 `afdx_app_tx`/`afdx_app_rx` 整包缓存转发后，在声明 63 VL 满载规格（BAG $= 1$ ms）下：稳态周期发送（`P1471`）顶层最大端到端时延实测为 80.336 µs，旋转三元组突发（`B2`）最大时延为 1175.432 µs；在 63 VL 混合分片场景中，8192 B 6 分片长包跨越 5 个 BAG 周期完成传输与顺序重组，顶层端到端时延实测为 5212.304 µs。

表 9-1　核心端到端时延流水账本（1471 字节不分片满载帧基线）

阶段与区段	典型拍数	典型时间 (µs)	上界时间 (µs)	详细流水构成与硬件依据
发送写缓存与入队	4	0.032	0.080	载荷写寄存 1 拍，写 `tx_desc` 并置 `enq_valid` 3 拍
调度引擎入队与授权	15	0.120	0.240	ENQ 状态 6 拍，GRANT 状态 9 拍
组帧准备与流缓冲	12	0.096	0.168	IP 校验和准备 11–12 拍，流缓冲双缓冲锁存 1 拍
MAC 起始响应	1	0.008	0.016	检测起始并输出首前导码 `tx_sof`（0.256 µs）
前导码发送至 SFD	7	0.056	0.056	7 字节 `0x55` 占 7 拍，第 39 拍输出 SFD（$L_{\text{tx}} = 0.312$ µs）
以太网帧传输	1518	12.144	12.144	帧头 42 字节、载荷 1471 字节、SN 1 字节、FCS 4 字节
MAC 帧尾与解析分类	6	0.048	0.064	MAC 帧尾判定 3 拍，解析器 FIN 状态机两级分类 3 拍
分体映射与引擎判定	14	0.112	0.288	单帧基线 14 拍；含 19 拍分片重组准入查询时为 30 拍日程
交付队头读取与流水	6	0.048	0.080	块 RAM 队头捕获 2 拍，读控制、环形缓冲与重组直通 4–6 拍
逐字节流式交付	1470	11.760	11.760	接收应用口每拍交付 1 字节，首字节至末字节（$n-1$ 拍）
无冲突硬件流水基线	3053	24.424	24.996	核心 m_{last} 至 d_{last} 基线代数和（叠加 30 拍引擎约 24.6 µs）
叠加 AGE 老化扫描	3054	24.432	25.036	接收端恰遇 AGE 扫描让出拍，与核心性能基线 24.432 µs 吻合
计入最坏在途争用与调度	—	—	26.652	计入最坏调度扫描与在途帧发送等待

无争用核心基线下端到端总时延为 3053–3054 拍（24.424–24.432 µs），即使计入分片架构的 30 拍接收引擎日程与顶层流式包装存储转发开销，单帧及混合分片时延均远低于赛题 1–2 ms 的指标上限。

9.2　片上 RAM 分配与物理存储规划

全系统共规划 34 组逻辑存储表，在 RTL 中封装为 250 个同步 RAM 模块实例（含 90 个通用 `afdx_ram_sdp` 实例与 160 个带本地写数据隔离或窄返回域的专用同步 RAM 模块），逻辑存储总容量 4,174,544 bit（约 509.6 KiB），经 eLinx 官方综合映射为 814 个物理嵌入式块 RAM（M4K/EBR），见表 9-2。所有存储器均采用严格的一写一读同步双端口时序，并在大容量缓冲写口显式保留写数据隔离 LUT，消除异步读写与短路径保持时间风险。

表 9-2 片上存储器分配与物理映射

存储表名称	逻辑规模（深 × 宽）	逻辑容量（bit）	模块实例	架构分工与物理优化说明
顶层应用流包装器（共 7 组逻辑表，69 个同步 RAM 实例：2 SDP + 67 专用）				
app_tx/payload	32 × 1024 × 9	294,912	32	发送包装 32 KiB 载荷 + 末字节标记（8 Bank ×4 子片，写隔离）
app_tx/desc_ids	32 × 16	512	1	发送包装已提交消息的 16 位 VLID 描述符队列（SDP）
app_tx/read_buf	8 × 9	72	1	发送包装 afdx_app_read 8 信用流式预取缓冲
app_rx/payload	32 × 1024 × 9	294,912	32	接收包装 32 KiB 载荷 + 末字节标记（启用 EARLY_DATA=1 预寄存）
app_rx/map	4096 × 16	65,536	1	接收包装 VL_idx → VLID 反查表（afdx_app_map）
app_rx/desc_ids	32 × 16	512	1	接收包装待输出消息的 16 位 VLID 队列（SDP）
app_rx/read_buf	8 × 9	72	1	接收包装 afdx_app_read 8 信用流式预取缓冲
包装层小计	—	*656,528*	*69*	顶层收发流包装总存储容量约 80.1 KiB
发送核心与分片调度（共 11 组逻辑表，15 个同步 RAM 实例：13 SDP + 2 专用）				
tx_payload	131072 × 8	1,048,576	1	128 KiB 载荷池（afdx_tx_payload_store，27 位写捆绑 + 写隔离）
tx_cfg	1024 × 32	32,768	1	128 条发送 VL 配置表（每条 VL 占 8 个 32 位字，SDP）
fragment_meta	128 × 142	18,176	5	分片首片锁存的 IP/UDP 头与单帧预算（4×32b + 1×14b SDP）
tx_desc	256 × 47	12,032	1	消息元数据表（14 位总长 1–8192 B、端口使能与 UDP 端口）
tx_state	128 × 63	8,064	1	发送调度状态（next_time、sn、q_count、队头/队尾）
fragment_progress	128 × 27	3,456	1	在途分片剩余字节数（14 位）与下一片 IP 偏移（13 位）
fragment_pointer	128 × 17	2,176	1	下一待发分片的起始页号（8 位）与页内偏移（9 位）
tx_free	256 × 8	2,048	1	空闲页池 FIFO，支持双页面回收端口并发归还
tx_mnext	256 × 8	2,048	1	发送消息链表（同 VL 下一条消息首页号）
tx_pnext	256 × 8	2,048	1	消息内分页链表（同一消息下一页号）
tx_rollback	16 × 8	128	1	单消息最多 16 页回滚表（afdx_tx_rollback_ram，M4K 映射）
发送小计	—	*1,131,520*	*15*	发送侧总存储容量约 138.1 KiB

接下页

表 9-2 （续）

存储表名称	逻辑规模（深 × 宽）	逻辑容量（bit）	模块实例	架构分工与物理优化说明
接收核心与分片重组（共 *8* 组逻辑表，*158* 个同步 *RAM* 实例：*67 SDP + 91* 专用）				
reasm_payload	16 × 8192 × 8	1,048,576	64	128 KiB 重组载荷池（2 Bank ×8 槽 ×4 条 8192 × 2，128 个写隔离 LUT）
rx_map	65536 × 13	851,968	64	32 Bank × 7b/6b 双写分片 SDP，16 路配置复制，三级或树读归并
rx_state	4096 × 43	176,128	11	4096 条接收 VL 状态（10×4b + 1×3b afdx_rx_table_slice）
rx_cfg	4096 × 32	131,072	8	4096 条接收 VL 配置（8×4b afdx_rx_table_slice 窄返回域）
rx_ring_a	4096 × 8	32,768	1	A 网环形载荷缓冲（4 KiB SDP），支持预写入与指针回退
rx_ring_b	4096 × 8	32,768	1	B 网环形载荷缓冲（4 KiB SDP），支持预写入与指针回退
reasm_meta	2 × 8 × 258	4,128	8	双 Bank 重组元数据表（每 Bank 含 key/header/admission/progress）
rx_dq	32 × 73	2,336	1	接收交付队列（SDP，含分片标志、4 位重组槽位、环形起点与端口）
接收小计	—	*2,279,744*	*158*	接收与分片重组总存储容量约 278.3 KiB
网络服务（共 *8* 组逻辑表，*8* 个 *SDP* 实例）				
snmp/req_ram	256 × 8	2,048	1	SNMP 请求报文缓冲（256 字节，支持单 VarBind 请求）
snmp/resp_ram	256 × 8	2,048	1	SNMP 应答与 Trap 组装缓冲（256 字节）
615a/load_ram	8192 × 8	65,536	1	615A 数据加载暂存区（8 KiB），支持主机读回与介质卸载
615a/header_ram	2048 × 8	16,384	1	615A LUH 头文件缓存区（2 KiB）
615a/pkt_rx	1024 × 8	8,192	1	615A TFTP 接收报文缓冲（1 KiB）
615a/pkt_tx	1024 × 8	8,192	1	615A TFTP 发送与超时重传报文缓冲（1 KiB）
615a/lur_ram	512 × 8	4,096	1	615A 上传请求文件（LUR）暂存区（512 字节）
615a/denial_ram	32 × 8	256	1	615A 拒绝原因文本表（消除状态机大位宽组合选择）
服务小计	—	*106,752*	*8*	网络服务总存储容量约 13.0 KiB
全设计合计	—	**4,174,544**	**250**	逻辑总容量约 509.6 KiB，综合映射为 814 个 M4K/EBR（占比 67.83%）

全系统逻辑 RAM 存储容量为 4,174,544 bit（约 509.6 KiB）。在 EQ6HL130（CSG484_H）目标器件中，经 eLinx 官方综合与打包映射，共消耗 814 个嵌入式块 RAM（M4K/EBR），占器件全部 1,200 个存储块资源的 67.83%，剩余片上存储资源裕量达 32.17%。

9.3 静态时序收敛与关键路径工程优化

在 125 MHz（8 ns 周期）并扣除 0.30 ns（300 ps）建立时间不确定度的约束条件下，端系统需要维持每拍 1 字节的线速吞吐，并驱动 814 个块 RAM（逻辑规模占器件 75.13% 的 CLB 阵列）完成 4096 条接收 VL 查表与 16 路 8192 B 分片重组。为实现布局布线后全量静态时序闭环（Timing Closure），系统在 RTL 微架构与物理实现层面采取了六项措施：

1. 管理面协议引擎 **CE_DIV** $= 2$ 多周期解耦：ARINC 615A 与 SNMP 协议包含深层状态分支。设计将核心状态机（`afdx_snmp_engine.v` 与 `afdx_615a_engine.v`）独立封装，通过 2 分频低扇出时钟使能复制树驱动，对内部 2747 个使能触发器施加 253 条 2 周期建立时间约束（16 ns 预算），并经 `afdx_mgmt_tx` 8 字节预取队列桥接 125 MHz 单周期发送域。
2. **`rx_map` 32-Bank** 双写分片与窄返回域切片：`rx_map` 划分为 32 个子库且每库拆分为 7b/6b 两块 SDP（共 64 块），将单写端负载减半，读侧采用“2-Bank $\rightarrow$ 4-Group $\rightarrow$ 最终 4-Group”三级寄存或树归并；`rx_cfg` 与 `rx_state` 拆分为 4 bit/3 bit 窄位 `afdx_rx_table_slice`，将跨阵列长连线转化为本地短跳流水。
3. 分片重组双 **Bank** 解耦与 **M4K** 写数据 **LUT** 门控隔离：16 个 8192 B 重组上下文按物理亲和性拆分为 2 个独立 8 槽位 Bank；64 条 8192×2 bit 载荷通道采用显式 `lut_mask="8888"` 二输入与门原语（共 128 个写隔离 LUT 驱动全部 256 个 M4K 数据引脚），同时在 `afdx_tx_payload_store` 与 `afdx_app_payload_slice` 写口保留本地写隔离逻辑，消除空闲写口短路径保持时间违例并隔离大扇出写总线。
4. 发送入口弹性级与 -8 拍超前准备解耦：`afdx_tx_buf` 入口插入两槽位 49 位弹性级（`afdx_tx_ingress_elastic`），切断上游仲裁器的反压扇出；`afdx_tx_builder` 采用 12 拍 IPv4 校验和准备序列与 `START_LEAD_REGISTERED = -8` 提前补偿，配合核心输出端双缓冲流寄存器，兼顾路径切分与 BAG 到期准点发包。
5. 跨层接口边界寄存与统计事件两级流水：`AFDX_top` 包装层在配置窥探与接收交付入口插入完整边界锁存（`delivery_*`/`configuration_*`）；`afdx_svc` 寄存 4 路发送授权，`afdx_cfg` 对跨核心统计脉冲打两拍流水（`_p`/`_q`），阻断长跨度组合反馈。
6. **`tiles_1030.edc`** 物理簇锚定与 **`PRES 1.30`** 布局布线闭环：构建脚本 `tools/build_1030.py` 联合加载时序约束 `elinx/constraints.edc` 与物理布局锚定约束 `elinx/tiles_1030.edc`（锁定 11044 个物理簇：10230 个 CLB + 814 个 M4K，0 缺失 / 0 漂移），并以物理优化参数 `PRES 1.30` 完成确定性布局布线。

表 9-3 汇总了官方综合估算与布局布线后静态时序（`full_timing.tan.pb` / `slack.rpt`）的签核结果。在 11044 个物理簇锚定与 `PRES 1.30` 布局布线下，综合阶段 164,745 个端点与布局布线后 96142 个时序端点的建立/保持违例数均为 0：269,131 条 8 ns 单周期路径在扣除 0.30 ns 时钟不确定度后的净建立裕量为 $+0.029$ ns（原始物理建立裕量 $+0.329$ ns），50,050 条 16 ns 多周期路径最差建立裕量为 $+0.720$ ns，319,181 条保持时间路径最差裕量为 $+0.604$ ns，等效净最高工作频率 125.45 MHz（物理裸 Fmax 130.36 MHz），完整闭环满足 125 MHz 线速运行指标。

表 9-3 综合与布局布线后全量静态时序签核指标（含 0.30 ns 时钟不确定度）

时序分析指标	官方综合估算值	布局布线后 STA 结果	签核标准与工程结论
分析主时钟与不确定度	8.000 ns / 0.30 ns	8.000 ns / 0.30 ns	125 MHz 主时钟，扣除 300 ps 建立不确定度
最差建立时间裕量（Worst Setup）	+4.148 ns	+0.029 ns	全量 96142 个时序端点 0 违例（裸裕量 +0.329 ns）
最差保持时间裕量（Worst Hold）	+0.192 ns	+0.604 ns	综合后与布线后保持时间均为正裕量
最高工作频率（Fmax）	259.61 MHz	125.45 MHz	含不确定度净 Fmax（物理裸 Fmax 为 130.36 MHz）
8 ns 单周期路径数 / 最差裕量	—	269,131 条 / +0.029 ns	最差路径：`engine|candidate -> transmit_ack_q`（3 级逻辑）
16 ns 多周期路径数 / 最差裕量	—	50,050 条 / +0.720 ns	253 条规则覆盖 2747 个管理面引擎触发器
同沿保持时间路径数 / 最差裕量	—	319,181 条 / +0.604 ns	全部正裕量，总负裕量 TNS = 0.000 ns
多周期与存储结构独立审计	—	0 非法 / 0 漏约束	经 `check_multicycle.py` 与 `audit_1030_storage.py` 核验

9.4 时序设计规则与流水划分

为使 4096 条 VL 的查表通路、16 路分片重组通路和逐字节数据通路满足严谨的工程规范，全部 34 个 RTL 文件严格遵守表 9-4 的十条时序组织规则。

表 9-4 时序设计规则

编号	规则内容与工程实现
1	全设计使用单一 125 MHz 全局物理时钟树，消除异步 CDC 亚稳态风险。
2	虚拟链路与分片重组状态集中于同步块 RAM，通过地址顺序访问；各 VL 复用数据通路与算术逻辑。
3	块 RAM 统一通过 `afdx_ram_sdp` 及专用同步存储模块例化，读写控制来自本地寄存器，大容量缓冲写口配备显式写数据隔离 LUT（`lut_mask="8888"`）。
4	RAM 读出数据必须经本地返回寄存器打拍后再做算术或比较；大容量分块表采用多级寄存或树归并。
5	复杂算术拆解到不同拍执行：RTC 时间戳换算每轮两拍迭代，615A 乘 100 采用移位相加分步求商。
6	大于 8 选 1 的多路选择与宽位比较（如 128 路发送 VLID 匹配、8 槽位重组键比对）分多级流水完成。
7	字节流水线连续推进；RAM 读取与帧头组装执行超前预读（组帧器采用 −8 拍提前量补偿 12 拍校验和与输出流水）。
8	同一 RAM 的同拍同址读写严格互斥，配置写操作必须获得引擎的 `cfg_ok` 许可或具备显式同拍旁路。
9	跨模块与跨层接口以本地寄存器定界为主；跨核心长距离脉冲（如统计事件脉冲）打两拍流水。
10	管理面复杂协议状态机通过 `CE_DIV=2` 局部使能树解耦为 16 ns 多周期路径，并经独立脚本做全量合法性审计。

第 10 章　可验证性设计与实测摘要

10.1　验证体系与自动化回归策略

设计构建了涵盖 RTL 行为级仿真、自动化全量回归测试、63 VL 顶层满载与混合分片验证、代码覆盖率签核以及布局布线后门级网表对照验证的多层次验证体系。验证平台采用多重断言判据：不仅核对测试平台控制台零错误打印，还通过独立的 Python/Tcl 参考模型（含 `tools/fragment_ref.py` 分片/重组参考模型）对顶层端口字节流记录与配置寄存器读回值进行逐字节比对。

图 10-1 展示激励、双网链路和两条校验路径。

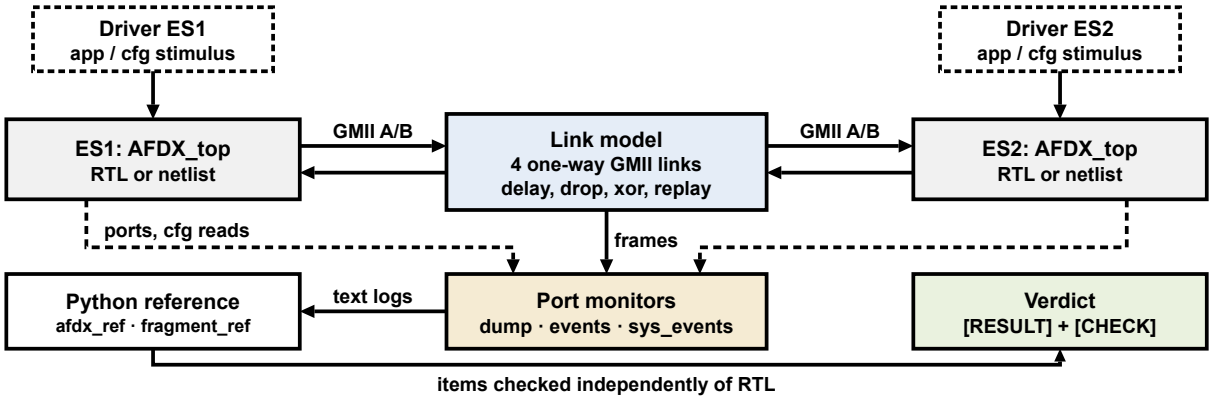


图 10-1　双端双网验证平台：在线断言检查仿真运行，端口记录供独立参考模型逐字节核对。

针对全设计 34 个 RTL 源文件（共 42 个设计单元），回归测试套件包含 67 项自动化测试用例（63 项快速回归 + 4 项长耗时系统参考回归，另含配置写隔离与 IPv4 分片/重组专项验证套件），在 ModelSim SE-64 2020.4 环境下累计通过 2,348,850,504 次自检断言与 4,180,179 项独立参考核对，全部用例判定 PASS。

10.2　性能与多虚拟链路变长/分片压力实测

性能仿真基于理想 125 MHz 时钟（每拍 8 ns，GMII 每拍传输 1 字节），从两个层次完整验证系统性能边界：顶层 **`AFDX_top` 63 VL 满载规格与混合分片实测（1026 答疑 R119 / 1030 答疑 R138-3/R139）**：按本团队声明满载网络规划（双网冗余全双工，单端激活 63 条发送 VL 与 63 条接收 VL，BAG = 1 ms，持续 100 ms，入口驱动脚本 `sim/run_app.do`），覆盖 5 档周期定长（`P1`、`P17`、`P256`、`P1024`、`P1471`）与 2 档突发激励（`B1` 固定顺序对齐突发、`B2` 旋转三元组突发），以及 60 条短帧 VL（1471 B）与 3 条分片长包 VL（1472 B 2 片、3000 B 3 片、8192 B 6 片）并发的混合场景，结果见表 10-1。

核心 **`afdx_es`** 分段时延与超规格极限压力基线（**1022 答疑 R97**）：针对 `afdx_es` 核心口按 $m_{\text{last}} \rightarrow$ SFD $\rightarrow d_{\text{valid}}/d_{\text{last}}$ 分段测量定长 77 VL 高负载（`gbe77`）及四组变长并发场景（`var_fixed`、`var_cycle`、`var_high`、`var_burst`），结果见表 10-2。

实测表明：在声明 63 VL 满载规格下，单向净吞吐达 741.384 Mbps（双向 1482.768 Mbps，超规格 77/81 VL 单向达 906.136/953.208 Mbps），稳态周期最大时延 80.336 µs、旋转突发最大时延 1175.432 µs、抖动 96.752 µs（注：上表 9.992 µs 为核

表 10-1　顶层 AFDX_top 63 VL 声明满载规格与混合分片实测结果（100 ms 双工）

测试场景	载荷配置	双向净吞吐 (Mbps)	最大时延 (µs)	R119 抖动 (µs)	判定与边界说明
P1–P1024	1/17/256/1024 B	1.008–1032.192	34.600–65.984	0.008–3.360	4 档周期定长全通过（0 错包）
P1471（满载）	63 VL × 1471 B	1482.768	80.336	0.016	单向净吞吐 741.384 Mbps（线占率 77.5%）
B1（固定突发）	63 VL × 1471 B	1477.415	1079.056	—*	3,150 对消息 BAG 间隔与载荷完整性全部通过
B2（旋转突发）	63 VL × 1471 B	1477.119	1175.432	96.752	满足抖动 $<$ 100 µs 与时延 $<$ 2 ms 判据
mixed_63vl	60 短 + 3 分片	—	5212.304	—	1472/3000/8192 B 并发，12,206 条双向消息 0 错

注：*B1 按固定 VL 次序连续注入两帧，次帧等待剩余 BAG（$\leq$ 1 ms）验证 BAG 最小间隔约束；跨帧抢占下的 BAG 等待抖动由每三相邻 VL 轮换注入次序的 B2 测量。

表 10-2　核心 afdx_es 多工况分段时延与超规格压力基线（直连 0 拍链路）

测试工况	激活 VL 数	单向吞吐 (Mbps)	L_{tx} 最大 (µs)	L_{rx} 首/末 (µs)	L_{app} 首/末 (µs)	R97 抖动 (µs)
gbe77	77	906.136	0.312	12.360 / 24.120	12.672 / 24.432	0.008
var_fixed	32	57.794	0.360	12.360 / 24.120	12.672 / 24.432	0.008
var_cycle	32	70.403	12.216	12.352 / 24.112	12.936 / 24.424	9.992
var_high	80	904.752	12.016	12.360 / 24.120	23.872 / 24.432	0.008
var_burst	32	57.794	21.128	12.352 / 24.112	32.824 / 32.888	0.000

心 afdx_es R97 口径，顶层 AFDX_top 变长循环 R119 实测为 19.544 µs），全面满足 $\geq$ 100 Mbps 吞吐、$<$ 2 ms 时延与 $<$ 100 µs 抖动指标。

10.3　代码覆盖率签核

对全工程 34 个 RTL 文件（共 42 个设计单元，含顶层流包装器与双 Bank 分片重组子模块）统一开启 ModelSim 代码覆盖率插桩（+cover=sbceft），测试平台不计入统计范围。合并全部 58 份 UCDB 覆盖率数据库后的统计结果如表 10-3 所示。

全设计语句覆盖率达 96.82%，分支覆盖率达 94.61%，状态机状态覆盖率为 83.94%（115/137），关键协议与分片重组通路均得到充分激励验证。

10.4　综合与布局布线后门级网表对照仿真

为验证官方 eLinx 综合与布局布线工具链（AFDX_top.vqm 真实厂商原语门级网表）与 RTL 行为语义完全一致，工程完成了四组门级网表仿真闭环：

表 10-3 34 个 RTL 文件（42 个设计单元）代码覆盖率汇总

覆盖率统计类别	覆盖项数 / 总项数	覆盖率	统计口径与未覆盖项分类说明
语句覆盖率（Statement）	12416 / 12824	96.82%	未覆盖项主要为极端超时保护与防御性异常熔断分支
分支覆盖率（Branch）	7230 / 7642	94.61%	覆盖正常收发、IPv4 分片重组与多类故障注入分支
条件覆盖率（Condition）	1306 / 1766	73.95%	覆盖多信号组合准入判定；未覆盖为互斥极端并发组合
表达式覆盖率（Expression）	3242 / 3987	81.31%	覆盖关键数据通路表达式；未覆盖项为防御性冗余掩码项
状态机状态覆盖率（FSM State）	115 / 137	83.94%	覆盖主干收发、分片重组及管理面状态；未覆盖为防御性保护态
状态机转换覆盖率（FSM Transition）	141 / 375	37.60%	覆盖核心正常与故障恢复跳转；未覆盖项主要为异常交叉跳转
翻转覆盖率（Toggle）	59358 / 82546	71.91%	未翻转主要为静态配置高位、32 位计数器高位及未用地址位

1. 顶层门级烟雾回归（**tb_elinx**）：命令行门级网表回归通过 **9238** 项自检断言（0 错误，耗时 52312.469 s），并在 eLinx IDE 图形界面完成行为级联动核验（耗时 0:02:18）。
2. 五功能综合门级对照（**tb_net_services**）：验证用户流收发、双网冗余去重、SNMP、ARINC 615A 与 SNTP/RTC 联合运行，通过 **1123** 项自检断言与 65 项独立核对（0 错误，耗时 23614.453 s），导出的 7 份外部记录文件与 RTL 逐字节一致。
3. **1030 IPv4** 分片重组门级窗口（**tb_frag1030_net**）：覆盖复位清零及 1472 B 两分片长包双端发送与顺序重组交付（前 199,578 拍门级窗口，耗时 34727.859 s），导出的 5 份记录（32 条记录项）与同周期 RTL 逐字节一致。
4. 高负载多 **VL** 加分片门级窗口（顶层流包装发送与接收双窗口）：覆盖 81 VL 高负载叠加 1472 B 分片发送窗口（耗时 26897.997 s）与接收重组窗口（耗时 27020.430 s），各通过 161 项断言且 11 份导出记录与 RTL 逐字节一致。

10.5 硬件可观察性与可控性机制

设计在硬件架构层提供三项调试与验证机制：

- 确定性事件脉冲端口：顶层提供应用包装丢弃脉冲 ev_app[2:0]（发送未命中 VLID、发送包装溢出/超长、接收包装溢出），核心提供 ev_tx、ev_a、ev_b（含 0x80 重组拒绝合并）、ev_rm_drop、ev_deliver 单拍指示脉冲，支持外部逻辑分析仪与仿真断言精确观测。
- 配置总线全透明读回：27 个硬件统计计数器与三项高层服务的所有内部寄存器均可通过 32 位配置总线在线读回。
- **RAM** 碰撞与存储隔离双重校验：仿真环境下内建同步 RAM 同拍同址读写碰撞实时断言；门级网表下配以 tools/audit_1030_storage.py 静态验证重组 Bank 隔离与 128 个写数据隔离 LUT 完整性。

第 11 章　工程交付与复现指南

11.1　统一的设计方法与架构收敛

本设计以纯 RTL 将带 2 字节 VLID 前缀的应用流包装、AFDX 线速收发、1–8192 B IPv4 分片与 16 路并发重组、完整性检查、冗余管理以及 RTC、SNMP、615A 三项高层服务收敛到同一套硬件架构中，核心工程方法归纳如下：

1. 单时钟同步数据通路与多周期使能域解耦。端系统逻辑统一运行于 125 MHz（8 ns 周期）主时钟，字节流传输、BAG 调度计时和 SNTP 时间戳采样共享同一时钟基准；对于复杂的管理协议状态机，通过局部使能两分频树（`CE_DIV=2`）构建隔拍更新的 16 ns 多周期路径（覆盖 2747 个触发器），在消除异步跨时钟域亚稳态风险的同时实现物理时序收敛。
2. 状态入表、多 **Bank** 分片与写数据隔离。全芯片 250 个同步 RAM 实例（90 个通用 `afdx_ram_sdp` 与 160 个专用同步 RAM 模块，综合后映射为 814 个 M4K/EBR 原语）承载载荷、配置、各 VL 状态、分片进度与重组上下文。128 条发送 VL、4096 条接收 VL 和 16 个重组槽位均按地址检索，大容量缓冲在物理入口配备显式写数据隔离 LUT（`lut_mask="8888"`），使组合逻辑深度与链路规模解耦。
3. 读写端口独占与无冲突时序仲裁。每张表的读写端口归属唯一：收发引擎独占状态表读写；配置总线对配置表的写入通过许可信号 `cfg_ok` 插入引擎空闲拍，从结构上消除同拍同址读写冲突；交付队列迁入块 RAM 并扩展分片槽位索引，消除宽位多路选择树。
4. 全流水寄存切分与精细时延补偿。所有 RAM 读出数据先经本地寄存器打拍再参与算术或比较；发送侧组帧器采用 12 拍 IPv4 校验和准备与 −8 拍提前量（`START_LEAD_REGISTERED = -8`）精确补偿内部读取与输出流缓冲延迟，保证到期首字节准点送出；RTC 对 5 拍写回流水精确补偿 40 ns，保障时钟同步精度。

11.2　工程交付清单与 34 个 RTL 源文件

工程完整源码交付于仓库 `rtl/` 目录，共包含 34 个 Verilog 文件（合计 15,856 行，封装 42 个设计单元），职责划分见表 11-1。

表 11-1　交付的 34 个 Verilog RTL 源码清单（共 42 个设计单元）

源文件名	所属子系统	模块核心功能与职责
`AFDX_top.v`	顶层封装	172 引脚赛题模板顶层，集成应用流包装器、配置窥探与 `afdx_es`
`afdx_es.v`	核心顶层	端系统核心顶层，整合双网收发核心、服务仲裁、三项服务与配置时基
`afdx_app_tx.v`	应用包装层	发送 2B VLID 前缀解析、3 级 128 路比较映射、32 KiB 缓存与超长回滚

接下页

表 11-1 （续）

源文件名	所属子系统	模块核心功能与职责
afdx_app_rx.v	应用包装层	接收消息缓存、idx → VLID 反查、2B VLID 前缀封装与溢出保护
afdx_app_map.v	应用包装层	4096 × 16 bit 接收 VL 索引至 16 位 VLID 专用同步映射 RAM
afdx_app_payload.v	应用包装层	8 × 4 KiB 分体包装载荷存储（含 _bank/_slice 共 3 个单元，写隔离）
afdx_app_read.v	应用包装层	8 信用流式同步预取队列，无缝桥接 4 拍块 RAM 读延迟
afdx_timebase.v	时间基准	维护 32 位 now 周期时基、16 位 now_c 冗余粗计时与 byte_en
afdx_cfg.v	配置与统计	18 位字地址配置总线译码、27 个统计计数器两级流水与 cfg_ok 写握手
afdx_svc.v	服务通道	4 路整消息轮询发送仲裁（授权寄存输出）与 UDP 目的端口接收分发
afdx_tx_core.v	发送核心	发送核心集成与 131072 × 8 载荷池 afdx_tx_payload_store（共 2 个单元）
afdx_tx_buf.v	发送核心	1–8192 B 分页写入、两槽入口弹性级与 16 × 8 回滚 RAM（共 3 个单元）
afdx_tx_free.v	发送核心	256 页空闲页池 FIFO，支持双页面回收端口并发归还
afdx_tx_engine.v	发送核心	128 条 VL 的 BAG 调度、单描述符按片授权与 3 张分片状态表管理
afdx_tx_builder.v	发送核心	42B/34B 帧头组装、12 拍 IPv4 校验和准备、载荷预读与 −8 拍提前补偿
afdx_mac_tx.v	发送核心	纯 RTL 自研双网锁步 MAC，前导码、FCS、IFG 与下溢检测
afdx_crc32_d8.v	通用算术	8 位并行以太网 CRC-32 组合逻辑生成与校验器
afdx_rx_core.v	接收核心	接收核心集成、32-Bank 双写分片 rx_map 与 afdx_rx_table_slice（2 单元）
afdx_mac_rx.v	接收核心	接收 MAC，SFD 定界、CRC-32 流式累加校验与 FCS 剥离
afdx_rx_parse.v	接收核心	单帧/首片（42B）与后续分片（34B）帧头检查、环形预写与帧尾分类
afdx_rx_engine.v	接收核心	30 拍 FRAME 日程：VL 查表、IC、RM、重组槽位查询预约与老化清理
afdx_rx_deliver.v	接收核心	读取 32 × 73 bit rx_dq，自双网环形缓冲逐字节流式送往重组/直通
afdx_rx_reassembly.v	分片重组	双 Bank ×8 槽位（16 上下文）顺序 IPv4 分片重组顶层与输出仲裁
afdx_rx_reassembly_bank.v	分片重组	单 Bank 8 槽位重组控制、连续偏移检查、整包非零 UDP 校验与 2 s 超时
afdx_rx_reassembly_payload.v	分片重组	重组载荷存储池（含 _payload/_slot/_lane 3 单元及写隔离 LUT）
afdx_rx_reassembly_ram.v	分片重组	重组元数据与接收窄分片表复用的专用同步双端口 RAM 封装

接下页

表 11-1 （续）

源文件名	所属子系统	模块核心功能与职责
afdx_ram_sdp.v	存储原语	通用简单双端口（SDP）同步 RAM 封装（全芯片共例化 90 处）
afdx_rtc.v	高层服务	64 位实时时钟、SNTPv4 纳秒级同步与 40 ns 流水补偿状态机
afdx_ntp_fraction.v	高层服务	本地纳秒时基与 NTP 32 位二进制小数 32 轮逐位迭代换算器
afdx_snmp.v	高层服务	SNMP 顶层包装、UDP 请求捕获与 CE_DIV=2 使能复制树
afdx_snmp_engine.v	高层服务	SNMPv1/v2c ASN.1/BER 编解码、36 项 MIB 读写与 Trap 生成（16 ns 域）
afdx_615a.v	高层服务	615A 顶层包装、TFTP 捕获、6 块协议 RAM 接口与 CE_DIV=2 使能树
afdx_615a_engine.v	高层服务	ARINC 615A 数据加载/介质卸载协议状态机与 CRC-16 硬件校验（16 ns 域）
afdx_mgmt_tx.v	高层服务	管理面 8 字节流式预取发送适配器（桥接 16 ns 引擎域与 125 MHz 仲裁域）

11.3 仿真测试复现指南

全套验证用例均可由纯脚本自动化一键复现，环境要求为 ModelSim SE-64 2020.4 或兼容版本（加上 Python 3 用于参考模型交叉校验）：

1. 顶层 **AFDX_top 63 VL** 满载规格与应用流验证：在 ModelSim 或命令行中调用顶层应用流运行脚本（可通过 VL_COUNT、MODE、PAYLOAD_BYTES、DUPLEX 切换 7 档场景）：

```
vsim -c -do "set VL_COUNT 63; set DUPLEX 1; set MODE 0; set PAYLOAD_BYTES
    1471; do sim/run_app.do"
```

2. **1030 IPv4** 分片重组专项与签核自检：一键运行 1472–8192 B 分片边界、16 并发上下文及故障注入专项，以及 1030 交付一致性核验：

```
powershell -NoProfile -ExecutionPolicy Bypass -File sim/run_fragmentation.
    ps1
python3 tools/run_1030_checks.py
```

3. 全量自动化回归测试（**63** 项快速回归 + **4** 项长耗时系统参考 = **67** 项）：在 PowerShell 下执行回归套件（附加 -IncludeLong 包含长耗时系统性能用例，附加 -Coverage 生成 58 份 UCDB 覆盖率库）：

```
powershell -NoProfile -ExecutionPolicy Bypass -File sim/regress.ps1 -
    IncludeLong
```

4. **8** 项核心评审用例无 **Python** 独立入口：在无外部脚本环境的纯净机器上，可在 ModelSim 批处理模式下执行 vsim -batch -do "do sim/run_judge.do; quit -f"（或 GUI 内 do sim/run_judge.do）连续运行 8 项系统级评审测试平台（实测 **8/8**

PASS，合计约 2.3 h 累计通过 1,949,513,089 次逐拍/逐字节在线断言检查、0 错误，完整输出归档于 `sim/reference/judge_result_frag1030.txt`）。

11.4 eLinx 物理实现与时序签核四步闭环

工程提供基于中科亿海微官方 eLinx 3.0.7 工具链的确定性命令行构建与独立审计脚本，支持全流程一键物理签核闭环：

1. 步骤一：确定性综合、物理簇锚定与布局布线（**`PRES 1.30`**）

```
python3 tools/build_1030.py --route --pres 1.30
```

自动加载 `elinx/constraints.edc`（8.000 ns 主时钟 + 0.30 ns 建立不确定度 + 253 条多周期约束）与 `elinx/tiles_1030.edc`（锁定 11044 个物理簇：10230 个 CLB + 814 个 M4K），完成综合、打包、布局布线并导出门级网表 `AFDX_top.vqm` 与物理时序库（完整构建耗时约 898.5 s）。

2. 步骤二：分片重组存储分体与写隔离 **LUT** 物理网表审计

```
python3 tools/audit_1030_storage.py <VQM_PATH>
```

静态解析导出的门级网表，确认重组元数据保持双 Bank 独立映射，且 64 条载荷通道的 256 个 M4K 数据引脚全部由 128 个显式 `lut_mask="8888"` 写数据隔离 LUT 驱动。

3. 步骤三：多周期路径约束合法性全量审计

```
python3 tools/check_multicycle.py <VQM_PATH> --constraints elinx/constraints
    .edc --output out/multicycle_audit
```

逐一核对全芯片 2747 个管理面引擎触发器的时钟使能网络与多周期声明，确认 0 非法单周期穿透、0 漏约束端点。

4. 步骤四：布线后静态时序全量导出与签核

```
python3 tools/report_routed_timing.py
```

解析布局布线后的 `full_timing.tan.pb` 与 `slack.rpt`，核验 269,131 条 8 ns 单周期路径（净建立裕量 +0.029 ns，物理裸裕量 +0.329 ns）、50,050 条 16 ns 多周期路径（最差建立裕量 +0.720 ns）及 319,181 条保持时间路径（最差裕量 +0.604 ns），输出 96142 个时序端点 0 违例的签核报告。

附录 A　源文件与默认参数

A.1　源文件

RTL 共 34 个 Verilog 文件（合计 15,856 行，42 个设计单元），均位于仓库 `rtl/` 目录。

表 A-1　RTL 源文件清单（34 个文件）

文件	作用
`AFDX_top.v`	赛题模板顶层封装：2B VLID 流式包装器、配置窥探与 `afdx_es` 核心
`afdx_es.v`	端系统核心顶层：双网收发核心、服务通道、三项服务、配置与时间基准
`afdx_app_tx.v`	顶层发送流包装器：2B VLID 前缀解析、3 级 128 路比较映射与 32 KiB 缓存
`afdx_app_rx.v`	顶层接收流包装器：消息缓存、VLID 反查、2B VLID 前缀插入与溢出保护
`afdx_app_map.v`	接收包装层 4096×16 bit idx $\rightarrow$ VLID 专用同步映射 RAM
`afdx_app_payload.v`	包装层 8×4 KiB 分体同步载荷存储（含 `_bank`/`_slice` 及写隔离）
`afdx_app_read.v`	包装层 8 信用流式同步预取队列（屏蔽 4 拍块 RAM 读延迟）
`afdx_timebase.v`	周期计数 `now`、粗粒度计时 `now_c` 与字节使能 `byte_en`
`afdx_cfg.v`	18 位字地址译码、寄存器、27 个计数器两级流水维护、配置表写握手
`afdx_svc.v`	应用与服务的整消息轮询发送（授权寄存）、按 UDP 目的端口分发接收
`afdx_tx_core.v`	发送核心集成模块，例化发送各子模块、输出流缓冲及 `afdx_tx_payload_store`
`afdx_tx_buf.v`	1–8192 B 分页写入、两槽入口弹性级与 16×8 回滚 RAM
`afdx_tx_free.v`	256 页空闲页 FIFO，支持双页面回收端口并发回收
`afdx_tx_engine.v`	BAG 与分片调度：扫描、入队、单帧/分片授权、分片进度表维护与起始确认
`afdx_tx_builder.v`	42B/34B 组帧、12 拍 IPv4 校验和准备、载荷预读与 -8 拍提前补偿
`afdx_mac_tx.v`	双网发送 MAC：前导码、FCS、帧间隔、发送下溢（TX Underrun）检测与异常终止
`afdx_crc32_d8.v`	逐字节 8 位并行 CRC-32 组合逻辑
`afdx_rx_core.v`	接收核心集成模块，例化双 MAC、解析器、32-Bank 双写分片映射表、重组与交付
`afdx_mac_rx.v`	接收 MAC：SFD 检测、CRC 校验、剥离 FCS
`afdx_rx_parse.v`	单帧/首片与后续分片帧头检查、载荷预写入环形缓冲、帧尾分类
`afdx_rx_engine.v`	30 拍日程：32-Bank 查表、配置检查、IC、RM、重组槽位查询预约与过期清理

接下页

表 A-1 （续）

文件	作用
afdx_rx_deliver.v	读取 32 × 73 bit rx_dq，从双网环形缓冲逐字节流式送出
afdx_rx_reassembly.v	双 Bank ×8 槽位（16 上下文）顺序 IPv4 分片重组顶层
afdx_rx_reassembly_bank.v	单 Bank 8 槽位重组状态机、连续偏移检查、整包 UDP 校验与 2 s 超时回收
afdx_rx_reassembly_payload.v	16 × 8 KiB 重组载荷存储池（64 条 8192 × 2 通道与 128 个写隔离 LUT）
afdx_rx_reassembly_ram.v	分片重组元数据与接收窄分片表复用的专用同步双端口 RAM 封装
afdx_ram_sdp.v	通用简单双端口（SDP）同步 RAM 原语封装（例化 90 处）
afdx_rtc.v	实时时钟与 SNTPv4 纳秒级时钟同步（含 40 ns 流水补偿）
afdx_ntp_fraction.v	纳秒与 NTP 32 位二进制小数的 32 轮逐位换算
afdx_snmp.v	SNMP 顶层包装：请求捕获、协议 RAM 接口与 CE_DIV=2 使能树
afdx_snmp_engine.v	SNMP 协议核心：ASN.1/BER 编解码、MIB 读写与响应生成（16 ns 域）
afdx_615a.v	615A 顶层包装：TFTP 捕获、6 块协议 RAM 接口与 CE_DIV=2 使能树
afdx_615a_engine.v	615A 协议核心：数据加载/卸载状态机与文件 CRC-16 硬件校验（16 ns 域）
afdx_mgmt_tx.v	管理面流式预取发送适配器：8 字节预取队列，供 SNMP 与 615A 复用解耦

A.2 默认参数

表 A-2 顶层与核心参数及默认容量

参数	默认值	含义
TX_PAGE_AW / TX_OFF_AW	8 / 9	发送缓存 256 页 ×512 B，共 128 KiB
LOOKAHEAD	512	提前选择窗口，512 拍（4.096 µs）
RX_RING_AW / RX_DQ_AW	12 / 5	每网接收环形缓冲 4 KiB，交付队列 32 项（73 位宽）
RX_CONFIG_BANKS	16	接收配置总线与描述符局部复制扇出分支数
CE_DIV	2	SNMP / 615A 协议引擎多周期时钟使能分频比（16 ns 域）
REASSEMBLY_CONTEXTS	16	接收 IPv4 顺序分片重组并发槽位数（2 Bank ×8 槽 ×8 KiB）
REASSEMBLY_TIMEOUT	2.5×10^8 拍	分片重组超时老化阈值（2.0 s）
单条应用消息长度	1–8192 B	> 1471 B（或 > max_payload）自动按 IPv4 分片传输
顶层应用包装缓存	2 × 32 KiB	afdx_app_tx 与 afdx_app_rx 各 32 KiB + 32 项描述符
发送 / 接收 VL 容量	128 / 4096	接收映射表 65536 项；声明满载规划为 63 条并发收发 VL
帧间隔（IFG）	12 拍	96 ns

接下页

表 A-2 （续）

参数	默认值	含义
上电表清零初始化	65,539 拍	核心 `init_done`（524.312 µs）；顶层 `tx_ready` 约 65,550 拍
SNMP 请求缓存	256 B	单 VarBind，支持 36 个 MIB 标量/表格节点
615A 加载区	8 KiB	头文件区 2 KiB，LUR 区 512 B，超时 2 s / 重传 1 次
RTC 同步周期	125000 拍	1 ms，可经寄存器修改

附录 B　服务寄存器

基本寄存器、计数器和配置表地址见表 8-1，服务寄存器见表 B-1。RW 为可读写，RO 为只读。

表 B-1　服务寄存器

地址	名称	读写	含义
RTC			
0x80	CTRL	RW	位 0 启用，位 1 Server 模式；写位 2 装入暂存时间
0x81	PERIOD	RW	Client 端请求间隔（拍），复位 125000
0x82	TX_VL	RW	发送 VL，低 7 位
0x83	UDP_PORT	RW	UDP 端口，复位 123
0x84	SEC	RW	写秒暂存值；读当前秒并锁存纳秒
0x85	NS	RW	写纳秒暂存值；读锁存的纳秒值
0x86	OFFSET_SEC	RO	最近偏差的有符号秒部分
0x87	OFFSET_NS	RO	最近偏差的纳秒余数
0x88	SYNC_COUNT	RO	Client 时钟同步次数
SNMP			
0x90	ENABLE	RW	位 0 启用代理
0x91	TX_VL	RW	响应与 Trap 的发送 VL
0x92	COMMUNITY_LENGTH	RW	1–16，复位 6
0x93–0x96	COMMUNITY_0–3	RW	community 内容，复位为 public
0x97	SAMPLE_CYCLE	RO	上次 MIB 读取时的 now
0x98	UPTIME	RO	sysUpTime，单位 10 ms
0x99	REQUESTS	RO	端口匹配的请求数
0x9A	BAD_COMMUNITY	RO	community 不符次数
0x9B	TRAP_ENABLE	RW	位 0 启用 Trap
0x9C	TRAPS_SENT	RO	已发送 Trap 数
0xA0–0xA3	DESCRIPTION_0–3	RO	sysDescr 字符串
615A			
0xB0	ENABLE	RW	位 0 启用
0xB1	TX_VL	RW	发送 VL
0xB2	STATUS	RO	位 1:0 空闲/进行中/成功/失败；位 15:8 原因码
0xB3	LOADED_BYTES	RO	本次已写字节数
0xB4	MEMORY_ADDRESS	RW	加载区读回地址，低 13 位
0xB5	MEMORY_DATA	RO	加载区读回字节
0xB6	LUS_COUNTER	RO	本次状态文件数
0xB7	RETRY_TOTAL	RO	累计超时重传次数
0xB8	REQUEST_PORT	RW	TFTP 请求端口，复位 59
0xB9	TIMEOUT_CYCLES	RW	超时拍数，复位 250000000
0xBA	RETRY_LIMIT	RW	重传上限，复位 1
0xBB	PROTOCOL_VERSION	RW	A4 或 A3

附录 C　内部接口信号

本附录按信号说明第 4.7 节汇总的内部接口。

C.1　消息接口

服务通道 afdx_svc 把选中的消息以字节流交给发送核心，见表 C-1。没有服务报文时，这组信号与应用口组合直通。

表 C-1　消息接口（afdx_svc → afdx_tx_buf）

信号	位宽	含义
m_valid、m_ready	1、1	字节握手；m_ready 只在 tx_buf 处于 IDLE、RECV、DRAIN 时为 1
m_data、m_last	8、1	消息字节、末字节标记
m_vl	7	发送 VL 索引，首字节锁存
m_port_en、m_sport、m_dport	1、16、16	m_port_en 为 1 时使用消息自带的 UDP 端口；首字节锁存

C.2　入队与授权

afdx_tx_buf 写完消息后，将 47 位描述符写入 tx_desc，再向引擎发出入队请求。引擎选中一条消息（或其中一片 IPv4 分片）后，将单帧/分片字段一次性交给组帧器；组帧器在帧开始发送时报告起始时刻，并在分片载荷读毕时回传下一片游标，见表 C-2。

表 C-2　入队、分片授权和发送起始指示接口

信号	位宽	含义
入队（*tx_buf → tx_engine*，请求应答）		
enq_valid	1	保持有效，直至收到 enq_ack
enq_vl、enq_head、enq_len	7、8、14	发送 VL 索引、消息首页号、整包字节数（1–8192）
enq_ack、enq_result	1、3	单拍应答；0 接纳，1 VL 未启用，2 队列满，3 超长（4 无空闲页由 tx_buf 自行判定）
tx_desc[46:0]	47	{len[13:0], port_en, sport[15:0], dport[15:0]}，以首页号为地址

接下页

表 C-2 （续）

信号	位宽	含义
授权（tx_engine → tx_builder，单深缓冲握手）		
grant_valid、grant_ready	1、1	grant_ready = ¬pending，即单帧待发槽位空闲
grant_a、grant_b	1、1	两网启用位（配置字 0 的 [1]、[2]）
grant_timed、grant_time	1、32	grant_timed 为 1 时，帧不得早于 grant_time 起始；为 0 时可立即发送
grant_vlid	16	配置字 2
grant_src、grant_dst	32、32	配置字 3、4
grant_sport、grant_dport	16、16	port_en 为 1 取消息自带端口，否则取配置字 5
grant_len、grant_sn、grant_page	11、8、8	本帧载荷字节数、AFDX 帧序号、本片起始页号
grant_head、grant_offset	8、9	消息首页号（首片保留至末片释放）、本片页内起始字节偏移
grant_ip_offset、grant_message_len	13、14	IPv4 8B 单位片偏移、整条应用消息总字节数
grant_ident、grant_first、grant_more	16、1、1	IPv4 包标识、首片标志（含 8B UDP 头）、MF 更多分片标志
发送起始与分片游标回传（tx_builder → tx_engine）		
start_valid、t_start	1、32	单拍脉冲；t_start 为输出首个前导字节时的 now
fragment_done、_vl、_page、_offset	1、7、8、9	非末片载荷读毕时回传下一待发分片的起始页号与页内偏移

C.3 组帧器与 MAC

组帧器将目的 MAC 地址至 SN 字段的帧内容以字节流送至 MAC，由 MAC 控制字节接收时刻，见表 C-3。

表 C-3 帧字节接口（tx_builder ↔ mac_tx）

信号	位宽	含义
s_valid、s_ready	1、1	字节握手；s_ready 在 byte_en= 1 且满足以下任一条件时为 1：前导码末拍；DATA 且 last_byte= 0；DRAIN
s_data_a、s_data_b	8、8	两网字节，仅第 11 字节不同
s_last	1	与 SN 字节同拍
s_en_a、s_en_b	1、1	起始拍锁存，选择本帧发往哪一网
tx_sof、tx_eof	1、1	首个前导字节、帧尾（FCS 发完）的单拍脉冲
tx_underrun	1	数据阶段上游未备好字节的下溢（Underrun）指示

C.4 帧描述与交付队列

每网解析器为正确帧生成一份 221 位帧描述（兼容不分片单帧与 IPv4 分片帧）。引擎读取描述并完成 30 拍判定与重组槽位查询后，向解析器发出提交或回退指示；对于通过判定的帧，引擎向 32 × 73 bit 交付队列写入一项，见表 C-4。

表 C-4 帧描述与交付队列项

字段	位宽	含义
帧描述（rx_parse → rx_engine，单深缓冲握手，共 221 位，高位在前）		
ip_src、ip_dst、ip_id	32、32、16	IPv4 源/目的地址与 16 位分片标识（用于分片重组五元组与伪头）
frag_offset、mf	13、1	IPv4 8B 单位片偏移与更多分片标志（均为 0 表示不分片单帧）
udp_len、udp_check	16、16	首片/单帧的 UDP 总长与校验和字段
vlid、sn	16、8	目的 MAC 第 4–5 字节 VLID 与帧末 AFDX SN
sport、dport	16、16	首片/单帧提取的 UDP 源端口与目的端口
n、start、arrival	11、12、16	本帧载荷字节数、环形缓冲起点与帧尾时刻的 now_c
desc_valid、desc_take	1、1	交接握手（按配置扇出分支复制）
resolve_valid、resolve_commit	1、1	引擎判定：提交则 committed 前进到本帧末，否则 trial 回退
交付队列项（rx_engine → rx_deliver，32 项 × 73 bit 块 RAM rx_dq，高位在前）		
frag[72]、slot[71:68]	1、4	分片标志（0 单帧直通，1 分片重组）与预约的重组槽位号（{bank,row}）
net[67]	1	0 为 A 网，1 为 B 网
start[66:55]、n[54:44]	12、11	环形缓冲起点和本帧载荷字节数
idx[43:32]	12	接收 VL 索引
sport[31:16]、dport[15:0]	16、16	UDP 源端口与目的端口（共 73 位）

核心交付口（重组模块输出 d_*）的字段与 afdx_es 应用接收口对应（目的端口在内部记为 d_dport，在核心记为 app_rx_port）。服务通道将这些信号同时广播给应用和三个服务。各服务在首字节到达当拍，根据 UDP 目的端口产生组合匹配信号 svc_rx_claim；匹配结果寄存并保持至末字节。端口匹配的消息由对应服务接收；其余应用消息在顶层 AFDX_top 中经 afdx_app_rx 加上 2 字节大端 VLID 前缀后自 rx_data/rx_valid 输出。

C.5 服务发送总线

三个服务的发送口打包成一组总线接到 afdx_svc，按 RTC、SNMP、615A 从低位到高位拼接，见表 C-5。各服务按相同的字节流握手约定驱动对应位段。

表 C-5 服务发送总线 svc_tx_*

信号	位宽	拼接
svc_tx_valid、_ready、_last、_port_en	各 3 × 1	[0] RTC，[1] SNMP，[2] 615A

接下页

表 C-5 （续）

信号	位宽	拼接
`svc_tx_data`	3×8	[7:0] RTC，[15:8] SNMP，[23:16] 615A
`svc_tx_vl`	3×7	[6:0]、[13:7]、[20:14]
`svc_tx_sport`、`svc_tx_dport`	各 3×16	[15:0]、[31:16]、[47:32]

附录 D　术语与参考文献

D.1　术语

表 D-1　本文专用术语

术语	含义
拍	一个 125 MHz 时钟周期，8 ns
PSN	同一网上同一 VL 上一帧的序列号
预写入 / 提交 / 回退	载荷先写入环形缓冲；通过检查后更新提交写指针，拒绝时回退预写入指针
候选	调度引擎选出、等待授权给组帧器的 VL
`pend_vl`	已授权但尚未开始发送的 VL
`cfg_ok`	引擎在本拍不发出配置表读的指示
有效副本	帧格式、FCS 与协议头检查通过，且满足接收配置和已启用的完整性检查的单网帧副本
页面释放	将页号送回空闲页池（经页面回收口 `ret1_*` 或 `ret2_*`）
端口匹配	服务根据 UDP 目的端口确定是否接收当前消息（信号 `svc_rx_claim`）
表初始化	复位后逐项清零收发配置表与状态表（`rx_map`、`rx_cfg`、`rx_state`、`tx_cfg`、`tx_state`）的过程
线路占用	一帧在线路上的占用时长 $C(n)$，单位为拍
Server / Client	SNTP 角色：Server 端应答请求并回显时间戳；Client 端周期发送请求并执行时钟同步校准
应答	请求应答式握手的确认脉冲（`ack`），用于确认接口请求，与协议响应报文不同
计时单位	`now_c` 的单位，32.768 μs
发送抖动	帧起始相对 BAG 允许时刻的偏差
起始指示 / 起始确认	组帧器在 `tx_sof` 有效拍经 `start_valid`/`t_start` 发送起始指示（TX Start Indication）；发送引擎据此完成发送起始确认（TX Start Confirm）并写回更新后的 `next_time`

D.2　参考文献

[1] ARINC Specification 664 Part 7: Avionics Full Duplex Switched Ethernet Network, 2005-06-27.

[2] ARINC Report 615A-2: Software Data Loader Using Ethernet Interface, 2002-05-10.

[3] ARINC Report 665-1: Loadable Software Standards, 2001-01-12.

[4] RFC 1350: The TFTP Protocol (Revision 2), 1992.

[5] RFC 2347: TFTP Option Extension, 1998.

[6] RFC 1157: A Simple Network Management Protocol (SNMP), 1990.

[7] RFC 3416: Version 2 of the Protocol Operations for the Simple Network Management Protocol (SNMP), 2002.

[8] RFC 3418: Management Information Base (MIB) for the Simple Network Management Protocol (SNMP), 2002.

[9] RFC 4330: Simple Network Time Protocol (SNTP) Version 4 for IPv4, IPv6 and OSI, 2006.

[10] IEEE Std 802.3: IEEE Standard for Ethernet, 2018.

[11] 中科亿海微. 全国大学生嵌入式芯片与系统设计竞赛 FPGA 创新设计赛道选题指南.